



DEPARTMENT OF MECHANICAL ENGINEERING

Technical University of Munich (TUM)

Bachelor's Thesis in Mechanical Engineering

Enabling Long-Term Autonomy in Outdoor Mobile Robots through Autonomous Docking and Resource Management

Author:	David Metzler
Supervisor:	Lennart Trösken
Advisor:	Lennart Trösken
Submission Date:	23.06.2026



I confirm that this bachelor's thesis in mechanical engineering is my own work and I have documented all sources and material used.

Munich, 23.06.2026

David Metzler

Acknowledgments

I would like to thank my supervisor, Lennart Trösken, for his guidance, his readiness to discuss ideas throughout this project, and his detailed feedback on both the system design and the manuscript. I am grateful to the Munich Institute of Robotics and Machine Intelligence (MIRMI) at the TUM for providing the robotic platform, the workspace, and the resources that made the experimental work possible, and to everyone there who helped with the hardware and the outdoor test campaign.

Finally, I thank my family and friends for their support and encouragement throughout my studies.

Abstract

Persistent, long-term autonomous cultivation—rather than the automation of a single bounded task—is the unmet promise of agricultural robotics, and it is the enabling technology that could make labour-intensive agroforestry economically viable. Its precondition is autonomous resource replenishment: however capable a robot’s perception or planning, it cannot remain deployed unless it can return to a station and recharge without human intervention. This thesis isolates that precondition, autonomous docking, and addresses it through two coupled questions—how to structure a control architecture for resource-aware long-term autonomy with docking embedded in it, and how the available sensing modalities and detection algorithms compare for the docking maneuver itself.

For the first, the thesis derives a Robot Operating System 2 (ROS 2)-based hybrid (deliberative–reactive) three-layer architecture in which a Behaviour-Tree executive links a resource-aware deliberative layer to bounded-latency reactive control. Its real-time on-robot Sense–Plan–Act loop is decoupled through shared databases from an asynchronous cloud agronomic simulation, and docking is realised not as a standalone behaviour but as an executive state entered whenever an energy-range prediction falls below a safety margin.

For the second, three station detectors—a hand-crafted AprilTag fiducial, a marker-free learned-keypoint (You Only Look Once (YOLO)v8-pose) detector, and a geometric Light Detection and Ranging (LiDAR) pipeline—are compared on identical poses against exact simulation ground truth and corroborated on physical hardware. No single detector dominates: their failure modes separate cleanly by range. The AprilTag is one to two orders of magnitude more precise (0.81 px, ≈ 4.6 cm) but short-range (~ 5 m) and weather-brittle; the learned detector holds 100 % detection to 30 m through rain, snow and fog but is an order of magnitude coarser; and the LiDAR supplies illumination-independent position to ~ 3 m but cannot resolve terminal orientation under a partial view. On the edge platform all three run in real time, so cost dictates how each is run rather than whether. The result is a range-layered perception strategy—learned vision for long-range capture, the AprilTag for sub-centimetre terminal alignment, the LiDAR as a geometric cross-check—which a closed-loop simulation campaign confirms end to end, docking 9 of 10 runs inside the ≤ 5 cm / 5° tolerance.

Together, the architectural and empirical contributions take one concrete step from the prevailing mission-based paradigm toward the persistent field autonomy that long-term cultivation demands.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
1.1. Motivation	1
1.2. Problem Statement and Scope of the Thesis	2
1.2.1. Goals and Scope	3
1.3. Structure of the Thesis	4
2. Fundamentals and State of the Art	6
2.1. Theoretical Requirements for LTA	7
2.1.1. What is autonomy?	7
2.1.2. What is long-term autonomy?	8
2.1.3. What does a robot need for long-term autonomy?	10
2.2. Analysis of Autonomy Architectures	13
2.2.1. Dimensions of Robotic Architectures	13
2.2.2. Deliberative Architectures	16
2.2.3. Reactive Architectures	18
2.2.4. Hybrid (Three-Layer) Architectures	20
2.2.5. Learning Architectures	22
2.2.6. Digital Twins	24
2.3. Comparative Analysis of Sensing Methods	28
2.3.1. Foundational Principles of Relative Localization	28
2.3.2. Localization Topologies	35
2.3.3. Sensor Modalities	38
2.3.4. Synthesis Evaluation Matrix	43
2.4. Solutions for Resource Management in Robotics	45
2.4.1. Robots Carrying All Resources On Board	45
2.4.2. Autonomously Docking Robots	46
2.5. Docking Triggering and Interface Technologies	47
2.5.1. Resource Depletion Prediction	47
2.5.2. Interface Technologies	48
3. Conceptual Analysis and System Design	50
3.1. System Requirements	50
3.1.1. Functional Requirements	50

3.1.2.	Non-Functional Requirements	51
3.2.	Derivation of the Proposed Architecture	52
3.2.1.	A ROS 2-Based Hybrid Approach	52
3.2.2.	System Overview: A Database-Decoupled Sense–Plan–Act Loop	52
3.2.3.	Cloud Decoupling and the Agronomic Simulation	53
3.3.	Concept Selection for the Prototype	56
3.3.1.	Selection of the Energy Transfer Interface	57
3.3.2.	Selection of the Data Transfer Interface	58
3.3.3.	Selection of Candidate Sensors for Evaluation	59
3.4.	The Docking Pipeline: A Sense–Plan–Act Realisation	63
3.4.1.	Sense: Perception and State Estimation	63
3.4.2.	Plan: Navigation, Energy Trigger, and Mission Sequencing	67
3.4.3.	Act: Actuation and the Three-Phase Docking Sequence	68
4.	Experimental Setup	70
4.1.	General Setup	70
4.2.	Performance Metrics	71
4.2.1.	Test Scenarios	72
4.3.	Selection of Experimental Parameters	74
4.3.1.	Physical and Geometric Parameters	74
4.3.2.	Environmental Parameters	74
4.4.	Real-World Experimental Setup	74
4.4.1.	Mobile Test Platform and Sensor Configuration	76
4.4.2.	On-Board Process Orchestration and Detection Pipelines	78
4.4.3.	Outdoor Field Campaigns	79
4.5.	Simulation-Based Experimental Setup	79
4.5.1.	Simulation Environment and Headless Rendering	79
4.5.2.	Simulation Software Architecture	81
4.5.3.	Simulation Parameter Sweeps	82
4.5.4.	Learned-Keypoint YOLOv8 Detector and Training Data	82
4.5.5.	Weather Augmentation: Decoupled 3D Gaussian Splatting	82
4.5.6.	Autonomous Docking Behavior Tree	83
4.6.	Graphical User Interface	83
5.	Evaluation and Results	85
5.1.	Geometric Detection Envelope: Fiducial versus Learned Vision	86
5.1.1.	Maximum Detection Distance	86
5.1.2.	Azimuth Coverage	86
5.1.3.	Levers on the Fiducial Range: Resolution and Tag Size	89
5.1.4.	The Sub-Pixel Detection Limit: Lateral Alignment Micro-Sweep	89
5.2.	Robustness to Adverse Weather	92
5.2.1.	LiDAR under Adverse Weather: Evidence from the Literature	93
5.3.	The Cost of Learning: Training-Data Ablation	95

5.4. LiDAR Geometric Detection and its Failure Modes	96
5.4.1. Detection Range	97
5.4.2. Angular Detection Envelope	99
5.4.3. Anatomy of the Detection Pipeline	101
5.4.4. Two Terminal Failure Modes	101
5.4.5. Implications for the LiDAR Detector	103
5.5. Closed-Loop End-to-End Docking Performance	104
5.5.1. Analysis of Performance and Failure Mechanisms	105
5.6. Real-World Validation: The Sim-to-Real Counter-Check	108
5.6.1. Learned Vision on Real Captures	109
5.6.2. LiDAR on Real Point Clouds	109
5.6.3. Outdoor Static Multi-Sensor Accuracy Sweep	109
5.7. Computational Cost on the Edge Platform	112
5.7.1. Baseline Cost of the Deployed Configurations	114
5.7.2. What Drives the Cost	114
5.8. Synthesis: A Range-Layered Docking Perception Strategy	116
6. Conclusion	118
6.1. Summary of Contributions	118
6.2. Discussion of Limitations	120
6.3. Conclusion and Future Work	121
A. Appendix	123
A.1. Bill of Materials (BOM)	123
A.2. Electrical Schematics and Wiring Diagrams	123
A.3. CAD Models of Custom Components	125
A.4. Manufacturing Drawing of the Docking Station	125
A.5. Data Model (Entity–Relationship Schemas)	127
A.6. Component Specifications and Datasheets	131
A.7. Foxglove GUI Layout Configurations	134
A.8. Source Code	134
A.9. Documentation	134
List of Figures	135
List of Tables	138
Glossary	140
Acronyms	142
Bibliography	144

Strukturhinweise und offene Punkte

FEEDBACK (supervisor), PARTIALLY DONE: (a) “Robot” label moved to the left of the point — DONE. (b) Still open, supervisor shorthand needs clarification: on the phase-offset sub-figure “add the speed-of-light annotation (c), add the missing symbol on the right, mark what is occluded, remove the stray element near the angle θ .” Unclear which symbol/element is meant — confirm with supervisor before editing.	29
that figure 2.13 needs to be fixed by me manually	34
FEEDBACK (supervisor): figure moved up to here as requested (“I would like Figure 2.9 to be here, not down there”). The supervisor also noted he will regenerate this figure himself; pending tweaks: enlarge the circular arrow in the proprioceptive panel, turn the rotated text to read normally, and reposition the “Robot”/internal labels.	36
check if we used 2D or 3D translation error	71
Real-world closed-loop runs (≤ 3) to be collected and the qualitative result inserted here / in Section 5.6.	73
the apriltag gives a pose, yolo only gives landmarks from which one could try to determine a pose (NPN I think is what you use there). But for our purposes we basically only use it as bounding box to determine the direction of the station. . .	86
note here that in our current configuration yolo does recognise the station from every angle but can’t tell us reliably from which angle we are seeing the station. This should not pose a significant challenge though, since we can assume that the GPS based global localisaiton already brings us in the detection zone of the apriltags. . .	88
GAP: physical closed-loop runs (≤ 3) to be collected 2026-06-16; insert the one-sentence qualitative result here and remove this note.	121

1. Introduction

1.1. Motivation

Agricultural robotics is no longer a laboratory curiosity but a commercially established field. Platforms such as FarmDroid for autonomous seeding and weeding [1], the Naïo Oz weeding robot [2], and the ecoRobotix ARA for spot spraying [3], alongside Global Positioning System (GPS)-guided field robots from established manufacturers [4, 5], demonstrate that individual agronomic processes can already be automated reliably. Their capabilities are increasingly underpinned by advances in artificial intelligence—deep-learning perception and control, and ever more affordable sensor suites [6]—which enable robust behaviour in environments that were previously intractable [7, 8]. Yet each of these systems still automates a *single*, temporally bounded step of conventional, large-scale cultivation: it is dispatched onto a large and largely homogeneous field, executes one mission such as seeding or spraying, and returns [9, 10]. By automating existing practices rather than rethinking the cultivation process itself, these platforms forgo the opportunities that small, persistent robots could unlock; none of them sustain long-term, multi-objective field management.

The next step is therefore not to automate one more process, but to cultivate an entire *agro-ecosystem* continuously over months and years. Agroforestry—the deliberate integration of trees and shrubs with crops—is an exemplary such system and the application domain of this thesis. It is widely recognised as a more sustainable alternative to conventional, mechanically-tilled monocultures, which are primary drivers of soil degradation and biodiversity loss [11]: trees act as ecosystem engineers that enhance nutrient cycling, water retention, and erosion control [11, 12]; the systems sequester carbon and buffer crops against climate extremes [13]; and, despite using less surface area per crop, studies report a 14–34 % increase in overall land productivity [11]. Their adoption nonetheless remains limited, and for an operational rather than an agronomic reason: the complex spatial arrangement makes agroforestry highly labour-intensive, particularly during tree establishment and routine care, leading to prohibitive implementation costs [11, 14, 15]. The narrow tree strips characteristic of these systems—often only two to three metres wide—are moreover poorly suited to conventional heavy mechanisation. Small, autonomous robots that perform biodiversity-aware, repeated interventions are thus widely seen as the enabling technology that could make agroforestry economically viable [16, 14, 17].

Cultivating such a system over its full life cycle, however, demands far more than executing an isolated task: the robot must plan over long horizons, reason about a continually changing context, and replenish its own resources without human intervention. This is precisely the regime of Long Term Autonomy (LTA). Robots operating currently within the mission-based paradigm cannot readily extend their operational horizons to continuous operation, because

of two fundamental bottlenecks [18]. Physically, they are constrained by resource depletion—battery drain, payload exhaustion—and by accumulating system limits such as thermal buildup or storage overflow. Cognitively, many historic control architectures assume fixed behavioural paradigms; even when a robot holds an accurate internal world model, such systems lack the adaptability to manage unforeseen situations or novel objects in unstructured surroundings [19, 20]. Achieving long-term autonomous cultivation therefore requires the convergence of four capabilities: (i) *resource management*, autonomously tracking and replenishing energy, water, and material, which directly entails autonomous docking; (ii) *task planning* over long horizons; (iii) *robust perception* under continually changing conditions; and (iv) *persistent world modelling*, maintaining an environmental representation that is updated over time.¹ At the same time, the environment is only semi-structured: regular tree rows provide stable reference points that make it tractable as a research platform, while its scalable biodiversity enables a gradual escalation of operational complexity, from relatively uniform crop zones toward multi-layered vegetation comprising diverse crops, dense bushes, tree canopies, and grazing livestock. Finally, the environmental complexity—unpredictable weather, thermal stress, and severe resource constraints—means that a robust architectural framework developed for this ecosystem is highly transferable to a broad range of other advanced field-robotics applications [24, 25].

Of these four pillars, this thesis isolates the first—resource management—because it is the precondition for all the others: however capable its perception, planning, or world modelling, a robot that cannot autonomously replenish its resources simply will not remain deployed long enough for any of them to matter. And within resource management, the bottleneck is not predicting *when* a resource will run out, but *replenishing* it: predictive resource tracking is useless unless the robot can complete the return mission and couple to its station. That maneuver—autonomous docking—requires precise spatial alignment to engage electrical, data, and fluid interfaces despite dynamic illumination, adverse weather, and irregular terrain that degrade perception and localisation [22, 23, 26, 27]. Crucially, docking cannot be treated as a standalone component: deciding when to abort a field task, routing home, and aligning with the station are functions that must be *embedded in an overarching autonomy architecture*. Bridging the gap to persistent field autonomy thus poses two coupled problems—how to structure such an architecture, and how to perceive the station reliably enough to dock within it—which the following section states explicitly.

1.2. Problem Statement and Scope of the Thesis

The thesis assignment defines two goals that build on one another: an architectural analysis for resource-aware LTA, and the construction and sensor-based evaluation of a docking

¹These four capabilities are a synthesis of the long-term-autonomy enablers surveyed by Kunze et al. [18] (navigation and mapping, perception, knowledge representation and reasoning, planning, and learning) and the system-level requirements demonstrated by the STRANDS long-term deployment [21]; the requirement for autonomous resource replenishment as a precondition for persistent operation is established in the docking literature [22, 23].

prototype embedded within it. These map onto two research questions.

RQ1 (Architecture). *Which architectural principles does an outdoor field robot require for resource-aware long-term autonomy, and how can autonomous docking be embedded within such an architecture?*

RQ2 (Sensing). *How do different sensor modalities and detection algorithms compare in positional accuracy, reliability, and computational efficiency during the autonomous docking of a mobile robot under varying environmental conditions?*

RQ1 is answered by the architectural analysis in Chapter 2 (Section 2.2) and the design decisions in Chapter 3 (Section 3.2); RQ2 is answered by the experimental design in Chapter 4 and the evaluation that follows it. The two questions are sequential: the architecture establishes *where* docking sits and *what* the perception subsystem must deliver, and the sensor comparison then determines *how* that subsystem should be built.

1.2.1. Goals and Scope

Answering these questions unites theoretical LTA architectural principles with an empirical, prototype-driven evaluation. The core objectives are:

1. **Architecture (RQ1):** Derive, from the requirements of resource-aware LTA, a conceptual control architecture for an outdoor field robot and show how autonomous docking is embedded within it.
2. **System and Mechanical Design:** Construct a functional, weather-resistant outdoor docking prototype featuring perceptual features (such as fiducial tags) that enable reliable alignment, and implement a multi-stage docking software pipeline (detection, approach, alignment) in ROS 2, optimised for the robot’s edge hardware.
3. **Comparative Experimental Evaluation (RQ2):** Conduct systematic tests in both simulated and realistic outdoor environments to benchmark vision-based (RGB camera) against laser-based (LiDAR) perception, evaluating:
 - *Positional and rotational accuracy* at the final interface;
 - *Robustness and success rates* across varying weather and lighting; and
 - *Computational efficiency*.

Scope and Limitations. The scope is deliberately confined to the *terminal* docking phase—local perception, alignment, and physical coupling—and the architectural reasoning that situates it within LTA. Three boundaries are drawn explicitly, as conscious simplifications rather than oversights:

- **No predictive resource model.** The robot does not forecast battery depletion from a thermodynamic or terrain-energy model. The docking decision is *threshold-based*: a return is triggered when the estimated remaining charge, less a conservative safety margin, would no longer cover the path cost back to the station. A predictive resource model is identified as future work.
- **Terminal phase only.** High-level global mission planning and long-horizon route optimisation are treated at the architectural level (Chapter 3) but are not implemented or evaluated; the prototype and experiments concern the local approach and alignment.
- **Architecture vs. prototype.** The full architecture (cloud-coupled agronomic simulation, fluid/material replenishment, agricultural actuators) is developed conceptually; only the docking subsystem is realised and tested in hardware and simulation.

1.3. Structure of the Thesis

The remainder of this thesis is structured as follows:

- **Chapter 2: Fundamentals and State of the Art (SOTA)** – Establishes the fundamentals of long-term autonomy (LTA) and evaluates the major autonomy architecture paradigms along a common set of dimensions—the survey underpinning RQ1. It further reviews the current SOTA in resource management, sensing technologies, and docking interfaces, providing the option space from which Chapter 3 selects.
- **Chapter 3: Conceptual Analysis and System Design** – Derives, from the requirements, a conceptual control architecture for resource-aware LTA and shows how docking is embedded within it (the contribution answering RQ1). It then narrows to the docking prototype actually built and makes the requirement-driven selection of energy interface, data link, and candidate sensors, closing with the concrete ROS 2 docking pipeline.
- **Chapter 4: Experimental Setup** – Outlines the experimental design formulated to validate the docking system. It describes the selection of experimental parameters, the physical testing setup for accuracy and weather robustness, and the software implementation spanning edge deployment and simulation.
- **Chapter 5: Evaluation and Results** – Presents the evaluation methodology and the subsequent results. It provides a comprehensive analysis of the positional and rotational accuracy, success rates, and robustness of both RGB camera and LiDAR-based docking approaches, concluding with a comparative discussion to recommend optimal use cases.
- **Chapter 6: Conclusion** – Concludes the thesis by summarizing the core contributions, reflecting on the current limitations of the system, and proposing actionable directions for future work.

2. Fundamentals and State of the Art

2.1. Theoretical Requirements for LTA

2.1.1. What is autonomy?

Autonomy is frequently misunderstood as a uniform, one-dimensional "widget" that can simply be added to a system to achieve absolute independence from human control [28]. In reality, autonomy is not a binary state. It is more accurately described as the absence of specific dependencies on external entities—such as human operators, programmers, technical infrastructure, or other autonomous systems—relative to a particular function and situation. To achieve this state without relying on continuous external intervention, autonomous systems must fulfill two fundamental requirements that are closely associated with physical and cognitive independence: **self-sufficiency** and **self-directedness**.

- **Self-sufficiency (Physical/Operational Independence):** This is the system's ability to "take care of itself" and maintain its operational existence. It involves physical independence from the environment, such as managing energy resources, surviving without constant maintenance, or executing physical interactions without relying on external technical systems (like a guided docking station).
- **Self-directedness (Cognitive Independence):** This represents a freedom from outside control when forming goals and navigating a task. As explored by Castelfranchi and Falcone, this encompasses both *decision autonomy* (knowing what choices are available and selecting them based on user-dependent preferences) and *action autonomy* (the ability to act on those choices and maintain robustness against environmental changes, uncertainty, or interference) [29].

Crucially, achieving high levels of self-directedness and self-sufficiency does not obviate the need for human-machine collaboration. As Bradshaw et al. note, the idea that autonomous systems are entirely independent is a "deadly myth"; instead, automation must remain human-centered [28]. Effective autonomy maximizes the respective strengths of both parties: the robot provides continuous, reliable execution (e.g., being always awake, persistently monitoring an environment without forgetting), while the human provides high-level cognitive oversight (e.g., interpreting complex contextual events like weather, and setting the overarching goals for the robot).

Levels of Autonomy: From Vehicles to Robots

The most widely recognised attempt to formalise levels of autonomy comes from the automotive industry, and it is worth recalling here precisely because it is so familiar—before explaining why it does not transfer to the setting of this thesis:

- **Vehicles (Structured Autonomy):** The automotive sector classifies autonomy using a rigid ladder of six discrete tiers, the SAE levels, ranging from full manual control (Level 0) to full self-driving without any human fallback (Level 5). Each level is defined by how the dynamic driving task is divided between human and machine and by the

boundaries of a tightly specified Operational Design Domain (ODD)—the road types, speeds, and weather under which the system is certified to operate [30]. This discrete classification works because the environment is highly structured (paved roads, clear traffic rules) and the operational domain can be enumerated in advance.

- **Robotics (Continuous Autonomy):** These assumptions break down for a field robot. An agroforestry environment cannot be enumerated as a fixed ODD, and the system must continuously trade off how much it can do alone against how much a human must supervise, rather than sitting at one fixed “level”. Robot autonomy is therefore better understood as a continuum: Beer et al. categorise it by the exact degree of human intervention and the robot’s ability to sense, plan, and act independently [31]. The SAE ladder is introduced here only as a shared reference point; the remainder of this thesis adopts the continuum view because the discrete, domain-bounded model cannot capture open-ended outdoor operation.

The Shift to Long-Term Autonomy

This distinction becomes critical when robots are deployed for extended periods. Unlike passenger vehicles on mapped roads, robots often operate in highly unstructured outdoor environments that degrade sensors and challenge navigation algorithms [20, 25]. To achieve *long-term* autonomy, a robot cannot simply rely on a static “level” of programmed automation. Instead, it requires robust artificial intelligence to handle edge cases over time [18]. Romero et al. argue specifically that lifelong, *open-ended* learning—the continual, autonomous acquisition of new skills and goals without a fixed end state—is a necessary ingredient for cognitive architectures that must adapt to an ever-changing world, rather than an optional enhancement [19].

2.1.2. What is long-term autonomy?

While one could intuitively assume that long-term autonomy (LTA) simply means allowing an autonomous system to operate for extended durations without supervision, the reality is much more complex. The problems that arise in an autonomous system running for a long duration fundamentally differ from short-term, mission-based autonomy.

If a fully modeled robot is deployed in a completely known and static environment, the challenge of LTA simply reduces to a problem of robustness, meaning the primary goal is just enabling the robot to remain operational for as long as possible [18]. In highly predictable and structured environments—such as a satellite orbiting in space—achieving LTA is relatively straightforward. However, without these simplifying assumptions, autonomous robots operating in unstructured environments face a complex set of interrelated challenges [18]. As with autonomy itself, we can separate the requirements for long-term operation into physical and cognitive categories.

The most effective way to determine what is necessary for long-term autonomy in mobile robots is to analyze the scenarios that currently necessitate human operator intervention. These failure cases highlight the gaps between short-term automation and true LTA.

Physical Interventions

Physical interventions are required whenever the robot can no longer sustain its own operational integrity or physically act on its surroundings, and a human must step in to restore it. The following are representative examples rather than an exhaustive list:

- **Robot edge cases:** The hardware experiences a critical failure, such as a broken motor or a blown tire.
- **Environmental edge cases:** The robot becomes physically immobilized by its surroundings—for example, the terrain changes so drastically between visits that a previously traversable path has turned into a small river after heavy rain, or the drivetrain becomes stuck in dense brush.
- **Resource depletion (and accumulation):** The robot runs out of a consumable such as power and requires manual charging, or it hits an accumulating limit such as thermal buildup or full data storage.
- **Routine maintenance:** The robot requires *regular*, scheduled physical upkeep, such as cleaning a degraded sensor.
- **Repairs:** The robot requires the *irregular*, unplanned replacement of broken or worn external components.
- **Task limitation:** A subtask of the robot's function physically cannot be performed by it and requires a human. For example, a weeding robot might not be able to remove weeds taller than roughly one metre.

Cognitive Interventions

Cognitive interventions are required whenever the robot lacks the adaptive intelligence to handle an ever-changing world and a human must supply the missing reasoning. In many long-term applications, the environment inevitably changes over the lifetime of the system [18]: short-term changes (an animal moving across the robot's field of view), medium-term changes (machinery or harvested crops relocated between visits), and long-term changes (seasonal shifts, plant growth, or surface wear). Each timescale of change can defeat a robot that reasons over a fixed internal model, which is exactly what makes the following cognitive shortcomings—again, representative examples rather than an exhaustive list—so consequential for long-term autonomy:

- **Lack of in-context information:** The robot lacks the immediate data required to determine the *current* state of the world and make a decision. Parts of the environment may not be fully known prior to deployment, and new objects may appear over time [18]; in artificial-intelligence terms, the robot struggles to deal with an open world [18]. For example, a tree that has grown into the planned path since the last visit is simply absent from the robot's current perception.

- **Lack of world knowledge and training data:** The robot lacks the data to build a world model capable of representing the state of the world. The scenarios encountered fall outside the system’s pre-training data, and the knowledge a robot starts with is highly unlikely to cover the behavior required of it during long-term operation [18].
- **Lack of cognitive complexity:** The system lacks the computational capacity or algorithmic sophistication to process complex decisions—one of several factors that can limit how rich and accurate a world model the robot can actually exploit.
- **Lack of goal generation:** The robot successfully completes all assigned tasks but cannot synthesize new objectives on its own [18].
- **Lack of performance:** The robot can in principle perform the task, but its speed or quality is insufficient for the current situation, so a human takes over to meet the requirement.
- **Lack of responsibility:** Humans are kept in the loop because a robot cannot be held legally or morally accountable if an action goes wrong.

2.1.3. What does a robot need for long-term autonomy?

Each failure case identified above implies a capability the robot must possess to avoid the corresponding human intervention. The following list mirrors those problems and states, in two to three sentences each, what it takes to address them.

- **Self-sustenance and resource management:** This directly answers *resource depletion (and accumulation)*, *routine maintenance*, and *repairs*. The robot needs advanced energy management and automated replenishment infrastructure—the autonomous docking studied in this thesis—so that consumables can be restored and accumulating limits (thermal buildup, full storage) periodically cleared without a human [22, 23]. Scheduled upkeep such as sensor cleaning likewise has to be designed into the docking cycle rather than left to an operator.
- **Physical fault tolerance and self-diagnosis:** This answers the *robot edge cases*—a broken motor, a blown tyre, a degraded sensor—that the original resource-centred list did not cover. The robot needs condition monitoring that detects and isolates hardware faults, predictive self-monitoring that flags degradation before it causes a breakdown, and graceful-degradation behaviours that keep the platform safe (or bring it home) when a component fails [18, 21].
- **Dynamic navigation and mapping:** This answers *environmental edge cases* and the *lack of in-context information*. Because the world changes across all three timescales, the robot cannot rely on static maps; it must maintain and update environment representations over long deployments, evaluating what to remember and what to forget as conditions shift [18, 32].

- **Persistent world modelling:** This answers the *lack of world knowledge* and *lack of cognitive complexity*. The robot must construct and maintain a model rich enough to represent the current state of the world and to predict the outcome of its actions, within the computational budget available on a mobile platform [33, 34].
- **Continuous online learning:** This answers the *lack of world knowledge* and *training data*. Since a robot cannot be pre-programmed for every open-world scenario, learning during operation—not just at design time—is crucial, including adapting to new tools and knowledge as they become available [18, 19].
- **Executive control systems:** This answers the *lack of goal generation* and *lack of performance*. An executive layer that responds to opportunities and failures keeps the robot from getting trapped in behavioral loops and provides recovery mechanisms for autonomy-hindering failures [18, 21].

Two intervention cases are deliberately *not* resolved by a new technical capability. The *task limitation* case—a subtask that the platform physically cannot perform, such as removing a weed taller than its tooling allows—is a bounded hardware-design decision rather than an autonomy gap: it is addressed once, at design time, by sizing the mechanism to the task envelope, and anything outside that envelope is by definition out of scope. The *lack of responsibility* case is a legal and ethical constraint, not a technical one, and is best handled by keeping a human in the supervisory loop. Every other intervention identified above maps onto exactly one of the six capabilities, and each capability answers at least one intervention, so the two lists are mutually exhaustive.

The picture that emerges is that long-term autonomy is not a property a robot either has or lacks, but a balance it must continually maintain. Each human intervention identified above marks a point at which that balance would otherwise break: the platform would run flat, seize up, lose its way, misjudge a changed scene, exhaust its prior knowledge, or stall without a next goal. The six capabilities derived here are best read not as an isolated feature list but as a set of four interlocking loops that keep the robot inside its envelope of self-sufficiency and self-directedness over time—the same four capabilities announced in Section 1.2 (resource management, robust perception, persistent world modelling, and task planning), now derived from the intervention analysis rather than asserted. A physical loop keeps the body alive and serviceable—replenishing energy and materials, clearing accumulating limits, and detecting and tolerating component faults before they become failures. A perceptual loop keeps the robot’s picture of the world current as terrain, objects, and seasons drift, deciding what to retain and what to discard. A cognitive loop turns that current picture into foresight, predicting the consequences of candidate actions and closing the gap between what the robot was trained on and what it now encounters by learning during operation rather than only at design time. An executive loop, finally, sits above the others: it generates new objectives, recognises when performance is inadequate, and recovers from the failures that would otherwise trap the system in unproductive behaviour.

What makes these loops a *system* rather than a checklist is their interdependence. World modelling is worthless without the perception that feeds it and the learning that corrects

it; online learning is dangerous without an executive layer to supervise it; and none of the cognitive machinery matters if the physical platform cannot keep itself powered and intact long enough to use it. The physical loop is thus the precondition for all the rest, which is exactly what grounds the scope decision of Section 1.2: this thesis isolates physical self-sustenance—and specifically autonomous docking, the mechanism through which energy, data, and material are restored—as the concrete capability it develops and evaluates, while treating the remaining loops as the architectural context into which a docking-capable robot must eventually fit.

2.2. Analysis of Autonomy Architectures

In autonomous robotics, a single, universal software architecture does not exist; different situations and operational environments inherently require different control architectures. The optimal structural paradigm varies significantly depending on the predictability of the environment, the complexity of the task, and the necessary level of autonomy. While we will formulate the precise requirements for our specific application—resource-aware long-term autonomy in agricultural settings—and derive the resulting structural design later in this work, this section will focus on how the different cognitive architectures for robotics work.

2.2.1. Dimensions of Robotic Architectures

To systematically evaluate, compare, and categorize the diverse paradigms in robotic control, it is helpful to view them through a structured taxonomy. While various authors have analyzed individual characteristics of control systems (e.g., [35]), a single, universally accepted taxonomy does not exist. Therefore, for the purpose of this analysis, we synthesize these scattered concepts into a standardized, multidimensional framework. Rather than viewing architectures as monolithic concepts, they can be decomposed across six continuous dimensions, split into two primary theoretical spaces: the Temporal and Information Space, and the Structural and Representation Space (see Figure 2.1).

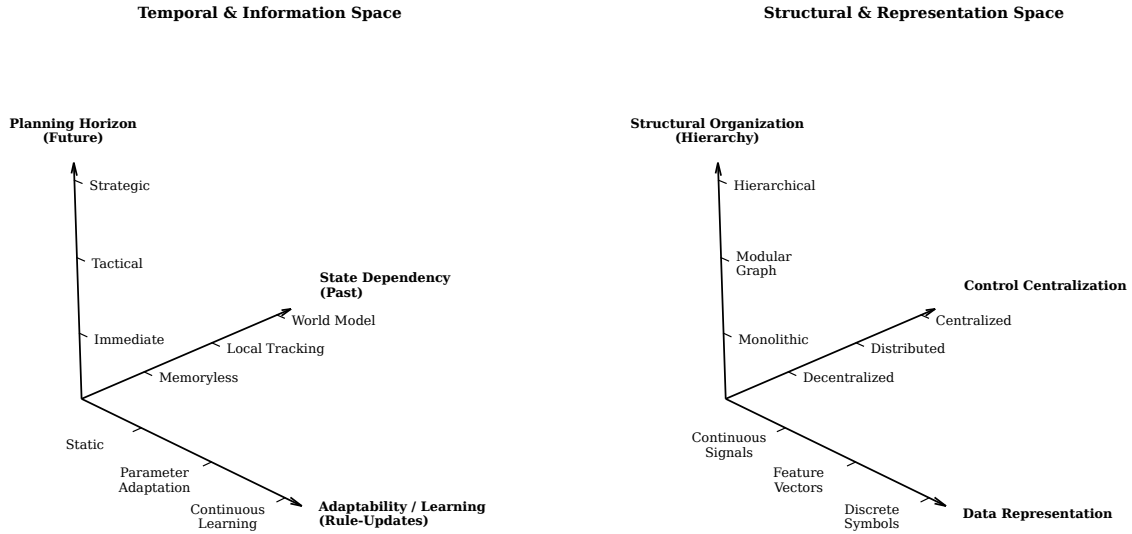


Figure 2.1.: The six dimensions of robotic architectures, divided into the Temporal & Information Space (left) and the Structural & Representation Space (right). These axes provide a framework for classifying the capabilities and structural paradigms of autonomous systems.

The Temporal and Information Space

This space defines how an architecture interacts with time—both computationally and operationally—and how it manages information regarding the past, present, and future.

- **Planning Horizon (The Future):** This dimension measures the temporal depth of the system's predictive capabilities. At the lowest end, *Reflexive* systems operate strictly in the immediate present, mapping current inputs directly to outputs without foresight. Intermediate *Tactical* systems predict short-term outcomes (e.g., local trajectory rollouts). At the highest end, *Strategic* systems compute long-term, multi-step plans required for complex goal achievement and resource management.
- **State Dependency / Memory (The Past):** This axis quantifies the system's reliance on historical data to make current decisions. *Memoryless* or *Reflexive* architectures utilize only instantaneous sensor readings [36]. Moving up the axis, systems employ *Local Tracking* (e.g., maintaining the state of a nearby dynamic obstacle). The highest complexity involves maintaining a *Deliberative World Model*, a comprehensive, persistent internal representation of the global environment.¹
- **Adaptability / Learning (Meta-Time):** This dimension describes how the rules governing the architecture change over its operational lifespan. *Completely Static* systems execute hardcoded logic without modification. Systems with *Parameter Adaptation* can tune specific variables (e.g., Proportional-Integral-Derivative (PID) gains) online. At the apex, *Continuous Learning* architectures actively update their own structural weights, rules, or representations based on cumulative experience.

The Structural and Representation Space

This space characterizes the internal data topologies and the organizational hierarchy of the software components managing the robot.

- **Level of Abstraction (Data Representation):** This axis details how physical reality is mathematically represented within the system. The lowest level processes raw, *Continuous Signals* (e.g., motor currents or raw pixel arrays). The intermediate level extracts *Feature Vectors* (e.g., bounding boxes or obstacle gradients). The highest level of abstraction utilizes *Discrete Symbols* (e.g., semantic tags like "Charging Station" or "Tree"), which are essential for logical reasoning and high-level task planning.
- **Control Centralization:** This dimension evaluates the concentration of decision-making authority. *Decentralized / Swarm* systems rely on independent, local agents interacting to

¹A *world model* is an internal, predictive representation of the environment that an agent builds from its stream of observations and uses to reason about states it cannot currently perceive and to anticipate the consequences of its actions before executing them [33, 34]. It is thus more than a static map: it is a compressed, generative model of how the world evolves, against which candidate actions can be simulated. The term is used in this sense throughout the thesis.

produce emergent behavior. *Distributed / Modular* architectures separate concerns across interconnected, semi-independent nodes. *Highly Centralized* systems route all critical sensor data to a single, monolithic executive process that dictates all actions.

- **Structural Organization (Hierarchy):** This axis describes the topological relationships between computational modules. *Monolithic* systems operate as single, tightly coupled programs. *Modular / Graph* topologies allow for asynchronous, peer-to-peer data flows between specialized components (common in modern middleware). *Strictly Hierarchical* organizations mandate a rigid, top-down chain of command, where high-level nodes dictate the operational state of lower-level reflexes.

Correlations and Interdependencies

While the six dimensions provide a theoretical continuum, physical and computational constraints dictate that robotic architectures cannot occupy arbitrary coordinates within these spaces. Strong correlations exist between specific dimensions, causing systems to naturally cluster into distinct archetypes.

The most prominent correlation exists between the **Level of Abstraction** and the **Planning Horizon**. Computing a strategic, long-term plan using raw, continuous sensor signals is computationally intractable due to the curse of dimensionality [37]. Therefore, systems that require a high planning horizon must mathematically compress their environment into discrete symbols. Conversely, maintaining a reflexive, immediate response requires bypassing heavy symbolic abstraction; reactive survival mechanisms inherently rely on continuous signals or simple feature vectors to minimize processing latency [38].

Similarly, there is a deep interdependency between **State Dependency** and **Control Centralization**. Architectures that rely on a deliberative world model naturally gravitate towards higher centralization or, at minimum, a heavily regulated distributed structure. Maintaining a consistent, synchronized global map across a completely decentralized swarm of nodes introduces massive data transmission overhead and frequent race conditions [39]. Thus, highly state-dependent systems typically centralize their world model to maintain data integrity, whereas memoryless systems easily parallelize into decentralized structures.

Finally, **Structural Organization** heavily influences **Adaptability**. Strictly hierarchical, hard-coded systems (such as early subsumption architectures) struggle to implement continuous learning, as structural rigidity prevents the dynamic reallocation of authority based on new experiences. In contrast, modular, graph-based organizations allow new learning nodes to be injected into the network or existing parameters to be tuned dynamically without destabilizing the entire control loop [40].

These fundamental correlations explain why the historical progression of robotics has consolidated around a few dominant paradigms. The following sections analyze these classical and modern archetypes to identify their suitability for resource-aware, long-term agricultural autonomy. Table 2.1 consolidates where each archetype falls along the six dimensions; the individual sections describe the underlying mechanisms that produce these placements.

Table 2.1.: Classification of the surveyed architectural archetypes along the six dimensions of Section 2.2.1. Reactive control is represented by both Motor Schemas (MS) and the Subsumption Architecture (Sub.). Spans (e.g. “Strategic $\leftrightarrow$ Reflexive”) indicate that an architecture deliberately occupies the full range of a dimension across its internal tiers.

Dimension	Deliberative (Sense Plan Act (SPA))	Reactive (MS)	Reactive (Sub.)	Hybrid (3-Layer)	Learning (VLA)
Planning Horizon	Strategic	Reflexive– Tactical	Reflexive	Strategic $\leftrightarrow$ Reflexive	Reflexive– Tactical
State Dependency	World Model	Memoryless	Memoryless	World Model $\leftrightarrow$ Memoryless	Local Tracking
Adaptability	Static	Parameter Adaptation	Static	Parameter Adaptation	Continuous Learning
Level of Abstraction	Discrete Symbols	Feature Vectors	Continuous Signals	Symbols $\leftrightarrow$ Signals	Continuous Signals
Control Centralization	Highly Centralized	Distributed	Decentralized	Distributed	Highly Centralized
Structural Org.	Monolithic	Modular / Graph	Strictly Hierarchical	Modular / Graph	Monolithic

2.2.2. Deliberative Architectures

The deliberative architecture, frequently referred to as the SPA paradigm, is one of the earliest and most traditional approaches to robot autonomy. In this sequential model, the robot first gathers sensor data to build or update a comprehensive, global world model. Next, it utilizes this internal model to formulate a complete plan to achieve its goal. Finally, it executes the planned sequence of actions.

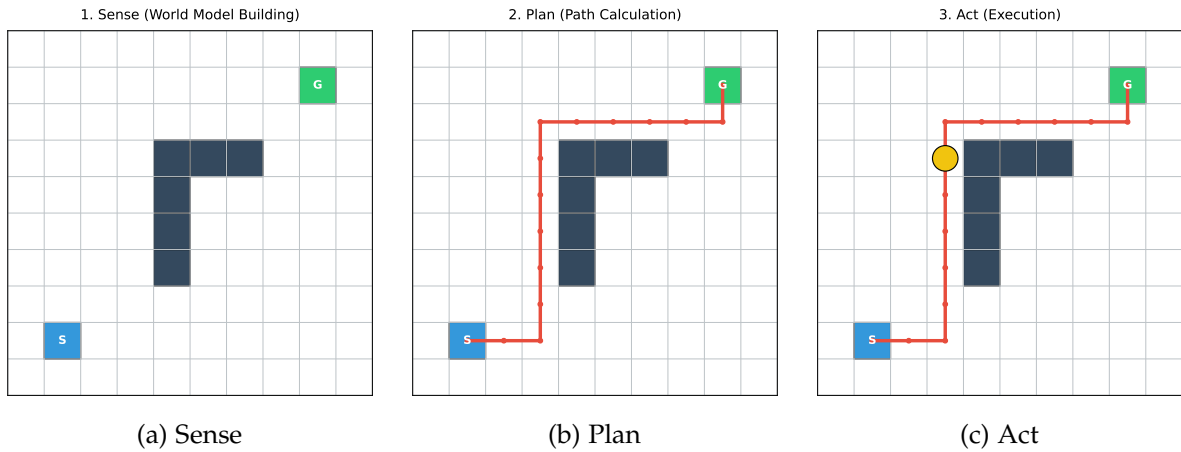


Figure 2.2.: The Sense-Plan-Act (SPA) architecture depicted in a discrete grid environment. (a) The robot maps obstacles to construct a world model. (b) A pathfinding algorithm computes a collision-free trajectory. (c) The controller executes the generated path.

The defining characteristic of deliberative systems is their reliance on heavy planning and extensive world modeling, which early pioneers of mobile robotics used to achieve "intelligent" behavior. Shakey the Robot (SRI International) used the STRIPS planner and a simplified, grid-based world representation to compute a sequence of discrete actions before executing any movement [41]. The Stanford Cart, developed by Hans Moravec in the 1970s and 80s, navigated obstacle courses using stereopsis to locate objects in 3D, build a detailed spatial model, and plan an obstacle-avoiding path [42]. The Cart also exposed the primary bottleneck of pure deliberative systems: it moved in short "lurches," rolling about one meter before stopping for up to 15 minutes to process new images, update its world model, and compute a new plan [42].

While deliberative architectures excel at finding optimal paths and complex, high-level goals in static environments, they are fundamentally limited in dynamic settings like agroforestry. The cost of maintaining a global world model and the time required for re-planning make pure SPA architectures too slow to react to sudden changes—a purely deliberative system might fail to avoid a dynamic obstacle while slowly calculating a path to a charging station. Nevertheless, the high-level predictive capabilities of deliberative reasoning remain essential for resource-aware self-maintenance: deciding when to abort a field intervention and route back for autonomous docking requires exactly the foresight and resource modeling that deliberative layers provide. This implies a strict architectural requirement: deliberative planning must be used for high-level routing and state management, but decoupled from immediate obstacle avoidance.

2.2.3. Reactive Architectures

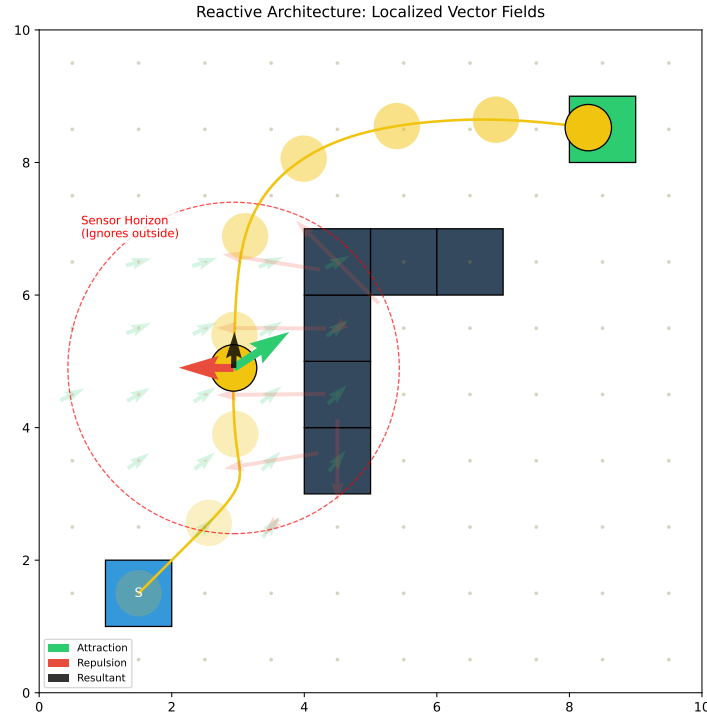


Figure 2.3.: A reactive architecture (Motor Schemas) navigating a discrete grid environment. In contrast to the SPA architecture, the robot does not maintain a global world model or pre-compute a trajectory. Instead, movement is driven continuously by a localized vector field. The robot reacts purely to stimuli—attraction to the goal and repulsion from obstacles—that fall strictly within its immediate sensor horizon, resulting in a direct, emergent trajectory.

To overcome the computational bottlenecks and the inability to react to sudden environmental changes inherent in the SPA paradigm, reactive architectures were developed to provide fast, reflex-based responses directly coupled to sensory stimuli without a comprehensive global world model [43]. Two prominent paradigms within this domain are Motor Schemas and the Subsumption Architecture.

Motor Schemas

Developed by Arkin, Motor Schemas treat behaviour as a dynamic network of concurrent processes rather than layers: each instantiated schema is an independent agent that, paired with a perceptual schema, computes a continuous velocity vector via a potential-field method, and all active vectors are summed to yield the robot's instantaneous motion [43]. For example, an obstacle-avoidance schema produces a repulsive field whose magnitude grows as the robot nears the obstacle within its sphere of influence. The approach naturally accommodates

perceptual uncertainty—uncertain obstacles exert weaker repulsion—and its modularity eases development and distributed real-time execution, but it inherits the classic potential-field pitfalls of local minima and dead spots, from which only higher-level planning can reliably recover [43].

The Subsumption Architecture

Introduced by Brooks in 1985, the Subsumption Architecture replaces centralized sense-plan-act control with a decentralized, horizontal decomposition into task-achieving behaviours that map sensors directly to actuators, arranged in a hierarchy of competence (e.g. avoid, wander, explore) of asynchronous finite state machines without shared memory [36]. Higher levels subsume lower ones by *inhibiting* their outputs or *suppressing* their inputs [36]. The result is extreme robustness and minimal latency—survival reflexes keep running even if higher layers stall—but poor scalability, as hand-tuning the suppression and inhibition timing across many layers becomes intractable. Pure Subsumption is rare today, yet its philosophy of decoupled, asynchronous processing underpins modern publisher/subscriber middleware such as ROS 2, which blends subsumption-style reactive local navigation with the model-based global planning Brooks originally rejected.

Resource awareness. Both reactive paradigms share a defining blind spot for this thesis: by minimising internal state they retain no model of the robot’s own resources. A purely reactive controller has no representation of remaining charge and no horizon over which to reason about a return trip, so it cannot decide *when* to abandon a task and dock. Fast reflexes are necessary for survival in the field, but resource-aware self-maintenance demands exactly the deliberative foresight that reactivity discards—motivating the hybrid combination below.

2.2.4. Hybrid (Three-Layer) Architectures

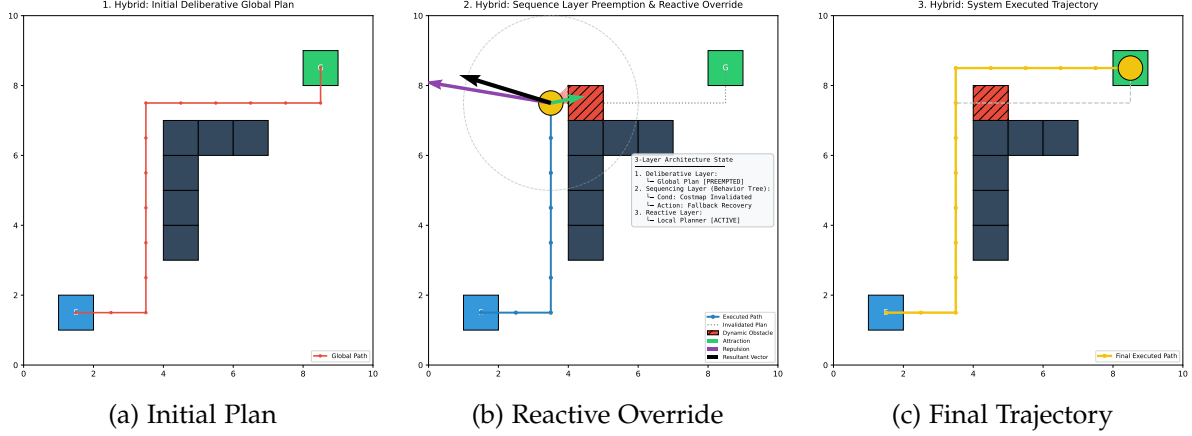


Figure 2.4.: A 3-tiered hybrid architecture managing a dynamic environment. **(a)** The deliberative layer generates a static, global trajectory. **(b)** The sequencer monitors execution and yields control to the reactive layer when an unexpected dynamic obstacle blocks the path, generating a localized vector field to avoid collisions. **(c)** After successfully navigating the obstacle, the reactive layer hands control back, allowing the robot to seamlessly rejoin and complete the global plan.

While purely reactive architectures successfully mitigated the computational bottlenecks of the SPA paradigm, they hit a "capability ceiling" due to their inability to manage long-term tasks, optimize paths, or predict future states [38]. Robust autonomy requires both the fast reflexes to survive dynamic environments and the deliberative foresight to achieve complex goals, which led to the convergence on a unified paradigm: the Three-Layer Architecture [38]. Its fundamental rationale lies in the handling of internal state: SPA struggles because its heavy reliance on internal state (e.g., global maps) frequently falls out of synchronization with the dynamic world, whereas reactive architectures minimize internal state entirely and leave the robot vulnerable to sensor noise, occlusions, and specular reflections [38]. The three-layer architecture resolves this dichotomy by organizing algorithms into separate operational tiers based strictly on their relationship with internal state and time [38].

Theoretical Foundation

The architecture decomposes the robot's software into three distinct, asynchronously operating layers [38]:

- **The Controller (Reactive Layer):** This layer maps sensors directly to actuators using fast, mostly stateless feedback loops [38]. The algorithms running here must execute within constant-bounded time and space limits to guarantee hard real-time stability [38]. Crucially, this layer is designed to "fail cognizantly"—meaning if a primitive behavior

encounters a situation it cannot resolve, it definitively detects the failure and alerts higher layers rather than attempting to avoid it or deadlocking [38]. Any internal state utilized here must be strictly ephemeral [38].

- **The Sequencer (Executive Layer):** Sitting above the controller, the sequencer utilizes conditional sequencing to manage the robot’s current task [38]. It retains memory of the past (internal state regarding the robot’s execution history) to conditionally determine which primitive behaviors the Controller should execute [38]. It handles contingencies but strictly avoids time-consuming computations like path searches or temporal projections [38].
- **The Deliberator (Planning Layer):** This layer handles computationally expensive, search-based algorithms that predict the future, such as global path planning or resource modeling [38]. Because calculating a route may take significantly longer than the rate of environmental change, the Deliberator runs as a separate thread, providing high-level plans or answering specific queries from the Sequencer without blocking the robot’s immediate reactive reflexes [38].

Modular Implementation and Relevance to Resource-Aware LTA

Translating this 1990s foundation into a modern distributed system requires robust middleware. In ROS 2, frameworks such as PlanSys2 realize the hybrid paradigm through a modular, multi-node architecture [44]: a symbolic PDDL planner fills the Deliberator role, computing global plans asynchronously; these plans are compiled into Behavior trees (Behavior Trees (BTs)) that act as the Sequencer; and the Controller layer is realized by deterministic ROS 2 lifecycle nodes that execute the primitive actions. This mapping is what makes the three-layer model directly relevant to resource-aware LTA. The unstructured field demands a fast Controller for cognizant obstacle avoidance, while *resource awareness*—predicting battery depletion, computing the return route to the docking station, deciding when to abort a task—is exactly the deep, state-heavy, predictive function that only a Deliberator provides, decoupled from the reactive loop so that planning latency never stalls a reflex. The Sequencer ties the two together: on a critical-battery event during a weeding intervention, a BT can suspend the current task, request a path to the station from the Deliberator, and hand terminal alignment to a specialised Controller [38, 44]. It is this clean separation of resource-critical foresight from real-time reaction that makes the hybrid model the natural fit for the self-maintenance bottleneck at the centre of this thesis.

Task Coordination Models: From State Machines to Behavior Trees

The Sequencer layer introduced above requires a concrete formalism to encode *which* primitive behavior the Controller should execute and *when* control should transfer between behaviors. Three coordination models dominate this role in robotics, forming a progression of increasing modularity.

The Finite State Machine (FSM) is the classical choice: the robot’s task is decomposed into a finite set of discrete states (e.g. *Navigate*, *Dock*, *Charge*), with transitions triggered by sensor events. FSMs are simple to reason about for small tasks, but they scale poorly: as behaviors are added, the number of explicit transitions grows combinatorially, and the resulting “spaghetti” of cross-links becomes difficult to maintain or extend. To tame this, Harel introduced *Statecharts*, which extend flat FSMs with hierarchy (nested superstates), orthogonality (concurrent state regions), and broadcast communication [45]. A Hierarchical State Machine (HSM) factors shared transitions out to a parent state, so that a global event such as “battery critical” can be handled once at a high level rather than re-specified in every leaf state.

Behavior trees (BTs) have since become the prevailing coordination model in modern robotics middleware, including the ROS2 navigation stack and the PlanSys2 executive discussed above [46, 44]. A BT is a directed tree whose execution is driven by signals called *ticks*, propagated from the root at a fixed frequency. Internal control-flow nodes—most importantly *Sequence* (run children in order until one fails) and *Fallback* (try children in order until one succeeds)—route ticks to leaf nodes, which are either *action* or *condition* nodes interfacing with the Controller. Each node returns one of three statuses: *Success*, *Failure*, or *Running* [46].

This tick-based model gives BTs two properties that make them well suited to the Sequencer role. First, they are inherently *reactive*: because the tree is re-evaluated from the root on every tick, a high-priority safety condition (e.g. an obstacle reflex or a low-battery check placed under a left-most *Fallback* branch) can preempt an ongoing action without explicit transition wiring. Second, they are *modular and composable*: any subtree is itself a valid BT, so complex missions can be assembled from reusable building blocks and the addition of a new behavior does not require editing existing transitions, unlike in a flat FSM [46]. A further practical advantage is that the symbolic, low-dimensional structure of a BT is amenable to automatic synthesis and optimization: prior work has shown that BT topologies and parameters can be learned via genetic programming or Reinforcement Learning (RL) from execution traces, rather than being hand-coded in full [47]. This combination of reactivity, modularity, and learnability is precisely why BTs are revisited as the executive layer of the proposed architecture in Chapter 3.

2.2.5. Learning Architectures

Principle of Operation

Recent advances in learning architectures have shifted the paradigm from modular, task-specific pipelines to unified, end-to-end generalist agents [48]. The SOTA relies heavily on Vision-Language-Action (VLA) models, which ingest multimodal inputs—RGB images, natural language instructions, proprioceptive states—and map them directly to low-level motor commands [49, 48]. These architectures typically combine a large pre-trained language model backbone with a visual encoder that fuses spatial and semantic features, and discretize the robot’s continuous action space into tokens so that actions can be predicted autoregressively

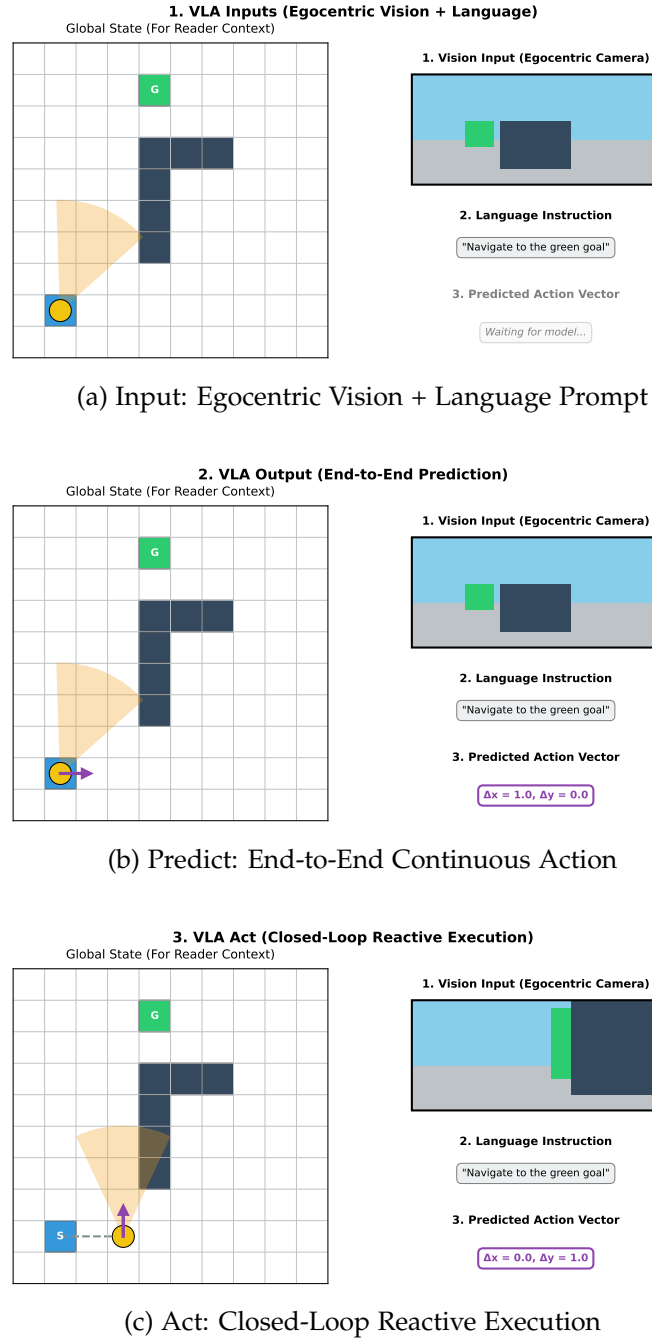


Figure 2.5.: The Vision-Language-Action (VLA) paradigm applied to navigation. (a) The policy takes raw egocentric visual observations and a natural language instruction as input. (b) Rather than generating a symbolic map or pre-calculating a trajectory, the model directly outputs an immediate low-level action vector. (c) Execution is closed-loop; the model continuously predicts next actions based on real-time visual feedback.

under a standard next-token prediction objective [49].

Advantages, Limitations, and Resource Awareness

The appeal of learning architectures is real: internet-scale pre-training yields strong zero- or few-shot generalisation to novel objects and environments; the language backbone grounds free-form instructions in physical affordances; deep RL can discover dynamic skills that are impractical to hand-script; and parameter-efficient fine-tuning (e.g. LoRA) lets one base model be specialised to new tasks or embodiments cheaply [48, 49, 50]. These strengths come with limitations that bear directly on the present application, however. The models are data-hungry (often hundreds of thousands of trajectories), their size makes real-time closed-loop inference on edge hardware costly, sim-to-real transfer is fragile without extensive domain randomisation, and as end-to-end black boxes they offer no formal safety guarantees and can produce linguistically plausible but physically unsafe actions [49, 48, 50]. Crucially for this thesis, such monolithic policies have no explicit notion of *resource state*: there is no separable component that reasons about remaining charge or schedules a return to the station, so resource-aware self-maintenance would have to be learned implicitly from data that, in agroforestry, simply does not exist.

Consequently, deploying a purely learning-based architecture in unstructured agroforestry—severe occlusion, unpredictable lighting, dynamic weather, and chronic data scarcity—is currently impractical: training from scratch fails for lack of data, while fine-tuning risks catastrophic overfitting and cannot bridge the embodiment and sim-to-real gaps without high-fidelity simulation that does not yet exist [49, 48, 50]. Learning is therefore best confined to a bounded, low-dimensional sub-problem (such as tuning an executive policy) rather than entrusted with the whole control loop—a point returned to in Chapter 3.

2.2.6. Digital Twins

The architectures surveyed so far describe how a robot *decides* and *acts*; a complementary concept addresses how the robot is *represented and predicted* throughout its operational life. The *digital twin* (DT) is relevant here not as an engineering monitoring tool but as the predictive world model on which a deliberative layer can plan—and, for a resource-aware robot, the substrate on which battery depletion or water demand along a candidate route is forecast before the maneuver is committed. The modern formulation originates with Grieves, who defines a digital twin as a virtual representation of a physical product coupled to it by a bidirectional data flow, such that the model evolves in lockstep with its counterpart and is used to anticipate undesirable emergent behaviour before it manifests physically [51].

Because the term is frequently conflated with any simulation, Kritzing et al. distinguish three cases by the *direction* of the data flow [52]: a *Digital Model* has no automated exchange (e.g. an offline CAD model or physics simulation); a *Digital Shadow* is updated one-way from the physical object but cannot act back on it; and only a true *Digital Twin* closes the loop with a fully automated, bidirectional flow—the model is updated from sensor data *and* its outputs are returned to act on the system. Genuine bidirectional twins remain rare; most reported

systems are models or shadows [52], integrating in practice four layers—data acquisition, a (physics-based, data-driven, or hybrid) predictive model, a low-latency network, and an application layer [53] (Figure 2.6). This gap is especially pronounced in agriculture and forestry: reviews find that the majority of implementations stall at the model or shadow stage, because living plants, weather, and soil are far harder to model than deterministic factory machinery [54, 55].

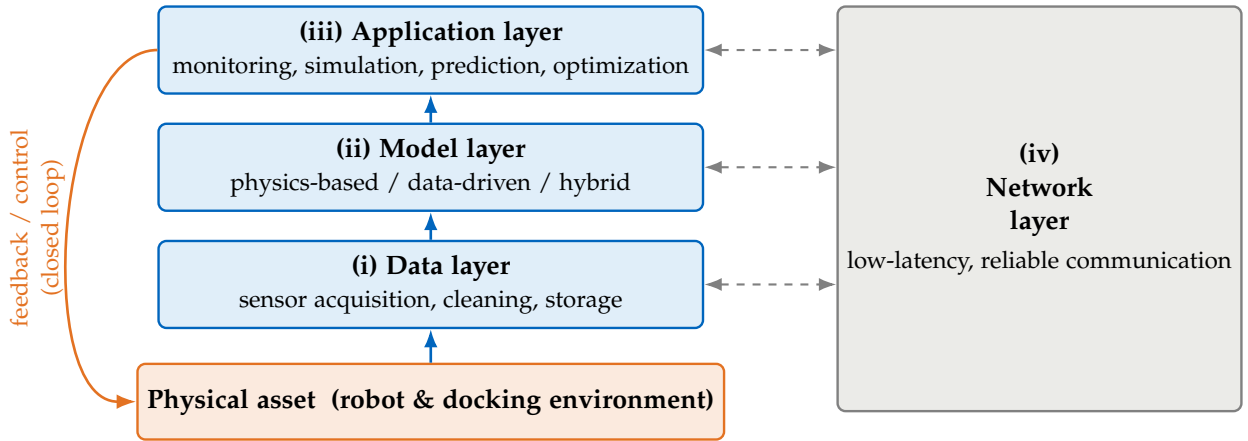


Figure 2.6.: The four constituent layers of a digital twin [53]. Physical state flows upward through the *data*, *model*, and *application* layers (blue), while the *application* layer’s outputs—predictions, optimized parameters, commands—are fed back to the physical asset (orange), closing the bidirectional loop that distinguishes a digital twin from a digital shadow. The *network* layer provides the low-latency communication that couples all layers.

For the present work the digital twin is relevant only as the predictive substrate of a deliberative layer: a continuously updated model of the robot and its docking environment lets the planner perform *prospection*—projecting battery depletion along a route or rehearsing a docking approach—before committing the physical robot. This thesis builds no such twin; the concept is surveyed because it frames the cloud-coupled agronomic simulation discussed conceptually in Chapter 3 and because it clarifies what the resource-aware deliberative layer would ultimately require.

Crop-Growth Models: Simulating the Living Field

If the digital twin is the predictive substrate, its *model layer* is precisely what is hardest to supply in an agricultural setting: the living plant. The factory machinery for which digital twins were originally conceived is governed by deterministic, well-characterised dynamics, whereas a field couples weather, soil water and nitrogen, and plant physiology in a stochastic, slowly-evolving system that no first-principles physics engine captures off the shelf—which is why the agricultural reviews cited above find most implementations

stalled at the model or shadow stage [54, 55]. The engine that fills this gap is the *process-based crop-growth model*, a class of agronomic simulators developed over four decades that integrate, typically on a daily time step, the canonical sub-processes of phenological development, light interception, photosynthesis, respiration, dry-matter partitioning, and soil water and nutrient balances to forecast biomass, yield, leaf-area index, and water demand from weather, soil, and management inputs. Three such models are representative of what the agronomic “brain” of Chapter 3 would be built on.

WOFOST (WORLD FOOD STUDIES), maintained by Wageningen University & Research and the European Commission’s Joint Research Centre since the 1980s, is a point-based crop model that simulates a single homogeneous canopy from daily weather and soil data [56]. Its defining abstraction is a hierarchy of *production levels*—*potential* (growth limited only by radiation, temperature, and CO₂), *water-limited/attainable* (adding the constraint of rain-fed water supply), and *reduced* (further depressed by nutrients, pests, and weeds)—which lets a planner reason explicitly about the gap between what a plot could yield and what it will yield under a given intervention. A recent reimplementation, *diffwofost*, ports the model to PyTorch so that its dynamics are end-to-end differentiable [57]; this is directly relevant to the deliberative layer, because gradient-based calibration and data assimilation let the model be fitted continuously to the robot’s sparse field measurements and propagate the explicit uncertainty that the cloud loop of Section 3.2 requires.

STICS, developed by INRAE, is a generic soil–crop model parameterised for a broad range of arable and forage species; on a daily step it simulates crop growth alongside one-dimensional vertical fluxes of water and nitrogen and the soil organic-matter cycle, and it explicitly represents management interventions such as fertilisation, tillage, and irrigation [58]. Its genericity and validated soil sub-models make it a common foundation on which more specialised systems are built.

Hi-sAFe is one such system, and the engine named in the conceptual architecture of this thesis. It addresses a limitation shared by WOFOST and bare STICS—both treat the canopy as horizontally homogeneous and cannot represent the spatial tree–crop interactions that dominate agroforestry. Hi-sAFe couples an instance of the STICS crop model, run per cell, to a new three-dimensional tree model (*sAFe-Tree*) on a shared daily time step, discretising the scene into a voxel grid and resolving above- and below-ground competition for light, water, and nitrogen, together with the plant plasticity that governs how trees and intercrops redistribute growth in response to that competition [24, 59]. It explicitly models the management levers an autonomous robot would actuate—irrigation, fertilisation, pruning, thinning—making it a natural fit for a system whose missions are exactly those interventions.

Mapped onto the Kritzinger taxonomy, all three are *Digital Models* as distributed: rich offline simulators with no automated coupling to a physical field. What the cloud architecture of Chapter 3 adds is the missing data flow—assimilating the robot’s per-plot measurements to update model state, and feeding the model’s forecasts back as prioritised missions—which is the step that would elevate such an engine from an isolated simulation to the predictive twin that closes the observation–planning–execution loop.

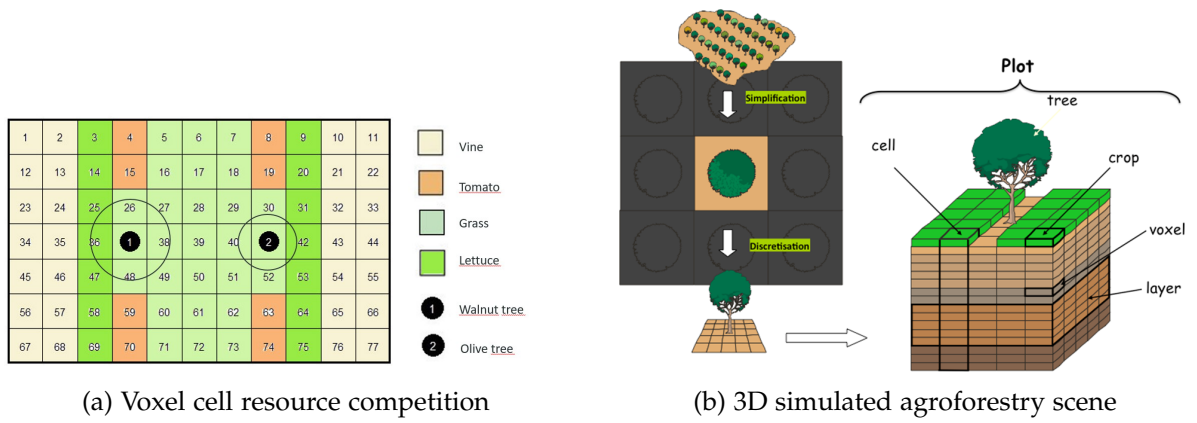


Figure 2.7.: The Hi-sAFe biophysical model and simulation scene. (a) Daily crop growth is simulated per voxel cell, resolving below- and above-ground competition. (b) A 3D simulated agroforestry scene showing tree–crop spatial interactions. Figure adapted from [60].

2.3. Comparative Analysis of Sensing Methods

The central sensing problem of autonomous docking is relational: a robot must determine its own pose in the coordinate frame of a known docking station. This section builds a structured understanding of that problem by decomposing it along three independent axes. First, the *foundational principles* describe how any measured signal—regardless of its physical nature—is processed mathematically to yield a position or orientation estimate. Second, the *localization topologies* categorize the origin of the position-relevant signal, which determines system architecture, power budget, and failure modes. Third, the *sensor modalities* survey the physical hardware available, evaluating each against key performance dimensions relevant to outdoor docking. A synthesis evaluation matrix at the close of this section integrates all three axes into a unified comparison across sensors.

The transition from global navigation to terminal connector alignment spans roughly three orders of magnitude in required precision: from tens of meters down to sub-centimeter. Figure 2.8 illustrates how distinct sensor constellations address each operational phase of this maneuver.

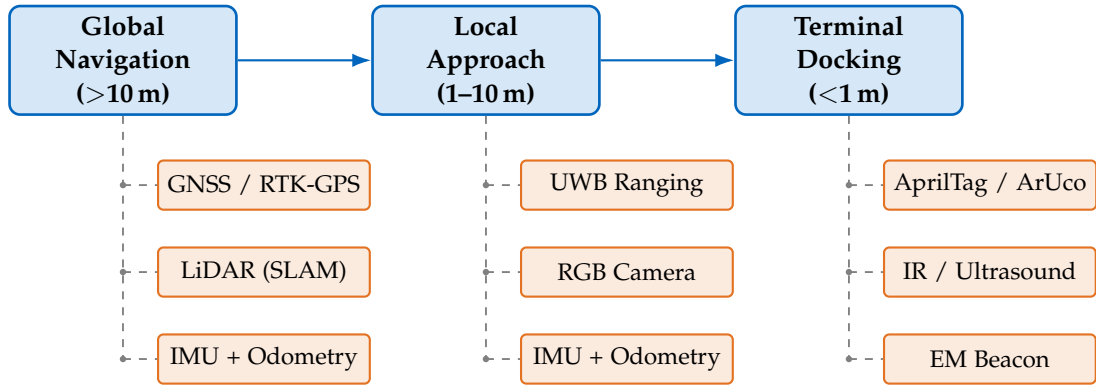


Figure 2.8.: Sensor modalities organized by docking phase. Each phase requires progressively higher precision at shorter range, necessitating complementary sensor constellations. Dots mark the junction between the vertical spine and each sensor branch.

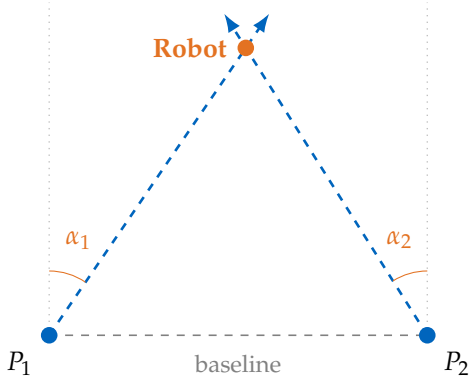
2.3.1. Foundational Principles of Relative Localization

Regardless of the physical signal or hardware used, all localization methods share a common pipeline: a raw sensor measurement is post-processed through a geometric or statistical algorithm to yield a position or pose estimate. This subsection surveys the canonical processing principles that underpin the sensor systems discussed later; the geometric and signal-based ranging principles below follow the standard treatments in the positioning literature [61, 62, 63].

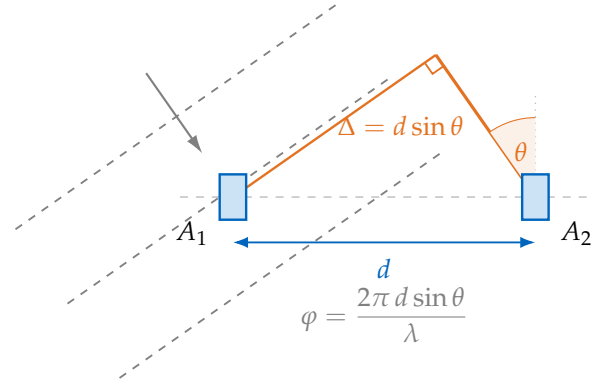
Triangulation (AoA). When a directional sensor or an antenna array measures the bearing angle α_i at which a signal arrives from a known reference point P_i , the robot's position lies

on the ray projected from P_i at angle α_i . With measurements from at least two non-collinear reference points, the position is determined as the intersection of two such rays—a technique known as triangulation. The achievable accuracy depends on the angular resolution of the sensor and the baseline between reference points; at short range, small angular errors translate to small position errors, whereas at long range, the same angular error produces large lateral deviations [61]. Antenna arrays realize Angle of Arrival (AoA) estimation by computing the phase difference of a signal received at spatially separated elements [62]. The same geometry applies to a purely visual setup: when two cameras with a known baseline both observe a common visual feature on the docking station, each camera measures the bearing to that feature directly from its pixel coordinate and intrinsic calibration (the angle between the optical axis and the ray through the feature pixel). The two bearings define two rays whose intersection fixes the feature’s position relative to the camera pair—the optical analogue of antenna-array AoA, and the principle underlying stereo triangulation. This is also the principle upon which the 3D depth camera used in this work bases its 3D detection.

FEEDBACK (supervisor), PARTIALLY DONE: (a) “Robot” label moved to the left of the point — DONE. (b) Still open, supervisor shorthand needs clarification: on the phase-offset sub-figure “add the speed-of-light annotation (c), add the missing symbol on the right, mark what is occluded, remove the stray element near the angle θ .” Unclear which symbol/element is meant — confirm with supervisor before editing.



(a) AoA ray intersection.



(b) Antenna-pair phase offset.

Figure 2.9.: Angle-of-arrival localization. Triangulation estimates the receiver position from the intersection of bearing rays (left), while antenna-array AoA derives those bearings from phase differences between spatially separated receiver elements (right).

Trilateration and Multilateration (Range and Time). Whereas triangulation works from measured *angles*, range-based methods work from measured *distances*, exploiting the known relationship between signal propagation time and distance [61]. Time of Flight (ToF) measures the round-trip or one-way travel time of a signal pulse between a reference point and the

robot; with the known propagation speed c , the range $r = c \cdot \Delta t / 2$ describes a sphere (in 3D) or circle (in 2D) of candidate positions centred on that reference point. A single range therefore constrains the robot to a sphere; *trilateration* recovers the unique position as the intersection of three or more such spheres from reference points at known locations, which is why at least three well-separated anchors are required for an unambiguous 3D fix [62]. Time Difference of Arrival (TDoA) avoids the need for a synchronized clock on the robot: instead of an absolute travel time, it uses the *difference* in arrival times of signals from multiple synchronized transmitters. Each pairwise difference defines a hyperboloid of constant range difference, and the robot's location is found at the intersection of the resulting hyperboloids—a relaxation that makes large anchor networks practical because only the anchors, not the mobile tag, must share a clock [62, 63]. Received Signal Strength Indicator (RSSI) instead infers range from how much a signal weakens with distance, using the Friis path-loss model, which relates received power to distance via $P_r \propto d^{-n}$ where n is the environment-dependent path-loss exponent [61]. RSSI-based ranging is computationally trivial and needs no special hardware, but because received power is corrupted by multipath fading, antenna orientation, and environmental absorption, it yields only meter-level accuracy at best and is unsuitable for precise docking [63].

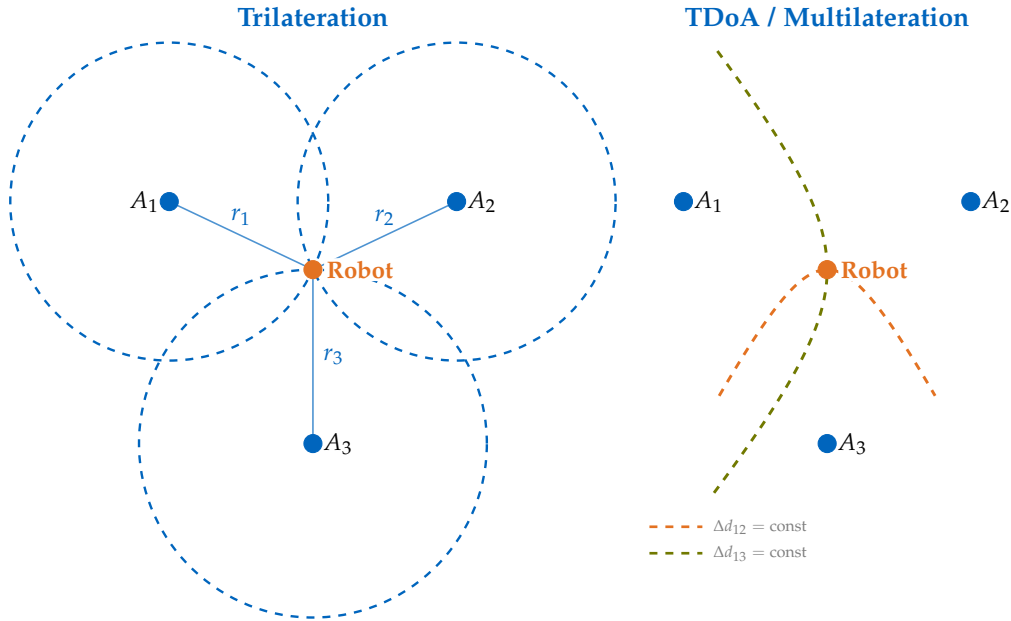


Figure 2.10.: Range-based localization. Trilateration (left) determines position from the intersection of range circles around known anchors. TDoA multilateration (right) replaces circles with hyperbolae defined by time-difference-of-arrival constraints; their intersection uniquely identifies the target.

Phase and Frequency Shifts. Coherent signal sources allow sub-wavelength distance resolution via phase comparison [62]. If the phase of a received sinusoidal signal is $\phi = 2\pi d / \lambda$,

measuring ϕ yields the fractional-wavelength distance; combined with coarse ToF ranging to resolve the 2π phase ambiguity, centimeter-level accuracy is achievable [61]. The Doppler effect encodes relative radial velocity as a frequency shift $\Delta f = 2v/\lambda$ (for reflected signals) or $\Delta f = v/\lambda$ (for one-way), which radar and ultrasound systems exploit for velocity estimation [62]. Figure 2.11 illustrates both effects.

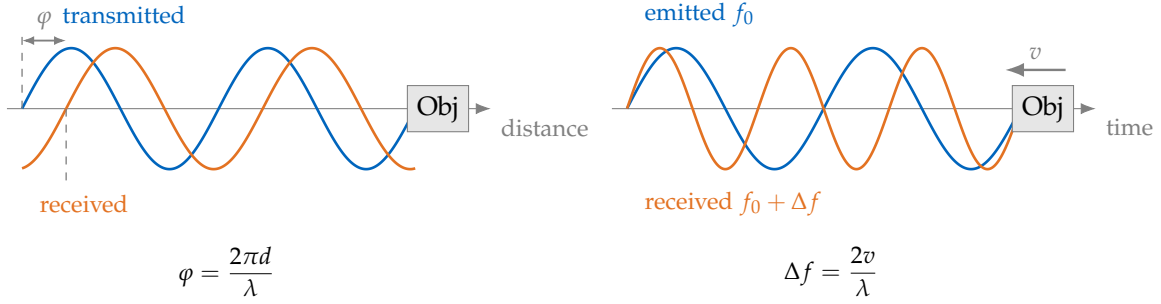


Figure 2.11.: Phase- and frequency-based ranging. **Left:** a received sinusoid is delayed relative to the transmitted reference by a phase offset $\varphi = 2\pi d/\lambda$, from which the sub-wavelength distance is recovered. **Right:** relative motion of the target compresses the returned waveform, producing a Doppler frequency shift $\Delta f = 2v/\lambda$ that encodes radial velocity.

Feature Matching. When the geometry or appearance of the docking station is at least partially known in advance, localization reduces to *matching* the current observation against a stored reference. The methods are best organized by *what* is matched, forming three families of increasing abstraction: matching raw geometry directly, matching hand-crafted features, and matching learned features. All three are viewpoint sensitive and degrade when the docking station is partially occluded or when the sensor’s view angle differs substantially from the reference.

(1) Direct geometric matching (point cloud registration). When a 3D sensor such as LiDAR or a depth camera observes a structured environment, its raw output can be registered *directly* against a geometric model or a reference point cloud of the docking station, without first extracting discrete features. Point cloud registration aims to find the optimal rigid body transformation $(R, t) \in \text{SE}(3)$ (where $R \in \text{SO}(3)$ is the rotation matrix and $t \in \mathbb{R}^3$ is the translation vector) that aligns a source/measured point cloud $P = \{p_1, p_2, \dots, p_{|P|}\}$ with a target/reference point cloud $Q = \{q_1, q_2, \dots, q_{|Q|}\}$. In the context of autonomous docking, particularly in unstructured outdoor environments, this process relies on several key algorithms to handle outliers, achieve fine alignment, and manage local minima:

- **Random Sample Consensus (Random Sample Consensus (RANSAC)):** Raw point clouds in outdoor docking environments are heavily corrupted by clutter (grass, soil, moving leaves) that acts as outliers. RANSAC [64] is a robust estimator that extracts

geometric primitives—the flat plates, backplates, or pillars of the docking station—from this clutter by repeatedly fitting a model to a random minimal subset of points (e.g. three points for a plane) and keeping the model with the largest consensus set of inliers, i.e. points within a perpendicular distance ϵ . Fitting planes to the dock structure and discarding non-matching points isolates the clean geometry for the subsequent fine-registration phases. Figure 2.12 illustrates this consensus thresholding.

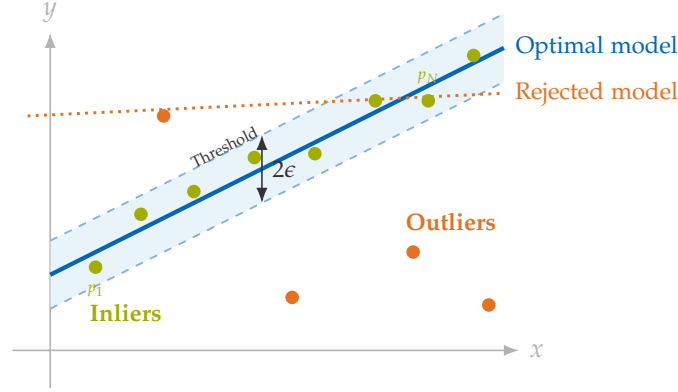


Figure 2.12.: Conceptual overview of the RANSAC algorithm for geometric model fitting. Outliers (orange) are discarded, and an optimal model (blue) is fitted using the inliers (green) within a consensus threshold band of 2ϵ .

- **Iterative Closest Point (Iterative Closest Point (ICP)):** Once a coarse alignment is established and outliers are minimized, the ICP algorithm [65, 66] is used to achieve fine-scale alignment. In its standard *point-to-point* formulation, the algorithm minimizes the sum of squared Euclidean distances between corresponding points:

$$E_{\text{point-to-point}}(R, t) = \sum_{i=1}^{|P|} \|Rp_i + t - q_{c(i)}\|^2 \quad (2.1)$$

where $q_{c(i)} \in Q$ is the closest point in the target cloud to the transformed source point p_i , computed as:

$$c(i) = \arg \min_j \|Rp_i + t - q_j\| \quad (2.2)$$

The algorithm alternates between two main steps: (1) finding the closest point correspondences under the current pose estimate, and (2) solving for the transformation (R, t) that minimizes the objective function, typically solved in closed form using Singular Value Decomposition (SVD) on the cross-covariance matrix. The process iterates until the change in error or the update step falls below a predefined tolerance (conceptually illustrated in Figure 2.13).

A key variant is *point-to-plane* ICP [66], which minimizes the distance along the surface

normal $n_{c(i)}$ of the target point:

$$E_{\text{point-to-plane}}(R, t) = \sum_{i=1}^{|P|} \left((Rp_i + t - q_{c(i)}) \cdot n_{c(i)} \right)^2 \quad (2.3)$$

For docking stations, which are typically constructed from planar metal or plastic plates, point-to-plane registration is significantly more robust. It allows points to slide along the planar faces of the dock rather than forcing them to align with arbitrary discrete points, accelerating convergence and reducing the probability of getting stuck in local minima.

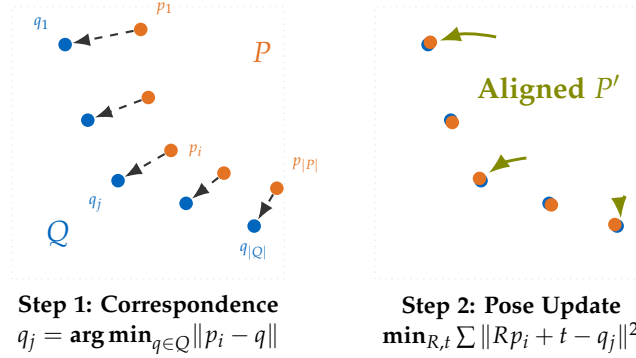


Figure 2.13.: Conceptual overview of the Iterative Closest Point (ICP) algorithm. **Left:** Step 1 searches for the closest target points $q_j \in Q$ for each source point $p_i \in P$. **Right:** Step 2 computes the optimal transformation (R, t) that minimizes alignment error, bringing the transformed source cloud P' into alignment with target cloud Q .

- **Normal Distributions Transform (Normal Distributions Transform (NDT)):** Standard ICP relies on point-to-point correspondence search, which is costly and sensitive to point-density variation and partial occlusion. NDT [67, 68] instead voxelizes the target cloud and models the points in each cell as a local Gaussian (mean μ , covariance Σ), turning the target into a continuous, differentiable probability density field (Figure 2.14). Registration then finds the rigid transformation (R, t) that maximizes the likelihood of the transformed source points under these distributions. Because the objective is smooth, it is optimized with gradient-based methods without nearest-neighbour search, handling noise and variable density gracefully and offering a larger convergence basin than ICP.

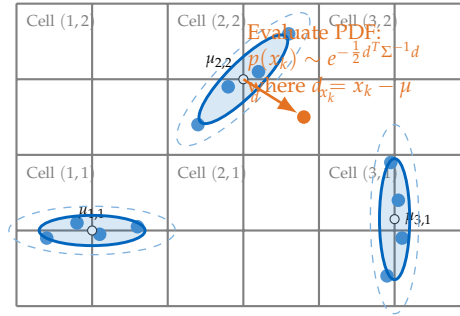


Figure 2.14.: Conceptual overview of the Normal Distributions Transform (NDT) algorithm. The target point cloud is voxelized into cells, and the points inside are represented as a local Gaussian probability density function (mean μ , covariance Σ , represented by ellipses). Source points x_k are registered by maximizing their likelihood on this continuous PDF.

- **Global Feature Matching (Fast Point Feature Histograms (FPFH)):** Since both ICP and NDT rely on local optimization, they require a reasonable initial pose estimate to prevent convergence to incorrect local minima. When no prior pose is available, global registration methods are used. A canonical example is FPFH [69], which describes local surface geometry around a point by computing multi-dimensional histograms of relative angles (angles between surface normals of neighboring points). By matching FPFH descriptors between the source and target clouds, a set of initial correspondences is established to estimate a coarse alignment transformation, which is subsequently refined using ICP or NDT.

that figure 2.13 needs to be fixed by me manually

Both registration and feature matching require sufficient geometric structure in the scene (such as corners, edges, or planes) and fail at very close range when the sensor enters its blind zone.

(2) Matching hand-crafted features (fiducial markers and classical detectors). Instead of the raw signal, this family matches engineered features whose form is fixed by a human designer. Two variants are relevant to docking. Classical image detectors—Harris corners, FAST, SIFT, and ORB—extract repeatable keypoints that are matched against a reference model or prior map [70]. More robust at the terminal docking phase are **visual fiducial markers** whose geometry is precisely known: systems such as AprilTags and ArUco encode a unique identity in a binary square pattern, and a single camera image suffices to compute the full six degree-of-freedom (6-DOF) pose via the Perspective- n -Point (PnP) algorithm (detailed below), which solves for the rigid transformation mapping the known marker corners to their observed image positions. Abbas et al. showed that with trigonometric correction for camera yaw, positional accuracy well below 1 cm is achievable at docking distances of 0.5–1 m [71].

(3) Matching learned features (deep learning). The most recent family replaces hand-crafted descriptors with features *learned* from data. Deep-learning-based object detection and pose estimation offer a marker-free alternative that can recognize the docking station from its natural appearance, but require large annotated training datasets and are computationally heavier than the previous two families.

Perspective- n -Point (PnP). When the relative pose of a calibrated camera must be determined from a set of known 3D points in the world coordinate frame and their corresponding 2D projections in the image plane, the localization task is formulated as the Perspective- n -Point (PnP) problem [62]. Given a set of n points with coordinates expressed in the object or world coordinate frame $P_i = [X_i, Y_i, Z_i]^T$ and their observed image coordinates $u_i = [u_i, v_i]^T$, the objective is to estimate the rigid body transformation $(R, t) \in \text{SE}(3)$ (where $R \in \text{SO}(3)$ is the rotation matrix and $t \in \mathbb{R}^3$ is the translation vector) that maps the 3D points to the 2D image plane. Under the pinhole camera projection model, this mapping is expressed as:

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = K \begin{bmatrix} R & t \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} \quad (2.4)$$

where K is the camera's intrinsic calibration matrix and s_i is a scale factor representing depth. For visual fiducial marker systems such as AprilTags, the physical geometry of the marker is known a priori (e.g., the coordinates of the four corners in the tag's local frame, where $n = 4$). Detecting the tag in the image yields the corresponding sub-pixel corner projections. Solving the PnP problem (specifically the planar P4P variant for coplanar point sets) yields the rotation R and translation t that define the 6-DOF pose of the camera relative to the tag. Since minimal solvers are sensitive to noise and can produce multiple geometrically valid solutions, the initial pose is typically refined by minimizing the reprojection error over all detected points using non-linear optimization techniques such as the Levenberg-Marquardt algorithm:

$$E_{\text{reproj}}(R, t) = \sum_{i=1}^n \|u_i - \pi(P_i, K, R, t)\|^2 \quad (2.5)$$

where π is the perspective projection function. This estimation provides the high-precision relative pose necessary to guide the robot during terminal docking alignment.

2.3.2. Localization Topologies

Beyond the mathematical processing principle, the *source* of the position-relevant signal fundamentally determines system complexity, energy budget, range, and failure modes. Four distinct topologies arise for docking scenarios; Figure 2.15 provides a visual overview.

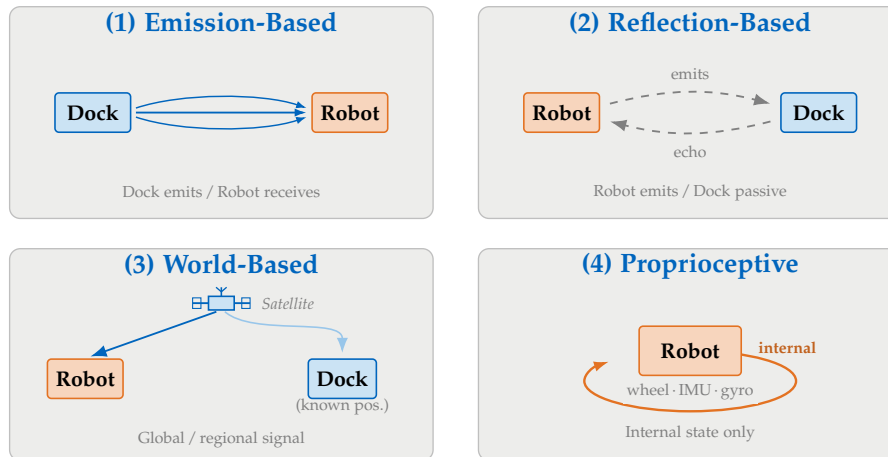


Figure 2.15.: The four localization topologies for autonomous docking. Emission-based and world-based topologies place the signal source outside the robot; reflection-based places the active emitter on the robot; proprioceptive requires no external signal at all.

FEEDBACK (supervisor): figure moved up to here as requested ("I would like Figure 2.9 to be here, not down there"). The supervisor also noted he will regenerate this figure himself; pending tweaks: enlarge the circular arrow in the proprioceptive panel, turn the rotated text to read normally, and reposition the "Robot"/internal labels.

Emission-Based Detection

In emission-based detection, the docking station actively emits a signal—electromagnetic, acoustic, or optical—that the robot perceives with an onboard receiver. Examples include Ultra-Wideband (UWB) anchors mounted on the dock transmitting ranging pulses, an Infrared (IR) beacon broadcasting a modulated IR signal, or an electromagnetic coil radiating at a known frequency. Since the station controls the signal, its strength, modulation, and timing can be optimized for maximum detectability at the required range. The robot's receiver is typically a simple, lightweight unit.

The principal disadvantage is the energy cost—and the initial hardware cost—imposed on the docking station: it must contain an active emitter and sustain continuous transmission whenever a robot may approach. Furthermore, emitted signals occupy a physical medium—radio spectrum, acoustic band, optical wavelength—and may interfere with other systems operating in the same environment, be jammed by strong competing sources, or influence the natural habitat in unintended ways (for example, acoustic emissions disturbing wildlife).

Reflection-Based Detection

Reflection-based detection uses signals that bounce off the docking station rather than originate from it. The reflected signal may be generated actively by the robot itself (as in

LiDAR, radar, or ultrasound) or passively by an ambient source (as in a camera observing a visually distinctive dock illuminated by sunlight).

The key architectural advantage is that the robot's own sensors—already present for obstacle avoidance and navigation—double as docking sensors, and the docking station can remain purely passive: no power supply, no electronics, no maintenance beyond keeping its surface reflective or its markers clean. This substantially reduces station cost and complexity.

However, reflection-based ranging carries an inherent range penalty. For active-emission-on-robot schemes, the signal must travel to the dock and back, incurring $4\pi d^2$ spreading losses twice (two-way path-loss versus one-way for emission-based). For passive ambient illumination (e.g., a camera and sunlight), the signal source cannot be amplified or modulated, limiting detection robustness under overcast conditions, heavy vegetation shadow, or dust haze. Specular reflections from flat metallic or wet surfaces can further produce false returns or shadow zones.

World-Based Positioning

World-based positioning exploits signals emitted by natural or infrastructure sources that are global or regional in scope: satellite constellations (Global Navigation Satellite System (GNSS)), the Earth's magnetic field, or celestial objects (sun trackers, star trackers). Unlike emission- or reflection-based methods, the robot does not need to perceive the docking station at all—it localizes itself in a global or regional reference frame, and the dock's known position in that same frame yields the relative pose.

What makes a world signal suitable for stable positioning? Three properties are critical. *Global availability*: the signal must be present at any location where the robot operates, without the need for local infrastructure. *Predictability and stability*: the signal must vary slowly or predictably, so that a calibration performed once remains valid over operational timescales. *Signal-to-noise robustness*: the signal must be detectable above background noise even under adverse conditions.

GNSS (GPS) achieves the first two properties but fails the third under dense tree canopies, where satellite visibility drops, causing multipath errors and ultimately signal loss. The Earth's magnetic field satisfies the first and second properties in open terrain but is disrupted locally by ferromagnetic robot components and high-current drive cables. Celestial navigation (sun/star tracking) requires unobstructed sky view and fails in overcast conditions. All world-based signals therefore serve best as coarse global anchors, fused with finer-resolution relative sensors rather than used alone [32].

Proprioceptive Positioning

Proprioceptive positioning uses only measurements of the robot's internal state—wheel velocity, angular rate, linear acceleration—to estimate the robot's own pose by dead reckoning. Its principal advantage is that it relies on nothing outside the robot: there is no external signal to lose, occlude, or jam, so a proprioceptive estimate is always available regardless of environmental conditions. Its very high update rate (100–1000 Hz), which bridges the gap

between slower exteroceptive updates, is a valuable additional benefit rather than the main one.

For dead reckoning to be useful over extended periods, the integrated signals must satisfy two requirements. First, the measurement noise should be *zero-mean* (or at least characterizable), so that random errors average out and do not cause runaway drift. Second, systematic biases—wheel radius mismatch, encoder eccentricity, gyroscope bias—must be identified during calibration and compensated. In practice, proprioceptive drift is bounded only over short time horizons; over minutes of operation, position errors grow to meters. Proprioceptive sensing therefore acts as a high-bandwidth prediction layer within a sensor fusion architecture, not as a standalone localization source [72].

2.3.3. Sensor Modalities

Given the localization principles and topologies above, this subsection surveys the concrete sensor hardware available. For each sensor type, the discussion covers: theoretical detection accuracy; susceptibility to interference and jamming; propagation behaviour in air; reflection characteristics on typical docking-station surfaces; noise from background radiation and sunlight; and the influence of weather (rain, hail, snow, intense sun). The emission-based, reflection-based, and world-based applicability of each sensor is evaluated explicitly.

Electromagnetic Waves

Electromagnetic sensors span a wide wavelength range, from centimetre-wave GNSS receivers through millimetre-wave radar, UWB radio, near-IR lasers, and visible cameras, to far-IR thermal imagers. Each wavelength band interacts differently with the atmosphere and with surfaces, leading to strongly differentiated performance profiles.

GNSS and Real-Time Kinematic GPS. The GNSS (GNSS) encompasses constellations such as GPS, Galileo, and GLONASS, which transmit L-band signals (1.2–1.6 GHz) from orbiting satellites. A receiver determines its absolute position by measuring pseudoranges to at least four visible satellites, yielding 3–5 m accuracy under open sky. Real-Time Kinematic (RTK) (RTK) augmentation uses carrier-phase differentials between a fixed base station at a known position and the rover receiver, achieving 1–3 cm horizontal accuracy at update rates of 5–10 Hz [32].

RTK-GPS operates exclusively in a *world-based* topology: the receiver localizes itself in a global coordinate frame, and the dock's known coordinates in the same frame yield the relative pose. The principal limitation in agroforestry environments is satellite visibility: under dense tree canopies the constellation drops below the minimum of four required satellites, causing complete signal loss, and partial canopy causes multipath reflections from trunks that corrupt pseudoranges. RTK also requires continuous telemetry from a base station within approximately 10 km. It is therefore reliable for open-field global navigation but unsuitable for terminal docking under vegetation.

RGB Cameras. Cameras sensitive to visible light (400–700 nm) are the most versatile and cost-effective sensing platform in robotics. In the context of docking, they are almost exclusively used in a *reflection-based* topology: ambient light (solar or artificial) illuminates the dock, and the camera captures the reflected image. The theoretical localization accuracy through PnP pose estimation with fiducial markers is well below 1 cm at 0.5–1 m range [71].

The principal outdoor limitation is the extreme dynamic range of solar illumination. Direct sunlight on a scene can overexpose the camera, washing out marker contrast; conversely, deep shadow may render features undetectable. Rain droplets and dust on the lens degrade image quality substantially, while vegetation may intermittently occlude the marker. Background solar radiation covers the visible spectrum and limits the signal-to-noise ratio for any feature that is not significantly brighter or darker than its surroundings. Weather effects (rain, snow, fog) attenuate light in the visible band, reducing usable range. There is no viable world-based positioning signal in the visible spectrum at the scale of a docking maneuver.

Depth Cameras. Depth cameras augment RGB with per-pixel range information, enabling direct 3D reconstruction of the scene. Structured-light cameras (e.g., Intel RealSense [73]) project an IR pattern and measure its deformation; ToF depth cameras measure per-pixel phase shifts of a modulated IR signal. Both operate at near-IR wavelengths (800–940 nm). Structured-light cameras achieve ± 2 mm accuracy at 0.3–4 m in controlled indoor conditions, but fail completely in direct sunlight, because solar radiation in the near-IR overwhelms the projected pattern. ToF depth cameras are similarly limited outdoors by solar noise. They are therefore most useful as a terminal-phase complement to RGB cameras in shaded or indoor-outdoor transition zones. Depth cameras operate exclusively in a reflection-based topology.

LiDAR. LiDAR sensors emit pulsed laser beams—typically at 905 nm or 1550 nm—and measure ToF of returned pulses to construct high-resolution point clouds [74]. Modern multi-layer rotating LiDARs cover a full 360° azimuth at 10–20 Hz with ranging accuracy of 2–4 cm at distances up to 200 m.

LiDAR operates in a *reflection-based* topology: the robot emits laser pulses that reflect off the dock structure. Its insensitivity to ambient illumination changes makes it the backbone sensor for Simultaneous Localization and Mapping (SLAM) in outdoor agricultural environments [75]. At 905 nm, solar background radiation introduces noise that limits detection in high-sun conditions; 1550 nm systems are significantly less affected because solar irradiance is lower in that band, but the laser source is more expensive and eye-safety considerations differ. Rain, snow, and dense fog scatter and absorb near-IR pulses, degrading both range and point-cloud density. Retroreflective tape on the docking station strongly enhances return signal, extending reliable detection range and improving accuracy.

Below approximately 30 cm, most commercial LiDARs enter an optical blind zone; at close range, point clouds are too sparse to resolve fine docking features at sub-centimeter scale. LiDAR-based point cloud matching (ICP, FPFH) is therefore appropriate for the local approach phase rather than terminal alignment.

Radar. Millimetre-wave radar (24 GHz short-range or 76–81 GHz automotive FMCW) transmits and receives electromagnetic waves in the millimetre-wave band. By measuring frequency shifts of the chirped return (FMCW) or the arrival time, radar derives both range (~ 10 cm accuracy at 50 m) and radial velocity via Doppler. Radar operates exclusively in a *reflection-based* (active-on-robot) topology.

The outstanding advantage of radar is its weather robustness: millimetre waves penetrate rain, fog, snow, and dust with negligible attenuation, making radar the only electromagnetic modality that maintains consistent performance under severe weather. Metal surfaces (typical on docking stations) reflect radar strongly; non-metallic surfaces have lower radar cross-section, requiring a corner-reflector or metal patch on the dock. Ground returns create multipath interference, particularly on flat agricultural terrain, which complicates close-range positioning. No useful world signal exists in the radar bands for relative localization.

Ultra-Wideband (UWB) Radio. UWB radio transmits sub-nanosecond impulse bursts spanning 3–10 GHz with fractional bandwidth exceeding 20%. This extreme bandwidth enables Time-of-Flight ranging with typical accuracy of 5–15 cm using time-stamped two-way ranging (TWR) between anchor nodes at known positions (mounted on or near the docking station) and a tag on the robot [32]. TDoA with synchronized anchors allows simultaneous localization of multiple tags; update rates of 50–100 Hz are achievable with current chip implementations.

UWB operates in an *emission-based* topology: the anchors broadcast ranging pulses that the robot's tag receives (or vice versa). Its principal advantage over optical methods is robustness to ambient illumination and moderate non-line-of-sight penetration through vegetation—unlike RSSI-based Wi-Fi positioning, UWB derives range from time-of-flight rather than signal strength, making it insensitive to power fluctuations and antenna orientation. The infrastructure cost is modest (typically four to six anchors per docking area), but anchors must be calibrated to known positions and powered continuously. Because UWB yields range and bearing rather than full six-DOF pose, it complements camera-based fiducial detection: UWB narrows the robot's positional uncertainty during the local approach phase (1–10 m), enabling reliable fiducial detection once the robot is close enough.

Thermal Camera. Far-IR cameras ($8\text{--}14\ \mu\text{m}$) image the thermal emission of objects rather than reflected light. They are unaffected by ambient illumination and can penetrate light fog and smoke. In a docking context, powered electronics (motor drivers, battery management) on the dock generate waste heat that creates a distinctive thermal signature, enabling *emission-based* detection—the dock's own thermal radiation is the signal source. However, thermal cameras offer no geometric depth information, and their spatial resolution is typically much lower than visible cameras ($\sim 320 \times 256$ pixels for uncooled microbolometer arrays). Localization accuracy via thermal feature matching is insufficient for sub-centimeter terminal alignment but may be useful for coarse approach guidance. Intense direct solar radiation can heat surfaces differentially, creating thermal clutter that masks the dock's heat signature.

Infrared Proximity Sensors. Short-range IR proximity sensors emit modulated near-IR light and detect the reflected intensity from nearby objects. They provide a simple, high-frequency (up to 1 kHz) binary or analog proximity signal at ranges below ~ 1 m. They operate in a *reflection-based* topology, require no computation, and are immune to ambient light when using modulated emission with narrow bandpass filters. They do not provide full 6-DOF pose information and are therefore used exclusively as proximity confirmation at the very end of the docking sequence, verifying that the connector has reached the dock before engaging the electrical coupling [76].

Acoustic Sensors

Acoustic sensors encode spatial information in pressure waves rather than electromagnetic radiation. For robotics, the relevant frequency range spans from audible frequencies (< 20 kHz) up to the low ultrasonic range (~ 200 kHz), with higher frequencies offering better directivity and range resolution at the cost of increased atmospheric attenuation.

Ultrasonic Sensors. Piezoelectric transducers emit ultrasonic pulses (typically 40–200 kHz) and measure the ToF of the echo reflected from nearby surfaces. Range accuracy is approximately 1 cm at distances of 0.02–4 m, depending on transducer frequency and signal processing. Ultrasound operates in a *reflection-based* topology (active-on-robot), with the same two-way path-loss penalty as LiDAR.

Critically, the speed of sound in air varies with temperature ($c \approx 331 + 0.6T$ m/s with T in $^{\circ}\text{C}$), requiring temperature compensation for accurate ranging in outdoor environments where air temperature fluctuates by 20–40 K over a day. Wind disturbs the propagation path of acoustic waves, introducing additional ranging errors. Rain and snow attenuate ultrasound modestly but more significantly than electromagnetic waves. The broad emission cone (typically $15\text{--}30^{\circ}$) of standard transducers introduces angular ambiguity: the first echo may return from any surface within the cone, not necessarily the intended target. Focused transducer arrays or phased-array designs narrow the beam and enable AoA estimation in addition to ranging, but at higher complexity. Outdoor sun and thermal background radiation have negligible effect on acoustic sensors, giving them an advantage in high-sun conditions where optical sensors struggle. No world-based acoustic signal is available for docking-relevant localization.

Magnetic Fields

Magnetic sensing encompasses both passive measurement of the Earth’s static field and active detection of fields emitted by a purposely designed coil system.

Magnetometer / Compass. A three-axis magnetometer measures the local magnetic flux density vector. In open terrain far from magnetic disturbances, this vector aligns with the Earth’s geomagnetic field ($25\text{--}65\ \mu\text{T}$ depending on latitude), providing heading information after correcting for the local magnetic declination. Heading accuracy is typically $0.5\text{--}2^{\circ}$ after

hard-iron and soft-iron calibration, making the magnetometer a useful global compass for the navigation phase. However, this constitutes *world-based positioning* at the coarse heading level only—it yields no range information and therefore cannot localize the robot relative to the dock. The magnetometer is severely disturbed by hard ferromagnetic components (steel chassis, drive motors) and by alternating magnetic fields from high-current cables. Careful placement away from these noise sources and multi-point calibration are mandatory for usable outdoor performance.

Electromagnetic Beacon. An active EM beacon system places an AC-driven coil at the docking station, generating a time-varying magnetic field at a known frequency. The robot's onboard magnetometer or dedicated search coil detects the gradient and direction of this field, enabling 3D localization of the dock via magnetic dipole field equations. By operating at a frequency distinct from DC (Earth's field) and from power electronics switching frequencies, the beacon signal can be separated from interference. Gutnik and Groper demonstrated this approach for Autonomous Underwater Vehicle (AUV) terminal docking, achieving 3D localization accuracy below 3 cm at ranges up to ~ 2 m without line-of-sight [77]. This constitutes an *emission-based* topology: the dock emits, the robot receives. The primary engineering challenge is shielding the robot's sensitive magnetometer from its own motor currents and power electronics, which generate competing magnetic fields. Weather has no effect on magnetic field propagation; the EM beacon is operable in rain, snow, fog, or darkness without any degradation. Different coil geometries—single coil, orthogonal triaxial coil, phased coil arrays—and modulated signal patterns offer trade-offs between detection range, angular resolution, and disambiguation of the approach direction.

Proprioceptive Sensors

Proprioceptive sensors measure internal robot state without any external signal. They yield relative pose increments rather than absolute positions and are therefore used exclusively in the proprioceptive topology.

Inertial Measurement Unit (IMU). An Inertial Measurement Unit (IMU) integrates a three-axis accelerometer and a three-axis gyroscope, measuring linear accelerations and angular velocities along all three body axes at rates of 100–1000 Hz. Integration of the gyroscope signal yields orientation (roll, pitch, yaw); double integration of the accelerometer yields velocity and position.

Modern MEMS-based IMUs are compact (a few mm²), low-cost (<10 EUR for consumer grade), robust to all weather conditions, and produce zero-latency data. Their theoretical angular resolution is well below 0.01°/sample at rated bandwidth, but the practical limit is set by bias instability: a small constant offset in the gyroscope signal that, when integrated over time, causes the heading estimate to drift at rates of 0.5–10°/hour for MEMS devices (compared to 0.001–0.01°/hour for tactical-grade fiber-optic gyroscopes). Accelerometer bias similarly causes velocity drift that grows quadratically with time; an uncompensated 1 mg

bias error produces 50 cm of position error after only 10 s. Temperature change alters bias characteristics, requiring in-situ thermal compensation.

For outdoor agricultural robots operating on sloped or rough terrain, the IMU serves two indispensable roles. First, its roll and pitch estimates correct the gravitational projection in the odometry equations, preventing systematic errors on inclines. Second, its high-bandwidth angular rate signal provides fast heading correction during sharp turns, where slower exteroceptive sensors cannot keep up. During brief loss-of-lock of external sensors (e.g., GPS signal dropout under canopy), IMU integration sustains a short-term position estimate until the external signal is reacquired. The Extended Kalman Filter (EKF) continuously fuses IMU predictions with periodic corrections from LiDAR, GNSS, and docking sensors to bound the accumulated drift [72].

Wheel Encoders and Odometry. Wheel encoders—typically magnetic or optical, with 100–4096 counts per revolution—measure the angular displacement of each drive wheel. For a differential-drive robot with wheel radius r and axle baseline b , the encoder increments $\Delta\theta_L$ and $\Delta\theta_R$ yield the displacement $\Delta d = r(\Delta\theta_L + \Delta\theta_R)/2$ and the heading change $\Delta\psi = r(\Delta\theta_R - \Delta\theta_L)/b$. This computation—*odometry*—runs at 100–1000 Hz and constitutes the lowest-latency, lowest-cost position estimate available.

The fundamental limitation is wheel slip: on deformable or wet soil, the wheel may rotate without translating, causing the encoder to overcount the actual displacement. This systematic error is unbounded and cannot be distinguished from true motion without an external reference. Additionally, slight errors in the calibrated wheel radius r and baseline b —even 1% each—cause heading drift of $\sim 0.57^\circ$ per meter of travel, equivalent to ~ 10 cm lateral error at 10 m range. Terrain irregularities cause the robot to oscillate in roll and pitch, further corrupting the flat-ground kinematic model.

Despite these limitations, wheel encoders remain essential in all phases of the docking maneuver. Their high update rate and deterministic latency make them the preferred motion-model input to the prediction step of any Kalman-type state estimator, providing a stable backbone onto which all other sensors are fused [72].

2.3.4. Synthesis Evaluation Matrix

Table 2.2 integrates the preceding analysis into a single reference page. Each column represents one sensor modality; rows capture the performance metrics most relevant to outdoor autonomous docking. Qualitative ratings (Very High / High / Medium / Low / None) reflect the relative standing of each sensor on that metric; quantitative figures are indicative for representative commercial hardware. The topology columns (Emission / Reflection / World) use ✓ where the topology is naturally applicable and ✗ where it is not, with a dash where the category does not apply.

Table 2.2.: Synthesis evaluation matrix of sensor modalities for outdoor autonomous docking. Ordinal scale (robustness, cost rows): **0** = None, **1** = Very Low, **2** = Low, **3** = Medium, **4** = High, **5** = Very High. LOS column: **N** = not required, **P** = partial, **Y** = required. Phase column: **G** = Global, **L** = Local, **T** = Terminal, **A** = All. Quantitative figures are indicative for representative hardware. [†]Depth camera accuracy is for controlled indoor conditions only; outdoor performance is effectively none due to solar saturation. [‡]Magnetometer accuracy is heading (angular), not position.

Metric	Wheel Encoders	Drift	—	5	N/A	N	0	—	—	✗	✓	1	1	1 kHz	A
	IMU	Drift	—	5	N/A	N	0	—	—	✗	✓	2	2	1 kHz	A
	Magnetometer	~1° [‡]	Global	3	4	N	2	✗	✗	✓	✗	2	2	100 Hz	G
	EM Beacon	<3 cm	3 m	5	5	N	0	✓	✗	✗	✗	2	2	100 Hz	T
	IR Proximity	~5 mm	1 m	5	4	P	2	✗	✓	✗	✗	1	1	1 kHz	T
	Ultrasound	~1 cm	4 m	4	3	P	0	✗	✓	✗	✗	1	1	50 Hz	T
	Thermal Camera	Feature only	30 m	4	3	P	3	✓	✓	✗	✗	2	4	30 Hz	L
	Radar (FMCW)	~10 cm	300 m	5	4	N	0	✗	✓	✗	✗	3	4	50 Hz	G/L
	Depth Camera	~2 mm [†]	~4 m	1	2	P	5	✗	✓	✗	✗	4	3	30 Hz	T
	RGB + Fiducial	<1 cm	~5 m	2	1	Y	4	✗	✓	✗	✗	3	1	30 Hz	T
	UWB	5–15 cm	50 m	4	4	N	0	✓	✗	✗	✗	2	2	100 Hz	L
	RTK-GPS	1–3 cm	Global	3	4	N	2	—	—	✓	✗	2	3	10 Hz	G
	LiDAR	~2 cm	200 m	3	3	P	3	✗	✓	✗	✗	3	4	20 Hz	G/L
		Typical accuracy	Maximum range	Weather robustness	Occlusion robustness	LOS required	Solar noise	Topology: Emission-based	Topology: Reflection-based	Topology: World signal	Topology: Proprioceptive	Computational cost	Hardware cost	Update rate	Primary docking phase

2.4. Solutions for Resource Management in Robotics

Section 2.3 surveyed how a robot perceives a docking station; this section steps back to the question that makes docking necessary in the first place. As established in the analysis of long-term autonomy, resource depletion—most prominently energy—is one of the principal triggers for human intervention, and self-sustenance is a defining requirement of LTA. Two broad strategies exist to address it: a robot may carry all required resources on board, or it may periodically replenish them at a dedicated station. This section surveys both as they appear in the SOTA.

2.4.1. Robots Carrying All Resources On Board

The first strategy keeps all required resources on the robot itself, avoiding any docking infrastructure. It divides into two regimes that differ fundamentally with respect to LTA: a robot may either carry a *finite* store sized for the mission, or it may continuously *harvest* energy in situ. The distinction matters because a finite store is, by definition, exhausted in finite time. A battery-only platform is therefore inherently mission-bounded: once the charge is spent the robot stops, and no amount of energy-efficient control—such as the predictive dynamic-window planning of Teso-Fz-Betoño et al., which lowers the energy cost of navigation [78]—changes the qualitative outcome, only the time at which it occurs. Battery-electric field robots such as those of Naïo Technologies accordingly operate on a per-shift basis and return to a human-attended charger between shifts [2, 9]. Such platforms are well suited to bounded missions but, on battery alone, cannot meet the persistent, self-sustaining requirement of LTA.

Genuine on-board self-sufficiency therefore requires an energy source that is replenished by the environment rather than depleted—in practice solar, wind, or nuclear. Each is demonstrated at the system level. Solar harvesting dominates in outdoor agriculture: the FarmDroid FD20 carries photovoltaic panels sized to power its daily seeding and weeding directly in the field, sustaining operation without grid charging under favourable conditions [1, 3]. The Saildrone autonomous surface vehicle combines wind propulsion with solar-charged electronics to remain at sea, unsupported, for missions of up to a year while sampling the marine environment [79, 80]. At the extreme, NASA’s Mars Science Laboratory rover Curiosity draws on a Multi-Mission Radioisotope Thermoelectric Generator, whose continuous decay heat has powered the vehicle for over a decade in an environment where neither charging infrastructure nor reliable sunlight can be assumed [81].

These examples show that on-board self-sufficiency is achievable, but they also reveal why it is neither simple nor general. Each platform is purpose-built around the constraints of its harvesting source: continuous availability (Saildrone’s persistent wind, the rover’s radioisotope) or a duty cycle that fits within a daily energy budget (the agricultural robots). Solar yield is dictated by weather, season, and—critically for agroforestry—canopy shading, making it intermittent and unreliable as a sole source for a small platform working beneath trees; enlarging the panel or battery to compensate adds mass and cost, which raises locomotion energy and yields diminishing returns. Nuclear sources sidestep this but are infeasible for a

small terrestrial agroforestry robot on cost, safety, and regulatory grounds. Most decisively, harvesting addresses energy alone: consumables such as water, liquid fertilizer, or seed for tree care cannot be gathered in situ and must be carried, so their depletion bounds the operational horizon regardless of the energy strategy. A purely on-board system is thus viable only where a single, reliable, continuous energy source exists and no other consumable is needed—conditions that the target agroforestry application does not satisfy.

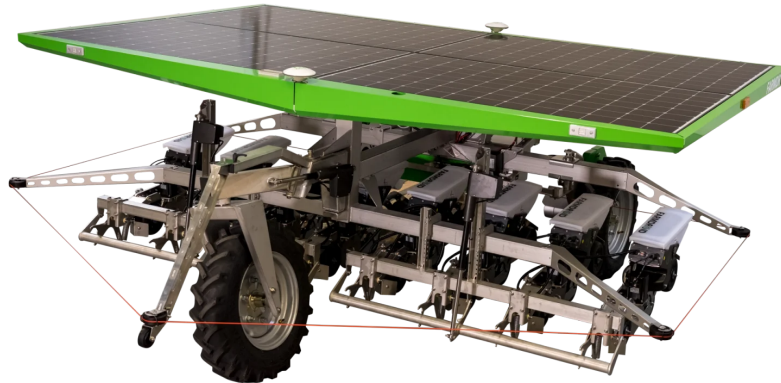


Figure 2.16.: The FarmDroid FD20 agricultural robot, which harvests solar energy via on-board photovoltaic panels to power seeding and weeding operations.

2.4.2. Autonomously Docking Robots

The complementary strategy decouples operational endurance from onboard capacity by returning the robot to a docking station to replenish energy (and, potentially, other consumables). Autonomous docking has been demonstrated across several domains, each illuminating a different facet of the problem.

In the consumer domain, robotic vacuum cleaners established the canonical pattern: the robot homes in on a station emitting an IR beacon and engages conductive charging contacts. The foundational engineering questions were articulated already by Birk, who examined how a mobile robot can reliably measure its battery state of charge in order to decide *when* to seek a recharge, and why naïve charging-control schemes are unsuitable for unattended operation [22]. For wheeled mobile robots more generally, Romanov and Tararin demonstrated a ROS 2-integrated docking procedure that localizes the station via ArUco fiducial markers, reporting docking accuracy within roughly 3 cm and explicitly contrasting the robustness of marker-based vision against IR- and LiDAR-based alternatives [23]. Singh et al. pursue the opposite extreme, showing that reliable terminal docking is achievable with a deliberately *minimal* perception stack [76]. Outside ground robotics, Gutnik and Groper address terminal docking of an autonomous underwater vehicle using an electromagnetic beacon—a regime without line of sight, where optical methods fail entirely (the beacon’s accuracy is quantified in Section 2.3) [77].

Across these systems a consistent picture emerges: docking is reliably solved in structured or controlled settings (indoor floors, calm water, fixed beacons), whereas autonomous docking of field robots in unstructured outdoor environments—with variable illumination, vegetation occlusion, and uneven terrain—remains comparatively underexplored. This gap is precisely the motivation for the present work.

2.5. Docking Triggering and Interface Technologies

Whereas the previous section described which robots dock, this section surveys the two technical sub-problems that any docking system must solve: deciding *when* to dock (resource-depletion prediction) and physically coupling the robot to the station to actually transfer the resource (interface technologies). Following the survey character of this chapter, the available technologies are described rather than ranked; the selection for the target system is deferred to Chapter 3.

2.5.1. Resource Depletion Prediction

A robot can only dock in time if it can predict, with a safety margin, when a resource will be exhausted. For the battery—the most safety-critical resource—this prediction rests on three distinct quantities. The *state of charge* (SOC) expresses how much energy currently remains, analogous to a fuel gauge; the *state of health* (SOH) captures the slow, irreversible loss of usable capacity as the cell ages over hundreds of cycles; and the *remaining useful life* (RUL) forecasts how many cycles or how much time remain before the cell can no longer meet operational requirements. The docking decision depends on all three: a robot must budget its return trip against the present SOC, but that SOC is expressed as a fraction of a capacity that SOH steadily erodes, so that a fixed voltage threshold becomes optimistic as the battery ages.

Birk already identified reliable SOC measurement as the linchpin of unattended recharging [22]. Modern estimation methods are commonly grouped into model-based approaches—equivalent-circuit or electrochemical models whose internal states are tracked with observers such as the EKF—data-driven approaches that learn the mapping from measurable signals (voltage, current, temperature) to SOC, SOH, or RUL, and hybrid combinations of the two. Thelen et al. review this landscape with an emphasis on *probabilistic* machine learning, arguing that diagnostics and prognostics for batteries must quantify their own uncertainty: a point estimate of remaining life is of little use for a safety-critical docking decision, whereas a calibrated confidence bound allows the planner to dock conservatively when the prediction is uncertain [82].

Energy is not the only depletable resource. Predictive self-monitoring extends to mechanical and sensory degradation: the field of fault detection and prognostics-and-health-management aims to detect incipient component failures—a degrading motor, a drifting or fouled sensor—before they cause an unplanned breakdown, so that a maintenance docking can be scheduled proactively [83, 84, 85]. Consumables such as water or fertilizer are comparatively straightforward, their depletion being predictable by integrating metered flow against a known tank

volume. A subtlety common to all cases is that the prediction must account for the cost of reaching the station: the energy or time required for the return journey is itself terrain- and route-dependent, coupling depletion prediction to energy-aware path planning [78, 27]. This predictive capability is not implemented in the present prototype and is identified here as a direction for future work; it is surveyed because it constitutes the trigger that the docking maneuver studied in this thesis ultimately serves.

2.5.2. Interface Technologies

Once the robot has aligned with the station, a physical interface transfers the replenished resource. The relevant technologies fall into three categories: electrical power, data, and fluid or material.

Electrical Power Transfer

Conductive (galvanic) transfer establishes direct metallic contact between mating connectors—spring-loaded pins, sliding shoes, or probe-and-socket pairs—and is the most widespread approach, used by the consumer and wheeled-robot systems above. It offers high efficiency and high power density with simple electronics, at the cost of requiring precise terminal alignment and of exposing bare contacts, which are prone to wear, oxidation, and contamination by water and dust in outdoor settings. *Inductive* (wireless) transfer instead couples energy through a magnetic field between a transmitter coil at the station and a receiver coil on the robot, eliminating exposed contacts and tolerating both misalignment and full environmental sealing. Its drawbacks are lower end-to-end efficiency and sensitivity of that efficiency to coil separation and alignment; Hijikata et al. address the latter for mobile robots through magnetic-resonant coupling with active impedance matching, which maintains transfer efficiency as the robot’s position and load vary [86]. Further methods such as capacitive coupling and optical (laser) power beaming exist but remain niche for mobile-robot charging owing to low power or strict line-of-sight requirements.

Data Communication

A docking robot exchanges data both continuously during operation and in bulk at the dock (e.g. uploading logs and maps, downloading mission updates). *Wired* links established through the docking contacts offer high bandwidth and immunity to radio interference but require physical mating. At the embedded level, robots predominantly use fieldbus and serial standards: the CAN (Controller Area Network) bus, valued for its differential, noise-robust signaling and deterministic arbitration in automotive and robotic systems; EtherCAT and industrial Ethernet for high-bandwidth, low-latency control; RS-485/Modbus for simple multi-drop links; and board-level buses such as I²C, SPI, and USB for sensor and peripheral integration. *Wireless* links dominate during free operation: Wi-Fi for high-bandwidth telemetry, Bluetooth Low Energy for low-power short-range exchange, and cellular links for remote supervision; *optical* (IR/visible-light) and *magnetic* near-field links provide

interference-immune, short-range alternatives. Networked-robotics paradigms such as the Internet of Robotic Things integrate these channels into a unified communication architecture spanning robot, station, and cloud [87].

Fluid and Material Transfer

For agroforestry tasks such as irrigation and fertilization, the docking station must additionally replenish liquid consumables—water and liquid fertilizer—and, potentially, granular material such as seed. Automated fluid coupling typically employs self-sealing dry-break or quick-connect couplings that minimize spillage and ingress when mating and de-mating, often guided by a funnel or chamfered geometry that absorbs residual misalignment, with transfer driven either by gravity or by a pump at the station. Solid or granular material is transferred via hoppers and augers or pneumatic conveyance. The principal engineering challenges—tight alignment tolerances, reliable sealing against contamination, and, in cold climates, freeze protection—parallel those of established automated refueling systems in other domains. Fluid and material transfer is not realized in the present prototype; it is included here for completeness, as it defines the interface requirements for the intended extension of the system toward autonomous tree care.

This chapter has surveyed the technical building blocks of long-term autonomous docking: the theoretical foundations of autonomy and LTA, the architectural paradigms evaluated along six dimensions, the sensing modalities for relative localisation, and the resource-management and interface technologies. The agronomic motivation for the target application—why agroforestry is a desirable but labour-bottlenecked practice that small persistent robots could enable—was established in the Motivation of Chapter 1 and is not repeated here. The recurring conclusion across the survey is that persistent operation in such a semi-structured outdoor environment hinges on robust, resource-aware self-maintenance, and thus on reliable autonomous docking. The following chapter turns these surveyed options into concrete, requirement-driven design decisions: it derives a control architecture for resource-aware LTA (answering RQ1) and selects the energy interface, data link, and candidate sensors for the docking prototype that the experiments then evaluate (RQ2).

3. Conceptual Analysis and System Design

This chapter delivers the first contribution of the thesis and sets up the second. Building on the survey of Chapter 2, it first consolidates the system requirements (Section 3.1), then *derives a conceptual control architecture for resource-aware long-term autonomy* and shows how autonomous docking is embedded within it (Section 3.2)—the answer to RQ1. Section 3.3 then narrows from this whole-system vision to the slice actually realised as a prototype—the docking subsystem—and makes the requirement-driven selections of energy interface, data link, and candidate sensors for it. Section 3.4 finally describes, in concrete ROS 2 detail, the docking pipeline that was built and tested as a single Sense–Plan–Act loop. The deliberate separation between the conceptual architecture (Section 3.2) and the concrete prototype (Sections 3.3–3.4) is what keeps the architectural contribution distinguishable from implementation detail.

3.1. System Requirements

The target system is a mobile agroforestry robot that must operate persistently in unstructured outdoor environments without human intervention. Translating this goal into engineering constraints produces two groups: functional requirements that define *what* the system must do, and non-functional requirements that constrain *how* it must behave.

3.1.1. Functional Requirements

- **Real-time reactivity.** The system must close sensor-to-actuator loops with bounded, deterministic latency. Collision-avoidance reflexes must respond faster than the robot’s mechanical response time; a purely deliberative system that pauses to replan is therefore unacceptable at the reactive layer.
- **Robustness and fault tolerance.** Under partial sensor failure or unmodeled environmental disturbances, the system must “fail cognizantly”—detecting the failure, degrading gracefully, and alerting higher layers rather than producing unpredictable behaviour [38]. This applies equally to hardware faults (a blocked LiDAR beam, a GPS drop under canopy) and to algorithmic failures (localisation divergence).
- **Predictive resource management.** The system must continuously estimate its remaining battery capacity and, accounting for the energy required to return to the docking station, predict the latest safe departure time. Resource management is tightly coupled to navigation: the return-trip energy cost depends on the planned path, which itself depends on the current occupancy map.

- **Physical resource replenishment.** The system must autonomously couple to a docking station and transfer electrical energy at a rate sufficient to restore a depleted 24 V/50 Ah lithium iron phosphate (LFP) battery within a reasonable operational window—establishing a minimum target of 200 W transferred power. In the extended system vision, the station will additionally replenish fluid consumables (water and liquid fertilizer) required for tree care.
- **Precision terminal alignment.** The final docking maneuver must achieve a positional accuracy of less than 5 cm and a rotational accuracy of less than 5° at the interface connector to ensure reliable electrical engagement regardless of approach path variance.
- **Agricultural task execution.** Beyond self-maintenance, the robot must execute agronomic field tasks—weeding, watering, fertilizing, soil measurement, and plant measurement—planned by a cloud agronomic simulation and dispatched via the onboard mission controller.

3.1.2. Non-Functional Requirements

- **Modularity.** The software architecture must be built on mature, open-source robotics middleware that provides decoupled, independently replaceable components, reducing development time and enabling individual subsystems to be tested and upgraded independently.
- **Optimizable mission logic.** The executive layer must be structured so that the mission-sequencing logic can be adapted or optimized—through parameter tuning, learned Behavior tree modifications, or structural reconfiguration—without requiring a complete software rewrite [47].
- **Edge deployability.** All hard-real-time functions—sensor processing, localisation, obstacle avoidance, and docking guidance—must run on embedded edge hardware colocated with the robot, without relying on a continuous cloud connection. Cloud connectivity is reserved for asynchronous, computationally expensive tasks such as the agronomic growth simulation.
- **Weather and environmental resistance.** The electrical interface must meet at minimum IP65 (dust-tight, water-jet resistant). The perception pipeline must remain functional across the full outdoor condition range expected in Central European agroforestry: rain, fog, direct sunlight, and overcast sky.
- **Long-term maintainability.** Subsystem boundaries must be clearly defined so that future extensions—additional sensors, updated planners, new agricultural actuators—can be integrated without destabilising the existing system.

3.2. Derivation of the Proposed Architecture

This section presents the architectural contribution of the thesis (RQ1): starting from the requirements above and the dimensional survey of Section 2.2, it derives how a control architecture for resource-aware long-term autonomy in outdoor agriculture must be structured, and where autonomous docking sits within it. The description here is deliberately *conceptual*—it concerns the target system as a whole and is given independently of ROS 2 node names or topic strings, which are deferred to the concrete prototype in Section 3.4.

3.2.1. A ROS 2-Based Hybrid Approach

The dimensional evaluation of Section 2.2.1 already locates each archetype in the design space, and the requirements of Section 3.1 pick out a region of that space rather than a single point. Resource-aware self-maintenance needs *strategic* planning over a *world model* (to predict depletion and route home), while survival in an unstructured field needs *reflexive*, memoryless reactions decoupled from that slow planning. No single classical archetype occupies both extremes: as established in Section 2.2, pure Sense–Plan–Act lacks the reactivity, pure reactive control lacks the resource foresight, and end-to-end learning architectures lack both an explicit resource state and the data to be trained safely in this domain. The hybrid (three-layer) archetype is the only one that spans the required range by construction, and the argument for *why not* the alternatives need not be repeated here—it is the dimensional analysis of Section 2.2.

A hybrid architecture satisfies the requirements of Section 3.1 when realised on mature middleware. Implementing it on ROS 2 directly answers *modularity*, *edge deployability*, and *maintainability*: the system decomposes into independent, independently replaceable lifecycle nodes communicating over Data Distribution Service (DDS), all hard-real-time functions run on the edge, and new sensors or actuators can be added without destabilising the rest [88, 44]. A reactive controller layer (e.g. a navigation stack such as Navigation 2 (Nav2)) supplies the bounded-latency reflexes; a deliberative layer supplies the resource-aware foresight; and the two run asynchronously so that planning latency never stalls a reflex [38].

For the executive layer that links the two, Behavior trees (BTs) are the natural choice, for the reasons surveyed in Section 2.2.4: they offer the reactivity of a finite state machine but compose hierarchically without the spaghetti-transition problem, and—crucially for the *optimizable mission logic* requirement—their low-dimensional symbolic structure can be tuned or evolved from execution data while the underlying reactive controllers stay deterministic and provably safe [46, 47]. This confines any learning to a bounded sub-problem rather than the whole control loop, exactly the restriction motivated in Section 2.2.5.

3.2.2. System Overview: A Database-Decoupled Sense–Plan–Act Loop

Figure 3.1 shows the target system as a concrete instance of this three-layer hybrid. At the macroscopic level it is a classical *Sense–Plan–Act* (SPA) loop, but the loop is deliberately

split across two execution domains operating at fundamentally different time scales. All architecture diagrams in this chapter share the notation summarised in Figure 3.2.

Two execution domains. The inner, time-critical domain (running on the robot, in real time) carries everything that must close a control loop with bounded latency: perceiving the environment, planning the next motion, and acting on the actuators. The outer domain (cloud compute, asynchronous) carries the compute-heavy agronomic prediction—a crop- and resource-growth simulation that decides *where* and *when* the robot should act next. Because this simulation is too expensive for the edge and need not run in real time, it is offloaded and executed asynchronously, so its latency never stalls the on-robot control loop.

Database-mediated decoupling. The two domains never call each other directly; they communicate exclusively through shared databases that act as producer–consumer buffers. The real-time sensing stage writes raw observations to a local store, from which an on-robot asynchronous stage distils the sparse, semantically meaningful observations uploaded to the cloud. The cloud pipeline consumes these together with external weather forecasts, advances its agronomic state, and emits a sorted list of waypoints and tasks plus an occupancy map. The on-robot planning stage reads its goals from exactly this store, and the acting stage writes executed outcomes back, closing the loop. This buffering is what allows a slow, best-effort cloud predictor and a fast, hard-real-time controller to cooperate without either blocking the other—a direct expression of the *edge deployability* requirement.

Resource awareness in the loop. Resource awareness is realised conceptually as an *energy-range prediction* that continuously compares the remaining battery charge against the energy needed to return to the docking station along the currently planned path. When the projected post-return charge would fall below a safety margin, the executive layer preempts the active field task and inserts the station as the highest-priority goal. This is where docking is *embedded* in the architecture: it is not a standalone behaviour but a state the executive layer enters whenever the resource loop demands it. The scope limitation stated in Section 1.2.1 applies here—the prediction is threshold-based rather than a predictive thermodynamic model.

External inputs. Two sources cross the system boundary: the physical *environment*, observed by the on-board sensors, and an external weather service (sunrise/sunset and forecast data), which feeds both the robot’s local scheduling and the cloud growth simulation.

3.2.3. Cloud Decoupling and the Agronomic Simulation

The database layer that decouples the real-time loop from the asynchronous cloud simulation is central to the architecture’s ability to support long-term operation; it is therefore part of the conceptual contribution rather than an implementation detail. Four conceptual data stores, identified in Figure 3.1, serve as the producer–consumer buffers between the two domains; their entity–relationship schemas are given in section A.5 of the appendix.

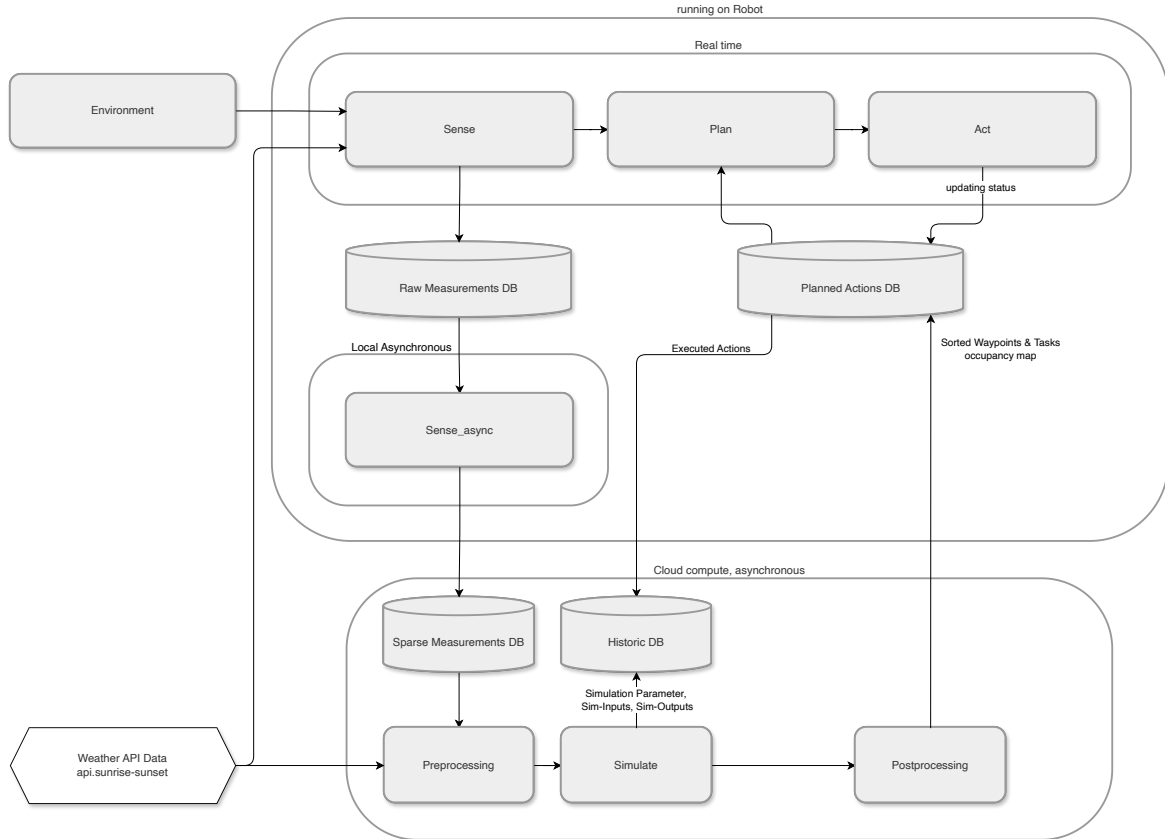


Figure 3.1.: High-level system architecture. The real-time Sense–Plan–Act loop running on the robot (top) is decoupled through shared databases from the asynchronous cloud pipeline (bottom) that simulates crop and resource development and returns the next set of waypoints and tasks. The concrete ROS 2 realisation of the three on-robot subsystems is given in Figure 3.4, Figure 3.5 and Figure 3.6; the underlying data model is given in section A.5.

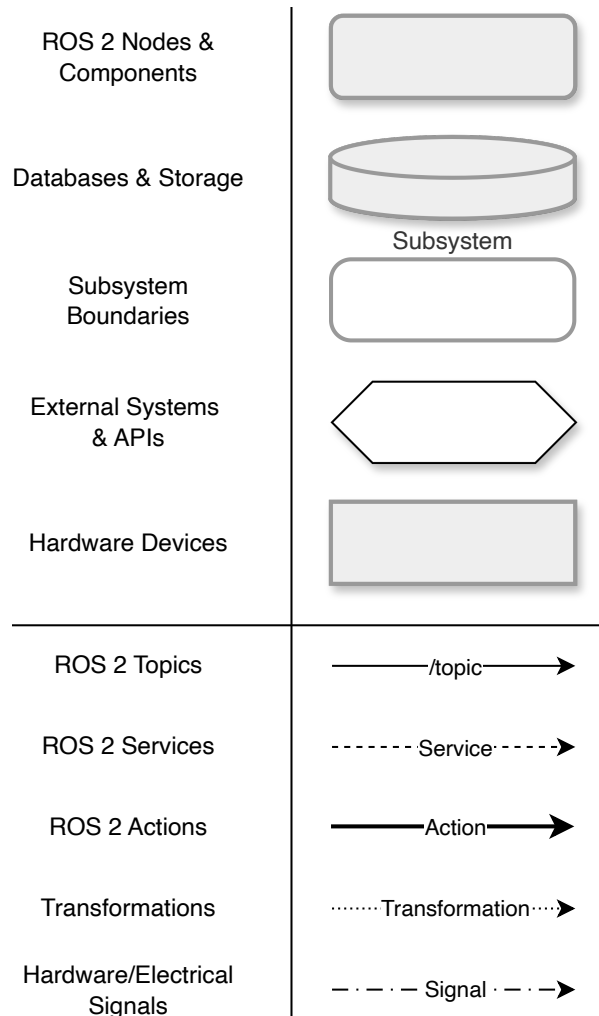


Figure 3.2.: Notation shared by all architecture diagrams in this chapter: component types (top) and edge types (bottom). Rounded rectangles are ROS 2 nodes or components, cylinders are databases, hexagons are external systems and APIs, plain rectangles are hardware devices, and enclosing rounded frames denote subsystem boundaries.

- A **raw-measurement store** captures the unprocessed sensor stream at maximum fidelity (compressed images and point clouds, each tagged with the robot pose at capture). Its high write rate keeps it local-only; it is retained for post-hoc analysis and for training future perception models and is purged on a retention window (Figure A.4).
- A **sparse-measurement store** holds the semantically distilled observations—vegetation and soil analyses organised hierarchically by field and plot, with associated weather data—compact enough for cloud transmission, and is the primary input to the agronomic simulation (Figure A.5).
- A **mission-and-task store** materialises the simulation’s output as an ordered list of waypoints and per-waypoint action tasks (weed, water, fertilize, measure, final docking), and is written back by the robot with executed parameters, status, and energy consumption, closing the observation–planning–execution loop (Figure A.6).
- A **simulation-history store** records each stateful crop-model run—engine, version, calibration, full internal state, and a link to its predecessor—so any run can be resumed or replicated, and links each run to the mission it produced (Figure A.7).

The cloud simulation itself is the agronomic “brain” of the system: a stateful, recurrent crop-growth model (for example the process-based agroforestry model Hi-sAF_e surveyed in Section 2.2.6) that assimilates the sparse measurements and weather forcing, forecasts per-plot quantities such as soil moisture and leaf-area index *with* an explicit uncertainty, and emits the next prioritised mission. This is precisely the predictive “digital twin” role surveyed in Section 2.2.6: a model that both tracks the field and feeds decisions back to the robot. Conceptually it also subsumes long-horizon route optimisation—visiting multiple agricultural waypoints under an energy budget is a variant of the Orienteering / fuel-constrained Traveling Salesman Problem—so that the edge side is left only with a deterministic point-to-point query (see Section 3.4.2). The cloud simulation and its databases are part of the architecture but, as Section 3.3 makes explicit, are not realised in the hardware prototype.

3.3. Concept Selection for the Prototype

The architecture of Section 3.2 describes the complete target system. The prototype built in this thesis realises only one slice of it: the **terminal docking subsystem**—the perception, approach, and alignment by which the robot couples to its station. Stated plainly so that the gap between vision and prototype is transparent rather than hidden, the following parts of the architecture are *not* realised in hardware:

- the **cloud agronomic simulation** and its databases (the system is exercised with pre-supplied waypoints instead);
- the **agricultural actuators** (weeding, watering, fertilizing, measurement);

- the **physical energy- and fluid-transfer hardware** at the station (the docking maneuver is evaluated up to terminal alignment, not electrical engagement).

What follows in this section are therefore the requirement-driven design decisions *for this docking prototype*: the energy interface it would couple to, its data link, and—most importantly for RQ2—the sensors whose comparison is the experimental core of the thesis.

3.3.1. Selection of the Energy Transfer Interface

Of the technology classes surveyed in Section 2.5.2, the candidate interfaces are evaluated here against the constraints derived in Section 3.1.1: a power rating > 200 W (allowing a full recharge of the onboard 24 V/50 Ah LFP battery in under 6 hours), IP65 environmental protection, no exposed live contacts when undocked, and mechanical compliance or error tolerance.

Conductive (galvanic) options

Standard magnetic-mating connectors like the Rosenberg MultMag family offer convenient self-centering but fail the power requirement, rating at only 54 W [89]. Heavy-duty alternatives, such as the magnet-guided ODU-DOCK Silver Line (ODU-MAC Blue Line series), can carry the required current and provide passive alignment, but offer only moderate tolerance to terminal alignment error [90]. Custom pogo-pin assemblies (e.g., Harwin spring contacts [91]) offer high flexibility and low cost, but require a separately engineered mechanical guide funnel to absorb positioning errors exceeding the minor travel of the spring-loaded pins.

Inductive (contactless) options

Inductive transfer eliminates exposed contacts entirely, maximizing weather resistance and safety at the cost of lower transfer efficiency [86]. The Wilerion etaLINK 3000 system most closely matches the complete requirement profile: it is rated for up to 3000 W at 15–60 V, carries IP65/IP68 sealing, and tolerates ± 30 mm of residual positional error by design—significantly relaxing the precision demand on the docking maneuver relative to conductive options [92]. However, because the manufacturer operates exclusively on a business-to-business model, a commercial supply channel could not be established during this project. A viable alternative is the WiBotic OC-301 transmitter and TR-302 receiver, a robot-grade 300 W system with IP67 protection and a generous ± 10 cm alignment tolerance [93, 94], though its cost ($> € 6000$) presents a significant barrier for a research prototype.

Assessment

Table 3.1 summarizes the candidate evaluation. Conductive and inductive interfaces present a clear trade-off: conductive options are highly cost-effective and efficient but require high alignment precision, whereas inductive systems provide superior safety and misalignment tolerance at an order-of-magnitude higher cost.

An inductive system like the etaLINK 3000 is technically the strongest choice for the target requirement profile. However, due to commercial availability constraints and the fact that the current prototype evaluates navigation and terminal alignment up to mechanical engagement rather than live electrical transfer, no physical energy interface is integrated. The final connector selection is deferred to the physical construction phase, where it will be co-designed with the station’s mechanical mating frame.

Table 3.1.: Evaluation of candidate electrical energy transfer options against the five governing requirements. Rating: + = meets requirement; o = partially meets; – = does not meet.

Option	Power >200 W	Safety	Align. tol.	IP65+	Cost
Rosenberg MultMag	–	+	+	o	+
ODU-MAC Blue Line	+	+	o	+	o
Custom pogo pins	+	+	o	+	+
Wilerion etaLINK 3000 (inductive)	+	+	+	+	o
WiBotic OC-301 (inductive)	+	+	+	+	–

3.3.2. Selection of the Data Transfer Interface

Data communication requirements differ fundamentally from power transfer: bandwidth is ample with modern radio technology, latency is not critical for the asynchronous cloud-sync use case, and a physical connection at the dock is unnecessary because radio coverage is available throughout the field. The governing constraints are: continuous wide-area connectivity throughout the field mission; high-throughput local channel for bulk data transfer when near the station; and independence from the power connector, so that a connector fault cannot interrupt data synchronization.

The following communication options were evaluated:

Table 3.2.: Evaluation of candidate data transfer options based on real performance and cost metrics compiled from cited sources. Cost figures refer to typical hardware procurement and monthly service subscriptions as of late 2025/2026.

Option	Typical Speed (Down/Up) Mbps	Latency	Hardware/Infra. Cost	Running Cost	Spatial Coverage
5G cellular	114–174 / 15–30 [95]	30–50 ms [95]	€100–200	€10–60/month	High
Wi-Fi (802.11ax)	300–900 / 300–900 [96]	2–15 ms [96]	€150–300	None	< 100 m range
Wired Ethernet	940 / 940 [97]	< 1 ms [97]	€50–100	None	Docked only
Starlink	80–220 / 12–40 [98]	30–60 ms [98]	€350–500 [98]	€55–130/month	Global

Specifically, the Wi-Fi connectivity at the docking station is implemented using a TP-Link EAP225-Outdoor access point [99], which provides outdoor-grade AC1200 dual-band Wi-Fi coverage.

- **5G cellular** – wide-area coverage, high bandwidth, available everywhere the robot operates.

- **Wi-Fi (802.11ac/ax)** with local fiber or Ethernet backhaul – high-throughput local-area link, ideal for bulk transfers near a fixed station.
- **Wired Ethernet** through the docking connector – maximum throughput and reliability when docked, but couples data and power interfaces.
- **Starlink** – viable in remote deployments without cellular coverage, but high latency and large terminal form factor make it unsuitable as a primary channel.

Table 3.2 summarizes the evaluation. The selected design combines a **5G cellular link** as the primary wide-area channel with a **Wi-Fi router at the docking station** as the local-area high-throughput link. The 5G link provides continuous telemetry upload, compressed sensor-log synchronization, and mission-update download regardless of the robot’s field position—without requiring a docking event. The Wi-Fi router enables high-throughput bulk transfer of raw ROS 2 bag files when the robot is in proximity of the station, offloading data volumes that exceed the practical bandwidth of the cellular uplink. One caveat applies: if the docking station is located inside an enclosed structure, 5G signal may be attenuated or unavailable while docked; in that scenario, the Wi-Fi router provides the sole connectivity channel.

This design deliberately avoids routing data through the physical docking connector, fully decoupling the data and power interfaces and keeping the robot network-accessible throughout the entire field mission. In the current prototype, a stationary 5G module connected via Ethernet to a Wi-Fi router provides the primary connectivity, supplemented by a dedicated 5G terminal integrated into the robot chassis as the fallback.

3.3.3. Selection of Candidate Sensors for Evaluation

Section 2.3 surveyed thirteen sensor modalities along six performance dimensions across four localization topologies. For the terminal docking use case, the synthesis matrix of Table 2.2 established the following selection criteria: sub-centimeter positional accuracy at terminal range (< 1 m); adequate robustness to Central European outdoor illumination and weather; acceptable computational cost for edge deployment; low additional hardware cost; and preferential use of sensor modalities already installed on the robot platform, since integrating a sensor that serves multiple purposes (navigation and docking detection) avoids redundant hardware.

Why this particular comparison. The experiment does not benchmark all thirteen modalities against one another, which would dilute rather than sharpen the result; it isolates the one comparison that actually decides how the docking perception subsystem should be built. After applying the criteria above, two—and only two—modalities are both viable at terminal range and already present on the platform for other reasons: the **RGB camera with AprilTag fiducials** and the **LiDAR**. They are chosen for the head-to-head because they sit at *opposite ends of the dimensions that matter most for outdoor docking*, so the comparison is maximally informative: the camera is an emission-independent, reflection-based optical sensor that is

cheap and the most accurate at terminal range but is intrinsically illumination-sensitive; the LiDAR is an active sensor that is illumination-*independent* and robust to lighting extremes but limited in close-range accuracy and point density. RQ2 is precisely the trade-off between them—accuracy versus robustness versus computational cost—and the remaining surveyed modalities are either unavailable on the platform, fail the terminal-accuracy criterion, or require additional powered infrastructure on the station, so none would change this central trade-off. RTK-GPS, the third on-board localisation sensor, is excluded from the comparison for a different reason and is retained only as the experimental ground-truth reference (below).

RGB Camera with Fiducial Markers

Of the reflection-based optical modalities, the RGB camera paired with AprilTag fiducial markers offers the highest terminal accuracy (sub-centimetre at terminal range under favourable illumination, Section 2.3). Its hardware cost is the lowest of all candidates, and its computational footprint on an NVIDIA Jetson using hardware-accelerated inference (NVIDIA Isaac ROS AprilTag pipeline) is modest relative to its accuracy. The principal weakness is sensitivity to illumination extremes: direct sunlight can saturate the image sensor and wash out marker contrast, while deep shadow reduces detection probability to zero. This sensitivity makes it essential to characterize performance specifically under the outdoor conditions that characterize the deployment environment—which is one of the primary objectives of the experimental study in Chapter 4.

The same RGB camera also admits a second, marker-free processing path: rather than reading a printed fiducial, a learned object detector (the deep-learning feature-matching family of Section 2.3.1) can recognise the docking station from its natural shape. This trades the fiducial’s terminal precision for a much longer detection range and greater weather robustness, at the cost of a training-data requirement. The two camera paths—hand-crafted fiducial and learned detector—are therefore not competing sensors but complementary processing strategies on one modality, and their head-to-head comparison along the range and weather axes is a central result of Chapter 5.

LiDAR with SLAM

LiDAR provides illumination-independent 3D point-cloud measurements and serves as the backbone sensor for SLAM-based localisation throughout the field mission. For docking station detection specifically, the known geometry of the station structure is matched against the incoming laser scan using Iterative Closest Point (ICP) registration [65], yielding the station’s pose relative to the robot without requiring any powered marker on the station. This passive-station property is architecturally advantageous: the LiDAR detection pipeline imposes no power supply, electronics, or maintenance requirements on the docking station’s perceptual infrastructure. Its close-range limitations (the sub-30 cm blind zone and sparse returns, Section 2.3) restrict accuracy at the final coupling distance, so the LiDAR is best suited to the local approach phase (1–10 m). The comparative evaluation of RGB camera against LiDAR across both positional accuracy and environmental robustness constitutes the

central experimental contribution of this thesis.

RTK-GPS as Ground Truth Reference

RTK-GPS provides centimeter-level absolute positioning in open terrain and serves as the primary global navigation sensor during field traversal and the long-range approach to the docking station. Its documented limitations in agroforestry—signal degradation and multipath errors under dense tree canopies—exclude it from the terminal docking modality comparison. Instead, RTK-GPS in Fixed mode is used as the primary ground-truth reference in the experimental study: its centimeter-accurate geodetic position estimates serve as the baseline against which the camera and LiDAR docking-accuracy measurements are evaluated. Manual reference measurements provide an additional, independent ground-truth source to validate the RTK-GPS estimates themselves.

Alternative Sensor Configurations

Several alternative sensor configurations that in principle satisfy the design constraints were evaluated but ultimately not chosen. The most prominent alternative, which was subjected to preliminary experiments, involves using active infrared (IR) emitters positioned at the docking station in combination with the onboard IR camera of the Intel RealSense D435i (as depicted in Figure 3.3). This approach was rejected due to three major limitations: first, it demands additional active equipment and power supply at the station; second, it yields a severely limited temporal resolution of 1 Hz at best, as distinguishing the three required emitters necessitates sequential pulsing at distinct frequencies with intervening gaps; and third, the emitters' intensity failed to provide sufficient contrast against background radiation in outdoor settings, rendering detection unreliable. This lack of contrast is visible in Figure 3.3, where the emitter's brightness is not particularly prominent against the background. Furthermore, the D435i's own infrared projector (used for active depth tracking) emits significant IR radiation; while this projector can be disabled, it was active during the test shown in the figure, further complicating emitter isolation. Additionally, configurations relying on magnetic spools embedded in the ground were rejected because of their short detection range and complex physical implementation.



Figure 3.3.: Preliminary experiment using a handheld active infrared emitter and camera setup. This configuration was rejected due to low temporal resolution, active infrastructure requirements at the station, and poor contrast against solar background radiation.

3.4. The Docking Pipeline: A Sense–Plan–Act Realisation

This section details the architecture of the terminal docking pipeline at the level of the relevant ROS 2 nodes, topics, transforms, and coordinate frames. It deliberately spans two layers, and the distinction between them should be kept in mind throughout: only one part of what the diagrams depict was actually *built and tested*—the terminal docking maneuver, realised as a single SPA loop on the robot—while the surrounding nodes belong to the wider conceptual system architecture of Section 3.2 and were *not* realised in the current prototype. The two are kept visually distinct: components carrying the construction-sign badge (a construction emoji icon in the diagrams) are these conceptual additions, whereas every *unbadged* component was physically built and tested. The prose below follows the same convention, flagging conceptual nodes explicitly where they appear.

Although the pipeline reads directly as Sense–Plan–Act, it is not a classical deliberative SPA architecture: the *Plan* layer issues an abstract intent rather than a fully specified motion executed verbatim, and the lower layers continuously react to fresh perception. It is therefore better described as a *hybrid* (deliberative + reactive) architecture in which ROS 2 components act as Sense, Plan, or Act modules: *Sense* estimates the robot’s state and the station pose (Section 3.4.1); *Plan* decides when to dock and sequences the maneuver (Section 3.4.2); and *Act* executes the three-phase terminal alignment that concludes the pipeline (Section 3.4.3). The three subsystems are detailed at the node level in Figure 3.4, Figure 3.5 and Figure 3.6.

3.4.1. Sense: Perception and State Estimation

The Sense subsystem bridges physical hardware and software: it takes raw signals from a heterogeneous sensor suite, abstracts them into standardized ROS 2 message types, pre-processes them into a form the downstream estimators can consume (height-filtering and projecting the LiDAR scan, rectifying and undistorting the camera frames, time-synchronising the streams), and runs the detection front-ends that extract the station pose, feeding both the localisation/state-estimation and the docking-station-detection pipelines that supply the docking pose.

Hardware abstraction layer. The physical hardware comprises an Intel RealSense D435i [73] (providing both RGB and depth imaging), a Livox Mid-360 LiDAR, an RTK-GPS receiver, the PX4 flight controller’s internal IMU and barometer, a Bluetooth-linked LiTime battery, and CSI auxiliary cameras. Each device is served by a dedicated ROS 2 driver node (`jetson_camera`, `livox_ros_driver2`, `gps_position_publisher`, `litime_bms_ros_node`, `GStream`). These driver nodes are the only components that contain hardware-specific code; every downstream consumer subscribes exclusively to standardized topics, so individual sensors are replaceable without modifying any processing node.

Two-layer state estimation. Reliable pose estimation outdoors requires fusing measurements that arrive at different rates, carry different noise, and may be intermittently unavailable; the docking pose is only as good as this estimate, so state estimation is part of *Sense*, not a separate

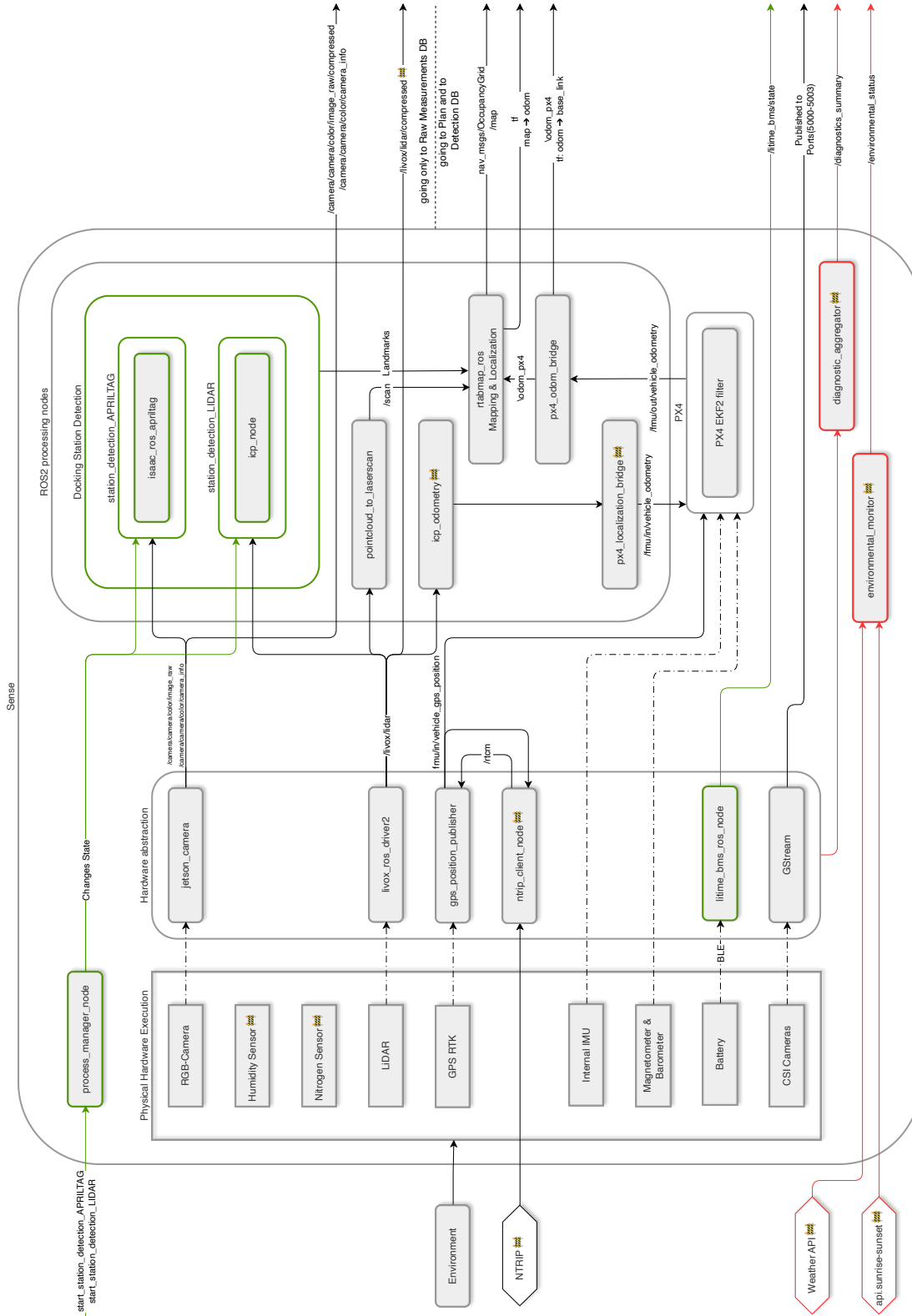


Figure 3.4.: The Sense subsystem. Physical sensors (left) are abstracted into ROS 2 topics that feed the localisation/mapping pipeline (rtabmap_ros, PX4 odometry bridges) and the two interchangeable docking-station detection pipelines (AprilTag and LiDAR). See Figure 3.2 for the notation.

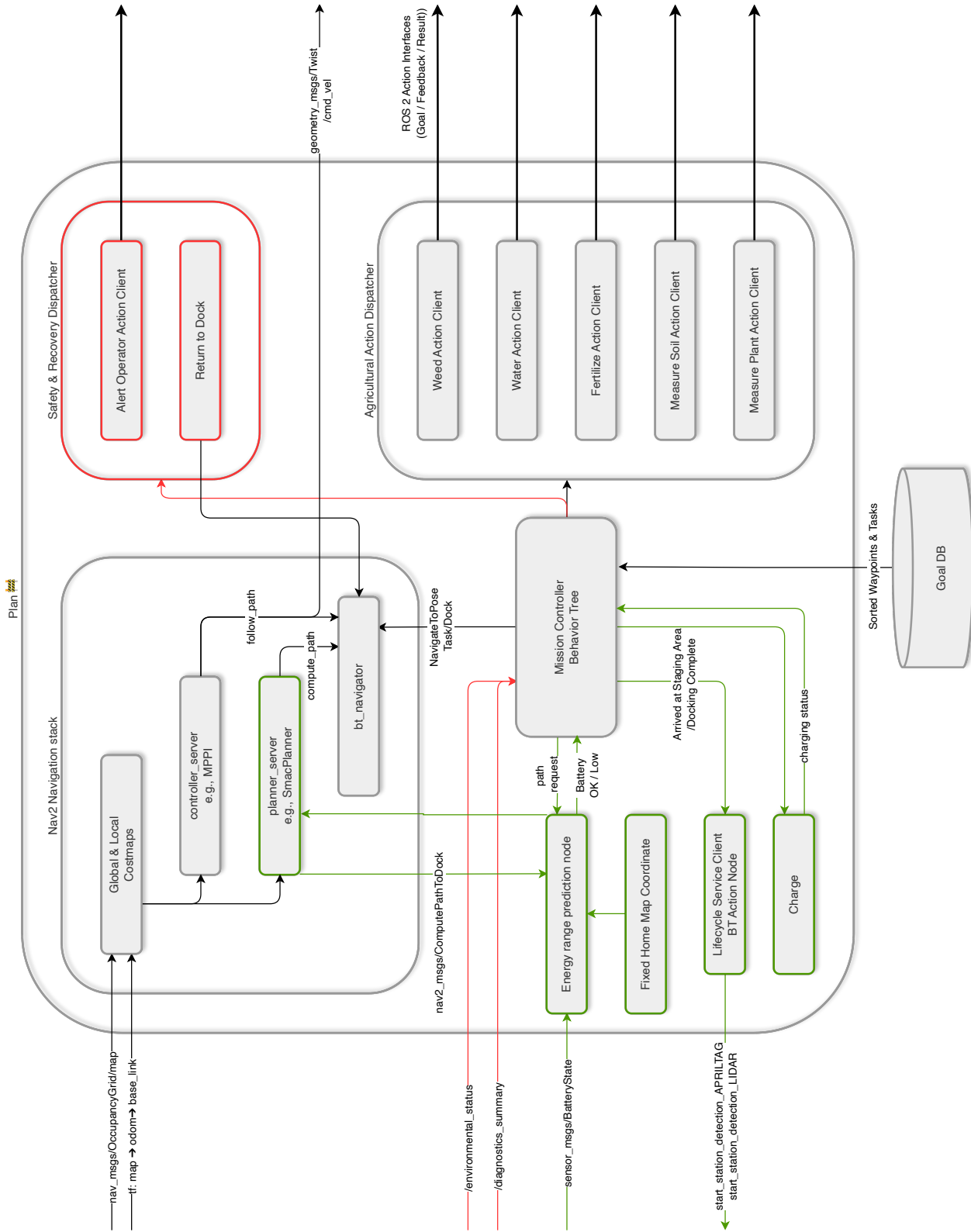


Figure 3.5.: The Plan subsystem. A Behaviour-Tree mission controller sequences the sorted waypoints and tasks from the goal database, invokes the Nav2 navigation stack for path planning and control, and dispatches the agricultural action clients and the safety/recovery behaviours. See Figure 3.2 for the notation.

3. Conceptual Analysis and System Design

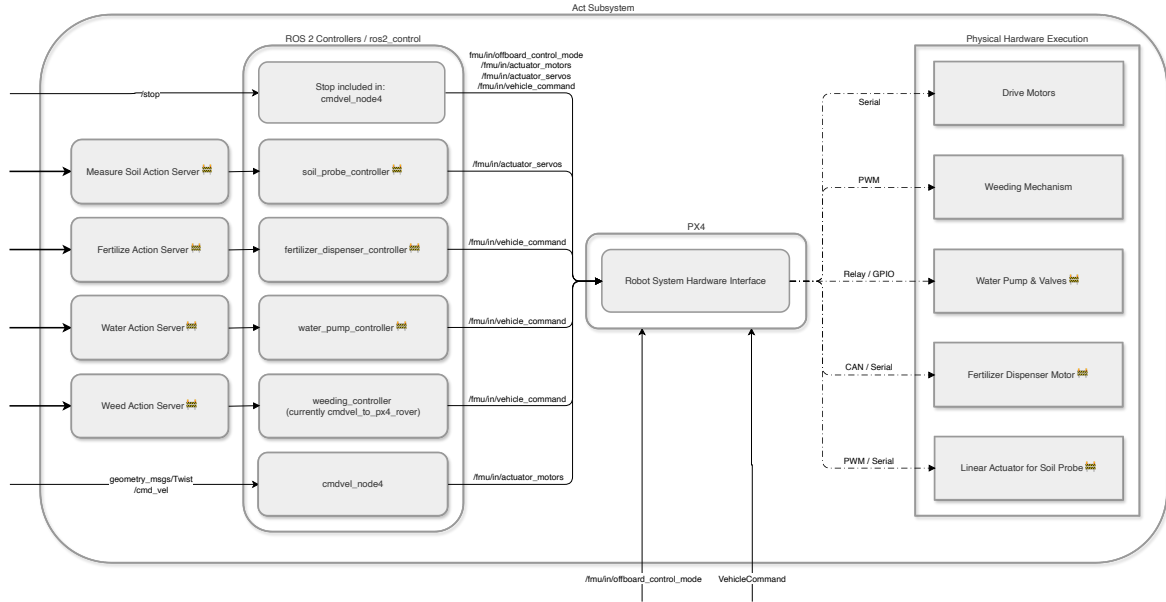


Figure 3.6.: The Act subsystem. Action servers translate abstract task goals into `ros2_control` controller commands, which the PX4-based robot hardware interface maps onto the drive motors and the agricultural actuators via the corresponding electrical signals. See Figure 3.2 for the notation.

stage. It is split into two complementary layers. *Layer 1 (high-rate, on the flight controller)*: the PX4 EKF2 runs at up to 250 Hz, fusing the IMU (three-axis accelerometer and gyroscope) with the barometer [72]; the robot carries no wheel encoders, so no proprioceptive odometry is available. This layer owns the *local* state—velocity, attitude, and short-horizon position increments relative to the odometry origin—and, running on dedicated microcontroller hardware, maintains bounded latency regardless of companion-computer load. Its output is bridged into ROS 2 by `px4_odom_bridge` as the `/odom_px4` odometry topic and the `odom→base_link` transform, a continuously available motion prior for all downstream consumers. *Layer 2 (global, on the Jetson)*: consumes this clean motion prior and augments it with lower-rate global corrections—LiDAR SLAM via `rtabmap_ros`, whose scan-matching bounds odometry drift and whose loop closures restore global consistency (publishing the `map→odom` correction), and RTK-GPS corrections streamed from an NTRIP server via `ntrip_client_node`, which anchor the map frame to a centimeter-accurate geodetic reference in open inter-row corridors. Under dense canopy the GPS contribution is degraded and SLAM loop closures primarily maintain global consistency. The resulting transform tree (`map→odom→base_link`) is the shared spatial reference consumed by Plan. A planned `px4_localization_bridge` node will close the outer loop by forwarding the globally corrected pose back to PX4’s EKF2 as an external position measurement.

Docking-station detection pipelines. Two detection pipelines, the two RQ2 candidates, run concurrently under a central `process_manager_node`:

- `station_detection_APRIETAG` uses the NVIDIA Isaac ROS AprilTag node to compute the station’s full 6-DOF pose from a single camera image via PnP estimation.
- `station_detection_LIDAR` applies ICP-based point-cloud registration in `icp_node` to the `/scan` laser scan to estimate the station’s pose from its structural geometry.

The process manager activates both via ROS 2 lifecycle service calls. Their estimates are fused to exploit the complementary strengths established in Section 3.3.3—LiDAR for robust intermediate-range detection independent of illumination, camera for high-accuracy terminal alignment. Because the robot’s map-frame pose is continuously refined by the two-layer estimator, the station-relative pose (initially expressed in the sensor frame) is automatically refined in the map frame as the robot approaches, narrowing the effective alignment error without any explicit change of coordinate system: the navigation goal simply tracks the improving station-pose estimate.

Diagnostic and environmental monitoring. The `diagnostic_aggregator` node collects health information from all sensor drivers and publishes a unified `/diagnostics_summary` topic that the mission controller monitors for fault detection and graceful degradation. The `environmental_monitor` node integrates weather-service and onboard ambient data to publish `/environmental_status`, informing scheduling decisions such as suspending irrigation during rainfall.

3.4.2. Plan: Navigation, Energy Trigger, and Mission Sequencing

The Plan subsystem is the executive layer of the hybrid architecture. It reads high-level goals from the goal database, orchestrates the Nav2 navigation stack for motion planning and control, and dispatches actions through dedicated client groups. Its central component is the **Mission Controller Behavior tree**, which continuously ticks through the robot’s current objectives and monitors the conditions that trigger transitions between them.

Nav2 navigation stack. The Nav2 framework [88] provides motion planning and control. Global and local costmaps are built from the `/map` topic and live laser scan data; the `planner_server` (SmacPlanner) computes collision-free global paths and the `controller_server` (MPPI controller) executes them with local reactive obstacle avoidance. The Nav2 `bt_navigator` orchestrates these servers and receives `NavigateToPose` goals from the mission controller via ROS 2 action interfaces.

Energy-triggered docking decision (*Plan decides when*). This is the concrete realisation of the energy-range prediction described conceptually in Section 3.2.2. The Energy range prediction node monitors the battery state (`sensor_msgs/BatteryState`) and the fixed dock home coordinate. On each planning cycle it requests a path-cost estimate from the planner

server for the current-position-to-dock route, converts the path length and terrain profile into a predicted energy consumption via a terrain-weighted power model, and compares this against the current state of charge minus a configurable safety margin. When the estimated charge after the return trip would fall to or below zero, the node asserts `Battery Low` to the Mission Controller Behavior tree, which immediately preempts the active agricultural task, assigns the docking station the highest priority, and transitions to the docking mission state. The safety margin is set conservatively to absorb worst-case return conditions (uphill terrain, heavy sensor load, wet ground), so the robot abandons an in-progress task rather than risk stranding. As noted in Section 3.2.3, the global multi-waypoint optimisation (a fuel-constrained TSP / Orienteering Problem) is offloaded to the cloud, which supplies a pre-sorted waypoint sequence; this reduces the edge-side computation to a deterministic point-to-point path-cost query and keeps execution predictable on the companion computer.

Docking sequencing. On entering docking mode, the mission controller commands the Lifecycle Service Client BT Action Node to activate the appropriate station-detection pipeline via the `process_manager_node` in `Sense`, and invokes the `nav2_msgs/ComputePathToDock` action, whose planner plugin is aware of the station's geometry and approach constraints. In the planned full system, once mechanical docking is confirmed (`/Docking Complete`), a `Charge` node will monitor the BMS charging current and notify the controller when the battery is restored, triggering a return to the mission; as noted in Section 3.3.1, no energy-transfer hardware exists in the current prototype, so the `Charge` node describes intended behaviour only.

Agricultural and safety dispatchers. For completeness of the realised graph (badged as vision in Figure 3.5), the Agricultural Action Dispatcher groups five action clients (`Weed`, `Water`, `Fertilize`, `Measure Soil`, `Measure Plant`), and the Safety and Recovery Dispatcher handles two safety-critical behaviours: `Alert Operator`, which broadcasts a diagnostic event to the remote supervisor, and `Return to Dock`, which issues an emergency navigation goal to the station, bypassing the normal goal queue. Both are triggered either by the diagnostic aggregator (critical hardware fault) or by the environmental monitor (unsafe weather or darkness).

3.4.3. Act: Actuation and the Three-Phase Docking Sequence

The Act subsystem is the functional (reactive) layer: it translates abstract goals from the executive layer into hardware commands with bounded, deterministic latency, organised into action servers, `ros2_control` controllers, and physical hardware connected through the PX4 hardware interface.

Drive control. Navigation velocity commands (`geometry_msgs/Twist` on `/cmd_vel`) are received by `cmdvel_node4`, which converts them into PX4 offboard control messages (`/fmu/in/actuator_motors`).

/fmw/in/offboard_control_mode) forwarded to the PX4 flight controller; the PX4 Robot System Hardware Interface then drives the physical drive motors via Serial. Using PX4 as the low-level motor controller retains its high-rate, safety-checked motor mixing and failsafe logic while exposing a standard ROS 2 velocity interface to the navigation stack above. The agricultural actuators (badged as vision) follow the same decoupled action-server pattern, so that adding a new actuator requires only a new action-server/controller pair without modifying any existing component.

The three-phase docking sequence (*Act executes*). The maneuver that concludes the pipeline is organised into three phases of progressively increasing precision and shortening range—*Global*, *Local*, and *Terminal*, realised in the docking behaviour tree as the GPS approach, vision align, and precise dock stages respectively; together with the *Sense* pose estimate and the *Plan* trigger, this is the terminal docking logic in full. This three-phase naming is used consistently in the experimental chapters (Chapters 4 and 5).

Phase 1 – Global approach. From its current field position, the Nav2 global planner computes a collision-free path to the station’s known map coordinate, which the robot follows using the MPPI controller with LiDAR-based local obstacle avoidance. RTK-GPS provides absolute corrections in open-sky sections; under canopy, SLAM loop closures maintain heading accuracy. The phase terminates when the robot enters an approximately 10 m staging area around the dock.

Phase 2 – Local approach with station detection. On entering the staging area, the Lifecycle Service Client BT Action Node activates both detection pipelines via the process manager. The LiDAR ICP pipeline estimates the station pose from its structural geometry at 1–10 m; the AprilTag pipeline begins returning marker poses once the fiducials enter the camera’s field of view. Both are fused with the SLAM-maintained robot pose to continuously refine the station’s position in the map frame, and the Nav2 navigator uses `ComputePathToDock` to drive toward this refined goal while maintaining obstacle avoidance.

Phase 3 – Terminal alignment and engagement. As the robot closes to approximately 1 m, the AprilTag pipeline enters its high-accuracy regime (Section 2.3), providing sub-centimetre PnP pose estimation at this range. The controller switches to a fine-alignment mode that uses the AprilTag-derived error signal directly, executing small corrective motions to align within the tolerance required by the energy-transfer interface. A docking failure—absence of a charging-current signal after the full approach—triggers the `Return to Dock` safety behaviour, which backs the robot clear and reattempts the approach.

4. Experimental Setup

This chapter outlines the experimental design and methodology used to evaluate the relative localisation modalities. To establish a robust benchmark, real-life data was collected outdoors under actual field conditions, and systematic parameter sweeps were performed in a high-fidelity simulation environment. Crucially, a hybrid evaluation workflow was adopted where parameters of the outdoor-collected data (such as atmospheric visibility and precipitation rates) were subsequently modified and augmented within the simulation environment, enabling a direct and controlled analysis of environmental degradation on real-world perception pipelines.

4.1. General Setup

To evaluate the candidate relative localisation sensors and algorithms under realistic operating conditions, a dual-environment test methodology combining high-fidelity simulation and real-world experiments was established. This approach exploits the complementary advantages of both domains to thoroughly address the research questions:

- **Simulation Environment:** High-fidelity simulation is essential for conducting systematic, repeatable parameter sweeps across the entire geometric and environmental envelope. In simulation, exact, analytical ground-truth coordinates of both the robot and the docking station are available, allowing for precise quantification of localisation errors. Additionally, simulation enables the decoupling of geometric factors (e.g., viewing distance and angle) from environmental degradation (e.g., rain, snow, and fog density), which is practically impossible in outdoor field testing. Finally, simulation provides a safe, controlled testbed for evaluating extreme weather conditions that would damage the robot, which is not fully weather-resistant.
- **Real-World Validation:** Field tests are critical to validate the simulation models and verify the presence of any sim-to-real gap. Real-world sensor data are subject to complex physical noise, imperfect calibrations, and unpredictable outdoor lighting conditions (e.g., solar glare, dynamic shadowing). Thus, the field tests serve as the ultimate confirmation that the perception pipelines can deliver the required terminal-docking accuracy under actual outdoor conditions.

4.2. Performance Metrics

To quantitatively compare the performance of the relative localisation pipelines (AprilTag, learned-keypoint YOLOv8-pose, and geometric LiDAR), five primary evaluation metrics are defined:

1. **Detection Rate (R_{det}):** The detection rate measures the reliability and availability of a relative localisation pipeline. It is defined as the percentage of frames or evaluation steps in which the detector successfully identifies the docking station and returns a valid pose estimate:

$$R_{\text{det}} = \frac{N_{\text{detected}}}{N_{\text{total}}} \times 100\% \quad (4.1)$$

where N_{detected} is the number of frames where a pose was successfully resolved, and N_{total} is the total number of frames in the evaluation sweep.

2. **Translation Error (e_{trans}):** The translation error quantifies the geometric distance between the estimated docking station origin and the ground-truth origin. It is computed as the Euclidean distance:

$$e_{\text{trans}} = \|\mathbf{t}_{\text{est}} - \mathbf{t}_{\text{gt}}\|_2 = \sqrt{(x_{\text{est}} - x_{\text{gt}})^2 + (y_{\text{est}} - y_{\text{gt}})^2 + (z_{\text{est}} - z_{\text{gt}})^2} \quad (4.2)$$

where $\mathbf{t}_{\text{est}} = [x_{\text{est}}, y_{\text{est}}, z_{\text{est}}]^T$ is the estimated translation vector relative to the robot base, and $\mathbf{t}_{\text{gt}} = [x_{\text{gt}}, y_{\text{gt}}, z_{\text{gt}}]^T$ is the ground-truth translation vector.

3. **Rotation (Orientation) Error (e_{rot}):** The rotation error evaluates the heading estimation accuracy, which is critical for terminal alignment. It is defined as the absolute angular difference between the estimated yaw orientation θ_{est} and the ground-truth yaw orientation θ_{gt} :

$$e_{\text{rot}} = |\theta_{\text{est}} - \theta_{\text{gt}}| \quad (4.3)$$

For the geometric LiDAR pipeline, which is prone to symmetric 90° and 180° orientation flips, e_{rot} directly exposes the alignment ambiguities.

For both the LiDAR and AprilTag pipelines, the translation and rotation errors across multiple frames or static sweep positions are summarised using the **Root-Mean-Square Error (RMSE)** to capture both systematic offsets and variance in a single metric:

$$\text{RMSE}_{\text{trans}} = \sqrt{\frac{1}{N_{\text{detected}}} \sum_{i=1}^{N_{\text{detected}}} e_{\text{trans},i}^2} \quad (4.4)$$

$$\text{RMSE}_{\text{rot}} = \sqrt{\frac{1}{N_{\text{detected}}} \sum_{i=1}^{N_{\text{detected}}} e_{\text{rot},i}^2} \quad (4.5)$$

check if we used 2D or 3D translation error

4. **Image-Space (Keypoint) Error (e_{pixel}):** For the vision-based pipelines (AprilTag and YOLOv8-pose), the localisation quality is heavily determined by the accuracy of feature extraction in the image plane before the Perspective- n -Point (PnP) solver is executed. We evaluate this using the mean Euclidean distance (in pixels) between the predicted 2D keypoint projections $\mathbf{p}_k^{\text{est}}$ and their corresponding ground-truth projections $\mathbf{p}_k^{\text{gt}}$:

$$e_{\text{pixel}} = \frac{1}{K} \sum_{k=1}^K \|\mathbf{p}_k^{\text{est}} - \mathbf{p}_k^{\text{gt}}\|_2 = \frac{1}{K} \sum_{k=1}^K \sqrt{(u_k^{\text{est}} - u_k^{\text{gt}})^2 + (v_k^{\text{est}} - v_k^{\text{gt}})^2} \quad (4.6)$$

where K represents the number of tracked keypoints. For the YOLOv8-pose network, $K = 8$, corresponding to the eight corners of the docking station’s bounding cuboid. For the AprilTag pipeline, e_{pixel} is evaluated on the tag’s center point or its four corners. Ground-truth pixel coordinates $\mathbf{p}_k^{\text{gt}} = [u_k^{\text{gt}}, v_k^{\text{gt}}]^T$ are obtained by projecting the 3D coordinates of the corners P_k using the camera intrinsics matrix $\mathbf{K}$ and the true relative pose:

$$s \begin{bmatrix} u_k^{\text{gt}} \\ v_k^{\text{gt}} \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R}_{\text{gt}} \quad \mathbf{t}_{\text{gt}}] \begin{bmatrix} X_k \\ Y_k \\ Z_k \\ 1 \end{bmatrix} \quad (4.7)$$

5. **Computational Cost:** To assess whether a pipeline is suitable for real-time edge deployment on the companion computer, we measure three computational metrics:
- **Inference Latency (T_{lat}):** The average execution time in milliseconds (ms) required to process a single frame or scan.
 - **Throughput:** Measured in frames per second (FPS) or Hz, which must exceed the sensor acquisition rates (30 FPS for the camera and 10 Hz for the LiDAR).
 - **Processor Load:** The mean number of busy CPU cores, CPU/GPU utilisation, and peak memory footprint (Process Resident Set Size (RSS) in MiB or GPU VRAM in MiB).

4.2.1. Test Scenarios

To systematically evaluate the localisation pipelines against these metrics, two primary test scenarios are designed:

Final Docking Maneuver

This scenario evaluates the closed-loop performance of the complete docking software stack under active feedback control. Starting from a variety of initial capture positions and orientations, the robot must execute the full three-phase docking sequence of Section 3.4.3—*Global* approach, *Local* approach, and *Terminal* alignment (realised in code as the GPS approach $\rightarrow$ vision align $\rightarrow$ precise dock behaviour-tree stages)—until it reaches the station and stops.

In the simulation environment, this is conducted as a closed-loop campaign in the high-fidelity testbed, where exact per-frame ground truth makes the terminal alignment error directly measurable and the runs are bit-for-bit repeatable. Ten runs are executed from a single fixed random seed, each spawning the robot at a uniformly random pose in the 3–10 m ring around the station centre with a random initial heading, and each driven by the full three-phase behaviour tree (GPS approach $\rightarrow$ vision align $\rightarrow$ precise dock) on the EKF-fused terminal pose described in Section 5.5. The campaign is run under clear conditions only and is intended as a demonstration of the maneuver *dynamics*, not as a statistical weather success-rate study—the per-detector weather robustness is characterised separately in the static studies of this chapter. A run is marked a failure if it exceeds a timeout of 120 s or fails to converge onto the station within it.

To assess performance, the following closed-loop metrics are recorded for each run:

1. **Docking Success Rate (S_{dock}):** The percentage of runs that reach the station and settle within the terminal-alignment tolerance of Section 3.1.1: a terminal translation error $e_{\text{trans}} \leq 5$ cm and orientation error $e_{\text{rot}} \leq 5^\circ$ at the interface plane.
2. **Terminal Alignment Accuracy:** The mean, standard deviation and median of the translation and rotation error at the moment of contact, computed across the docked runs to quantify precision.
3. **Docking Duration (T_{dock}):** The total time from controller activation to completion, broken down into the three phases (*Global*, *Local*, *Terminal*).

In the real world, the closed-loop maneuver is exercised only as a basic qualitative feasibility check, not as a statistical study: outdoor ground truth for the full maneuver is not available (Section 5.6.3), so a large run count would yield no reliable success-rate or accuracy figures. A small number of end-to-end runs (no more than three) are performed solely to confirm that the behaviour-tree transitions and the controller carry the physical robot into the station under real outdoor lighting.

Real-world closed-loop runs (≤ 3) to be collected and the qualitative result inserted here / in Section 5.6.

Edge Cases: Varying Environmental Conditions

This scenario isolates the failure limits of each sensor under degraded conditions:

- **Geometric Edge Cases:** We sweep range (1.5 to 30 m) and azimuth (0° to 360°) under ideal conditions to find the boundary of the geometric detection envelope. We also conduct micro-sweeps (± 10 mm steps) to identify sub-pixel aliasing effects near the range limits.
- **Atmospheric Edge Cases:** We systematically inject rain, snow, and fog of varying physical intensities (e.g., rain rate up to 200 mm/h, snow rate up to 25 mm/h, and meteorological visibility down to 5 m) onto fixed reference poses. This allows us to map

the degradation curve of each vision detector in response to atmospheric scattering and veiling.

- **Data Ablation Edge Cases:** We evaluate the learned detector’s robustness to varying training distributions by testing models trained on synthetic-only, real-only, and augmented datasets.

4.3. Selection of Experimental Parameters

The experimental parameters follow directly from the candidate sensors selected in Section 3.3.3 and are deliberately split into two independent groups, so that each source of detection failure can be studied in isolation. Holding one group fixed while varying the other isolates cause from effect: a drop in detection during a weather sweep can be attributed to the atmosphere rather than to a change in viewpoint, and vice versa. This single-variable principle governs the sweep design in Section 4.5 (geometric envelope) and the weather-augmentation study (environmental degradation).

Table 4.1 consolidates the exact numeric ranges of every parameter swept in this thesis; the individual sweeps that use them are described in Sections 4.4.3, 4.5.3, and in the weather-robustness results of Section 5.2.

4.3.1. Physical and Geometric Parameters

The first parameter group captures the *geometric and physical* factors that determine whether the docking station can be detected *at all* under otherwise ideal conditions: the distance between robot and station, the approach (azimuth) angle, the camera resolution, the apparent size of the marker, and the processing parameters of the LiDAR pipeline (such as the occupancy grid resolution and height-band filter limits). These parameters set the purely geometric detection envelope—how far away and at what angle the fiducial or structural geometry still projects onto enough pixels or returns sufficient laser returns for the detector to resolve it.

4.3.2. Environmental Parameters

The second group captures the *environmental* factors that degrade an otherwise geometrically detectable target: atmospheric illumination, and adverse weather conditions such as rain, snow, and fog. These conditions do not alter the physical geometry of the scene but instead attenuate or scatter the signal reaching the sensor, introducing noise, reducing contrast, or blocking the transmission path.

4.4. Real-World Experimental Setup

This section details the physical experimental setup used for outdoor data collection and sensor benchmarking. All experiments were conducted outdoors on level ground. The physical

4. Experimental Setup

Table 4.1.: Numeric ranges of the experimental parameters, split horizontally into simulation sweeps (further grouped by vision modalities, LiDAR modality, and simulated weather degradation) and real-world experiments. Each row is varied independently while the others are held at their reference value.

Parameter	Values / range	Reference value
Simulation sweeps and configurations		
<i>AprilTag & YOLO (Vision modalities)</i>		
Distance (max-distance sweep)	1.5–30 m in 0.5 m steps	—
Azimuth angle (orbit sweep)	0°–360° in 1° steps at $r = 3$ m	—
Camera resolution	640 × 480; 1280 × 960 px	640 × 480 px
Marker (tag) edge size	13 cm; 26 cm	13 cm
Lateral alignment micro-sweep	±10 mm steps at 5.5 and 7.0 m	—
<i>LiDAR modality</i>		
Distance (max-distance sweep)	1.5–8.0 m in 0.5 m steps	—
Azimuth angle (orbit sweep)	0°–360° in 5° steps at $r = 3$ m	—
<i>Closed-loop end-to-end docking (10 runs, single seed, clear weather)</i>		
Initial spawn distance	uniform in the 3–10 m ring	—
Initial heading	uniform random	—
Number of runs	10	—
Timeout limit	120 s	—
Real-world experiments (all modalities: AprilTag, LiDAR & YOLO)		
<i>Experiment 1 (outdoor static grid sweep)</i>		
Distance	1, 2, 3, 5, 7, 10 m	—
Azimuth angle	0°, 30°, 45°, 90°, 135°, 180°	—
<i>Experiment 2 (automated outdoor range and orbit sweeps)</i>		
Distance	2–10 m in 1 m steps	—
Azimuth angle	0°, 30°, 45°, 90°, 135°, 180°, 225°, 270°, 315°, 330°	—

Table 4.2.: Simulated environmental degradation parameters applied to vision modalities (AprilTag & YOLOv8-pose).

Parameter	Values / range	Reference value
Rain rate	5, 25, 50, 100, 200 mm/h	clear (0 mm/h)
Snow rate	0.5, 2.0, 5.0, 10.0, 25.0 mm/h	clear (0 mm/h)
Fog (meteorological visibility)	200, 100, 50, 20, 10, 5 m	clear ($V \rightarrow \infty$)

mobile test platform and the docking station in the outdoor experimental environment are shown in Figure 4.1. The docking station was fixed at a single location throughout the entire measurement campaign. Four AprilTag fiducial markers were permanently mounted at a height of 0.425 m above ground: three on the outer walls of the station (left, right, and the outward-facing back wall) and one on the inner back wall facing the station opening, so that the robot's camera can detect the station both from the cardinal approach directions outside and head-on through the opening. A technical manufacturing drawing of the docking station is provided in section A.4.



Figure 4.1.: The physical mobile test platform carrying the companion computer and sensor suite, positioned in front of the docking station in the outdoor experimental environment.

4.4.1. Mobile Test Platform and Sensor Configuration

All experiments were performed on a single mobile test platform carrying the full sensor payload.

The robot is built on a commercial tracked base with a footprint of approximately $0.8\text{ m} \times 0.5\text{ m}$. A continuous-track, skid-steered drive was chosen over a wheeled configuration for its traction and ground compliance on the uneven, low-grip outdoor terrain that characterises the deployment environment.

Computation and flight control are consolidated on a Holybro Pixhawk Jetson Baseboard [100], which mounts a Pixhawk 6X Pro flight controller and an NVIDIA Jetson Orin NX (Engineering Reference Developer Kit) companion computer on a single carrier board, linked internally over Ethernet. This division of labour realises in hardware the real-time/edge split of the system architecture: the Pixhawk runs the PX4 firmware and closes the hard-real-time

actuation loop, while the Jetson Orin NX runs the ROS 2 perception and docking stack, including the GPU-accelerated AprilTag detector, which depends on the Jetson’s CUDA cores. Power to the onboard electronics is regulated from the nominal 24 V battery using a Holybro UBEC 12A step-down regulator [101], and a Holybro PWM Extension Board [102] is connected to the baseboard to route additional PWM signals to the actuators. The complete power-distribution and signal-interconnection diagram is provided in Figure A.2.



Figure 4.2.: The Holybro Pixhawk Jetson Baseboard complete set [103].

The three localisation sensors are mounted at fixed, surveyed heights along the platform’s vertical axis, summarised in Table 4.3. The static transforms between these sensor frames and the robot base frame are published from the platform’s Unified Robot Description Format (URDF) model (subsubsection 4.4.2), so that detections from any sensor resolve into a common robot-centric frame.

Table 4.3.: Sensor payload of the mobile test platform and the mounting height of each device above ground. The localisation sensors are arranged so that the camera looks slightly downward onto the AprilTag markers at 0.425 m.

Sensor	Model	Height	Role
RGB camera	Intel RealSense D435i [73]	0.56 m	AprilTag detection
LiDAR	Livox Mid-360	0.35 m	Station detection, SLAM
RTK-GPS antenna	ArduSimple RTK Starter Kit (u-blox ZED-F9P [104, 105])	0.34 m	Ground-truth reference
IMU / barometer	Pixhawk 6X Pro (internal)	—	Odometry
Battery telemetry	LiTime Battery Management System (BMS) (Bluetooth)	—	State-of-charge monitoring

The RTK-GPS chain that provides the ground-truth reference is configured as follows. The receiver is part of the ArduSimple Medium Range Radio - RTK Starter Kit [105], which is based on the u-blox ZED-F9P module [104]. The ANN-MB-00 multiband antenna feeds the ZED-F9P receiver, which is wired directly to the Jetson Orin NX: it is powered from the Orin’s GPS2 5 V supply, streams its GNSS position/velocity solution to the Orin over UART, and is configured and monitored over USB. The NTRIP correction stream needed for an RTK-fixed solution is pulled from the internet by the Jetson Orin NX and forwarded to the receiver over this same USB link. In RTK-fixed mode this yields centimetre-level absolute positioning,

which is what makes the RTK-GPS estimate usable as the baseline against which the camera- and LiDAR-based docking measurements are compared.

4.4.2. On-Board Process Orchestration and Detection Pipelines

The same docking software used in simulation also runs on the physical robot. The architecture is consistent: both localisation pipelines publish the same relative-pose message on `/localization/docking_pose`, allowing the docking Behavior tree of subsection 4.5.6 to be reused unchanged. At the hardware boundary, Gazebo sensor plugins are replaced by physical drivers, and AprilTag detection is moved onto the GPU of the companion computer.

Setup of the ROS2 Workspace and Nodes

All on-board software runs on the companion computer under Ubuntu 22.04 and ROS 2 Humble. The code is organised into two colcon workspaces: a main workspace (`ws_sensor_combined`) and a separate vendor workspace (`ws_livox`) holding the Livox driver. A single launch file (`startup_combined.launch.py`) manages PX4 bridging, BMS telemetry, state publication, and remote monitoring bridges.

The detection and SLAM stacks are managed on demand by a dedicated `process_manager` node, allowing operators to bring individual sensors up and down during experiments. The manager runs a network preflight check (host-IP presence and ICMP reachability to the sensor) before launching the Livox driver to ensure cabling or addressing faults fail loudly. Each physical device is served by its hardware-specific driver node, and downstream consumers subscribe to standardised topics.

LiDAR Station-Detection Node

The LiDAR detector (`station_detection_LIDAR`) consumes the scan on `/livox/lidar`. Incoming points are height-filtered to remove ground returns and overhead clutter, then projected onto a 2D occupancy grid with 0.10 m cells. Connected-component labelling groups occupied cells into candidate clusters, and principal-component analysis (PCA) yields an oriented bounding box, scored against the known physical extent of the station (1.25 m $\times$ 1.25 m). The pose is refined by registering the reference model using ICP seeded from the visible-point centroid. The node publishes the estimated pose, a confidence value in $[0, 1]$, and visualization markers.

AprilTag Station-Detection Node

On the physical robot, AprilTag detection runs on the edge computer's GPU to prevent CPU saturation. The RealSense camera publishes RGB frames at 1280×720 and 6 fps to balance pixel footprint and computational load. Detection uses the NVIDIA Isaac ROS AprilTag pipeline: a `RectifyNode` and the CUDA-backed `AprilTagNode` are co-located in a single container utilizing NITROS zero-copy transport to bypass CPU round-trips. Detections are resolved to the common relative-pose message.

4.4.3. Outdoor Field Campaigns

Two field campaigns were conducted to systematically evaluate the localisation pipelines:

Experiment 1: Static Sensor Accuracy Sweep

To characterise stand-alone localisation accuracy across the docking approach envelope, a static sweep campaign was conducted. A polar measurement grid centred on the docking station was marked on the ground (Figure 4.3). The robot was placed at 36 pre-measured positions defined by six angles ($0^\circ, 30^\circ, 45^\circ, 90^\circ, 135^\circ, 180^\circ$) and six distances (1, 2, 3, 5, 7, 10 m). At each position, the robot remained stationary while 10 s of data were recorded simultaneously from all three sensors. The time-series approach provides a robust sample mean to estimate systematic errors and the sample covariance needed to characterise noise.

Experiment 2: Custom Outdoor Range and Orbit Sweeps

A second campaign was conducted to systematically benchmark range and angular detection capabilities under real-world conditions:

- **Straight-on approach range sweep:** The robot was placed along the straight-on approach (0°) at distances from 1 m to 10 m in 1 m steps.
- **Azimuth orbit sweep:** The robot was positioned at a fixed radius of 3 m around the station centre at 10 distinct azimuth angles: $0^\circ, 30^\circ, 45^\circ, 90^\circ, 135^\circ, 180^\circ, 225^\circ, 270^\circ, 315^\circ, 330^\circ$.

At each sweep position, sensor logs and detection outputs were captured for analysis, allowing direct comparison of detection rates, confidence margins, and estimated orientations.

4.5. Simulation-Based Experimental Setup

To systematically evaluate the localisation pipelines across their geometric and environmental envelopes, a high-fidelity simulation testbed was developed. Simulation offers three key advantages over physical testing: exact analytical ground truth, perfectly repeatable robot placement via teleportation, and the ability to decouple geometric parameters from atmospheric degradation.

4.5.1. Simulation Environment and Headless Rendering

The simulation runs in Ignition Gazebo Fortress (gz-sim 6.16) under ROS 2 Humble, using a differential-drive robot model and a fixed docking station. Four tag36h11 AprilTag markers (edge length 0.13 m) are fixed at a height of 0.425 m above ground, matching the physical station: three on the outer walls—*tag_left* (id 3), *tag_right* (id 4), and the outward-facing *tag_outer_back* (id 2)—and *tag_back* (id 1) on the inner back wall facing the opening. The simulated docking station is shown in Figure 4.4.

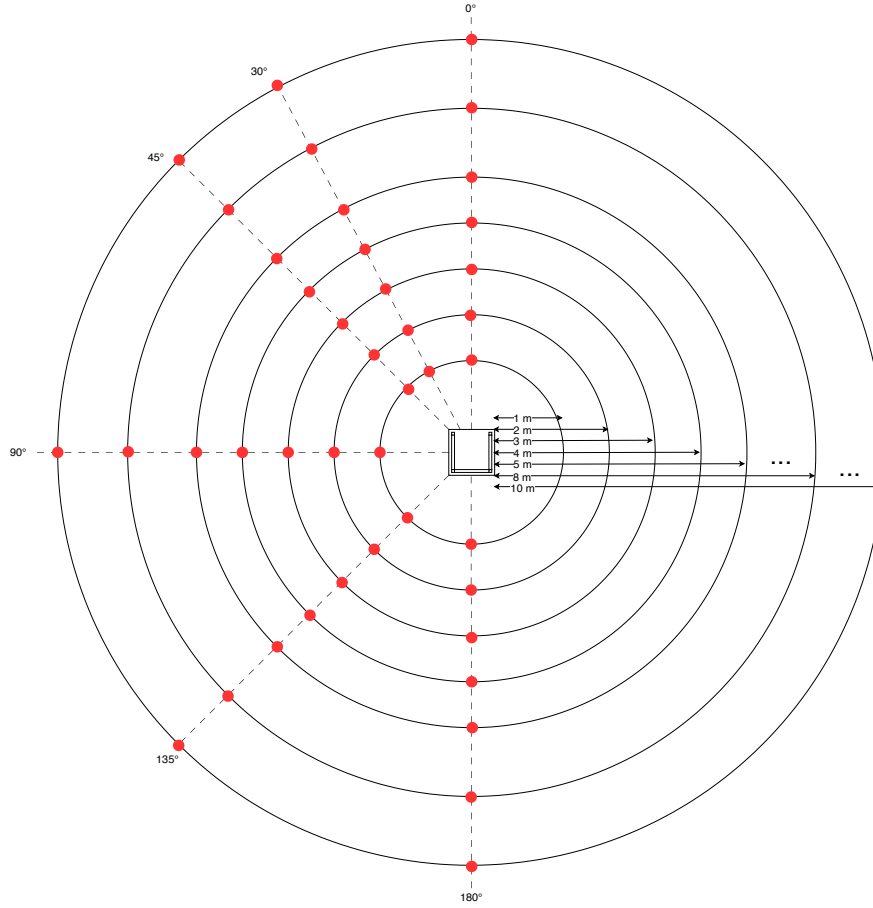


Figure 4.3.: Top-down view of the static sweep measurement grid. The docking station (red) is at the origin; blue squares mark the 36 robot positions at which sensor data were recorded. Marker symbols on the station indicate the four AprilTag faces.

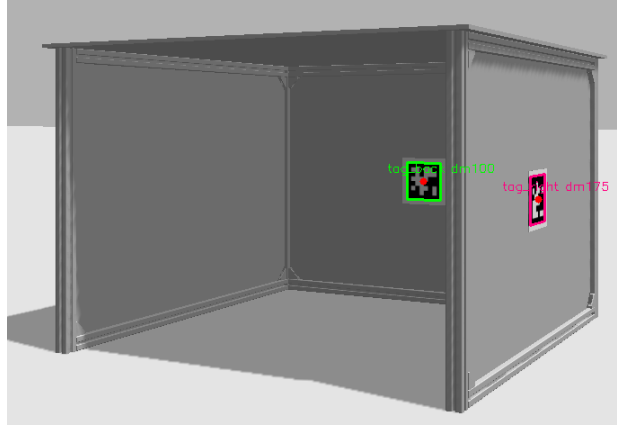


Figure 4.4.: The docking station with its four AprilTag fiducial markers: three mounted on the outer walls and one on the inner back wall facing the opening.

Headless Execution

Simulation runs were executed on a Lenovo Yoga laptop running Ubuntu 22.04 with an NVIDIA RTX 3050 (6 GB) GPU. The simulation is executed fully headless using a virtual framebuffer (Xvfb) combined with the Ogre2 render engine and Mesa software rasterisation (LIBGL_ALWAYS_SOFTWARE). This headless setup automates sweeps by teleporting the robot, rendering camera streams, and exporting CSV telemetry files. Camera resolution is a configurable parameter, and camera intrinsics are derived dynamically at runtime to prevent projection errors at non-standard resolutions.

Technical rendering and API quirks were resolved to ensure data integrity:

- **Command CLI Routing:** Commands were strictly routed through the Ignition CLI (`ign`) rather than the legacy Gazebo CLI (`gz`) to prevent silent teleportation failures.
- **Data Flow Verification:** Verification of rendering activity was performed by directly checking topic publish rates rather than merely verifying topic registration.
- **LiDAR Visualization Interference:** The GPU-LiDAR's ray-fan visualization was found to contaminate camera frames; the LiDAR sensor plugin was therefore disabled during vision-only sweeps.

4.5.2. Simulation Software Architecture

The simulation bridges Gazebo sensor streams into ROS 2 via `ros_gz_bridge`. The core design abstracts the localisation source from the downstream controller: the AprilTag, YOLOv8-pose, and geometric LiDAR (ICP) pipelines all publish their relative pose estimations onto the same `/localization/docking_pose` topic. A launch argument (`localization_source`) selects the active localizer, facilitating direct head-to-head comparisons under identical path trajectories and controllers. For sweeps, the docking controller is disabled (`run_controller:=False`) to prevent the robot from driving away from commanded teleportation poses.

4.5.3. Simulation Parameter Sweeps

Four systematic parameter sweeps were conducted under clear-weather conditions with the docking controller disabled:

Max-distance sweep: The robot was placed along the straight-on approach to the rear wall (180°) at distances from 1.5 m to 30 m in 0.5 m steps.

Azimuth orbit sweep: The robot traced a 360° orbit at a fixed radius of 3 m around the station centre in 1° steps, pointing toward the centre.

Resolution comparison: The max-distance sweep was repeated using a high-resolution camera (1280×960 px) to evaluate resolution limits.

Tag-size comparison: The sweep was repeated with doubled tag size (0.26 m) to analyze physical scaling.

Additionally, lateral alignment micro-sweeps (± 10 mm steps) were performed at distances of 5.5 m and 7.0 m to investigate detector aliasing. Identical sweeps were replayed for the LiDAR pipeline within its expected range (max-distance up to 8.0 m and orbit steps of 5° at 3 m).

4.5.4. Learned-Keypoint YOLOv8 Detector and Training Data

To extend perception beyond fiducial marker limits, a learned-keypoint pipeline was trained to recognise the docking station's shape. A YOLOv8-pose network was trained to regress the eight semantic corners of the station's bounding cuboid directly from RGB frames.

A sim-to-real training strategy was utilized. Exact 2D projections of the 3D corners were generated analytically in simulation across various viewpoints. To study the effect of the training distribution, four variant models were trained and compared:

- **Real only:** Trained strictly on a small, hand-labelled outdoor dataset.
- **Sim only:** Trained exclusively on auto-labelled synthetic renders.
- **Sim + Real:** Standard sim-to-real training combining both datasets.
- **Sim + Real + Weather:** Augmented with synthetic weather-degraded renders to incorporate rain, snow, and fog into the training distribution.

4.5.5. Weather Augmentation: Decoupled 3D Gaussian Splatting

To systematically analyze the effects of adverse weather on vision-based perception without collapsing simulator performance, weather rendering was decoupled from Gazebo. Instead, a clean Gazebo scene was reconstructed as a 3D Gaussian Splatting (3DGS) model [106]. Weather was then injected into this model as volumetric particle fields (rain, snow, fog) using WeatherEdit [107].

This decoupled approach has three major justifications:

1. Spawning thousands of physical particles in Gazebo collapses the real-time simulation factor and fails to model the veiling effects of fog.
2. Building a static 3DGS model first keeps the underlying scene geometry identical across all weather intensities, ensuring that variations in detection rate are strictly caused by atmospheric conditions.
3. WeatherEdit utilizes physical units (precipitation rate in mm/h, visibility in m), allowing for quantitative and citable degradation benchmarking.

To ensure sharp tag reconstructions suitable for detection testing, depth supervision was utilized to resolve texture blurring along view rays, raising the reconstruction PSNR above 60.

4.5.6. Autonomous Docking Behavior Tree

The docking controller implements a sensor-agnostic behavior tree within the `docking_controller` package. It coordinates the three docking phases of Section 3.4.3, which appear in the code as the behaviour-tree stages:



Because the tree interacts solely with the generic pose interface, the behavior logic is reused unmodified across all vision and LiDAR modalities.

4.6. Graphical User Interface

To facilitate real-time monitoring and debugging during both simulation testing and physical experiments, a custom visualization dashboard was designed using Foxglove Studio. Foxglove connects directly to the running ROS 2 stack via the `foxglove_bridge` protocol, enabling low-overhead data visualization without requiring a physical monitor attached to the robot.

The layout is structured to display the primary telemetry required to assess the docking state, as illustrated in Figure 4.5:

- **Camera Feed and AprilTag Detections:** Displays the raw RGB video stream from the onboard camera overlaid with the active AprilTag detections, showing tag IDs, coordinate axes, and detection margins.
- **3D World View:** Visualizes the robot’s estimated pose, coordinate frames (TFs), the LiDAR point cloud, the ICP-detected docking station geometry, and the computed target approach trajectory.
- **System Status and Docking State:** Shows the current active state of the autonomous docking behavior tree alongside key diagnostic metrics, such as estimated range to the docking target and velocity command overlays.

4. Experimental Setup

- **Plot Panels:** Tracks sensor signals, computed errors, and control command velocities over time to diagnose controller stability.

To facilitate replication and deployment, the full JSON layout configuration files for both the control interface (Figure 4.5) and the experimental testing layout are included in the repository and detailed in section A.7.

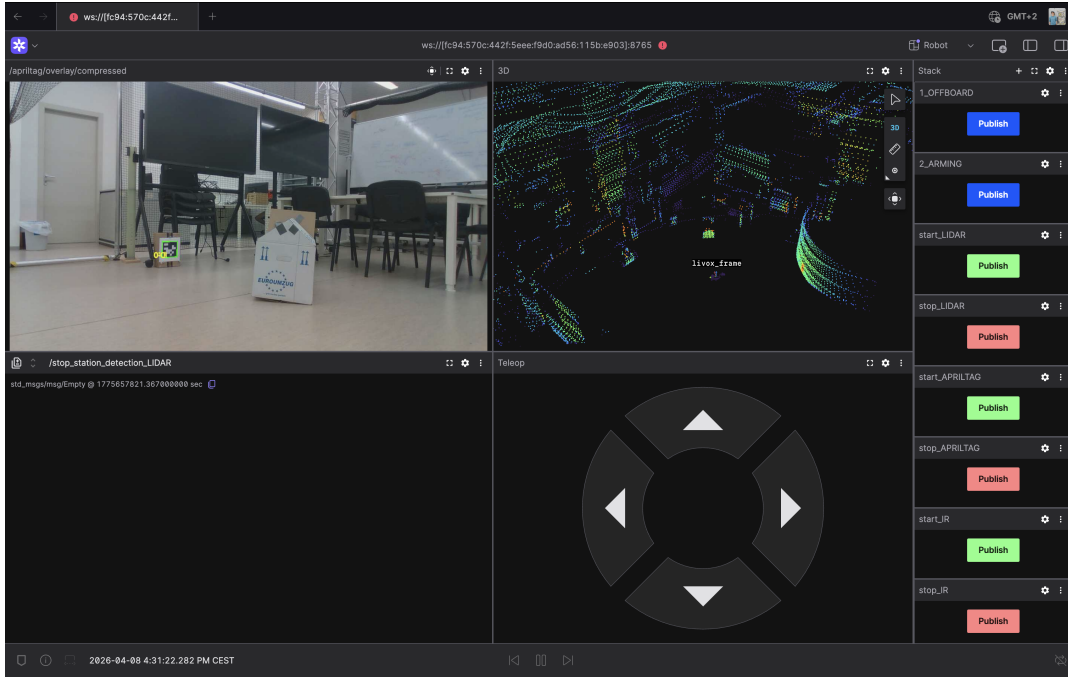


Figure 4.5.: The customized Foxglove user interface layout used for monitoring the robot’s sensor data, state transitions, and pose estimation during docking operations.

5. Evaluation and Results

This chapter reports the experimental core of the thesis (RQ2): how the candidate station detectors of Section 3.3.3 actually perform, and what that implies for how the docking-perception subsystem should be built. The experiments follow the design laid out in Chapter 4 and exploit the controlled simulation testbed of Section 4.5, whose exact per-frame ground truth and bit-for-bit repeatability make a fair, single-variable comparison possible in a way the outdoor campaign cannot.

The results are organised around the central thesis that no single detector spans the full precision range separating long-range capture from terminal alignment (Section 2.3), so the question is not *which sensor wins* but *which detector owns which range band*. Three detectors are compared on identical poses and identical ground truth: the hand-crafted **AprilTag** fiducial pipeline, the marker-free **learned-keypoint** (YOLOv8 pose) pipeline, and the geometric **LiDAR** pipeline. Section 5.1 establishes the geometric detection envelope of the two vision detectors—their range and angular coverage, the resolution and tag-size levers that govern the fiducial range, and the sub-pixel alignment limit at its edge; Section 5.2 degrades that envelope with adverse weather; Section 5.3 quantifies the price of the learned detector—its dependence on annotated training data; and Section 5.4 characterises the LiDAR pipeline and dissects its failure modes. Section 5.6 then closes the loop with a real-world counter-check—an outdoor multi-sensor accuracy sweep against RTK-GPS ground truth together with physical re-runs of the detection sweeps—confirming on physical hardware that the simulation findings hold outside the simulator. Section 5.7 adds the final axis—the computational cost of running each detector on the edge platform—before Section 5.8 synthesises all of these into the range-layered perception strategy that the data support.

A note on the docking maneuver itself. This chapter characterises the docking *perception* subsystem—the per-detector accuracy, range and robustness on which a successful maneuver depends—rather than the statistics of the closed-loop maneuver. The full closed-loop sequence (the behaviour tree and controller of Section 4.5.6) is exercised in simulation, where exact per-frame ground truth makes the end-to-end alignment error directly measurable and the runs are cheap and repeatable; on the physical robot it is exercised only as a small qualitative feasibility check (3runs), because outdoor ground truth for the full maneuver is unavailable (Section 5.6.3). A statistical *field* characterisation of the closed-loop terminal accuracy is deliberately out of scope: exact per-maneuver ground truth is hard to obtain outdoors, and the number of repetitions needed for reliable success-rate and accuracy figures would be impractical to collect. The terminal-alignment requirement of Section 3.1.1 is therefore assessed through the controlled static per-detector studies of this chapter, and the maneuver dynamics through simulation.

5.1. Geometric Detection Envelope: Fiducial versus Learned Vision

The first question is purely geometric: under perfectly clear conditions, *where* can each vision detector see the station at all, and how precisely does it localise it there? Two sweeps answer this—a straight-on max-distance sweep that probes range, and a full azimuth orbit that probes angular coverage—both run with the controller disabled so that the robot is teleported to exact, repeatable poses (Section 4.5.3). Throughout, detection *rate* (the fraction of frames in which the station is found) is reported separately from localisation *error* (how accurately it is placed when found), because the two detectors trade these against one another in opposite directions.

5.1.1. Maximum Detection Distance

Figure 5.1 contrasts the two detectors along the 180° straight-on approach from 1.5 to 30 m. The qualitative difference is stark. At this reference configuration—the 640×480 px camera and the 13 cm tag of the physical station—the AprilTag detector holds a 100 % detection rate only out to 4.5 m, beyond which it collapses abruptly to zero: past that distance the tag subtends too few pixels for the quad detector to resolve its internal 6×6 code grid, and detection fails completely rather than degrading gracefully. This short reference range is not an intrinsic property of the fiducial but of its apparent pixel footprint, and Section 5.1.3 shows it can be extended in simulation either by raising the camera resolution or by enlarging the tag; the deployed robot’s real-world fiducial range nonetheless stayed close to this reference (around 5 m, Section 5.6.3). The learned detector, keyed on the coarse silhouette of the whole structure rather than on a high-frequency pattern, sustains 100 % detection across the *entire* swept range to 30 m—in practice the difference between capturing the station from across a field and not seeing it until the final approach.

The precision ranking is exactly inverted. Within its working range the AprilTag is roughly an order of magnitude more precise in pixel space, holding a mean centre error of 0.81 px—which the PnP solver converts to a mean translation error of 4.6 cm—against the learned detector’s mean keypoint error of 8.48 px across the full range. Two features of the learned-detector error curve in the lower panel of Figure 5.1 matter for docking. First, beyond ~ 3 m the keypoint error settles to a low and stable 1–4 px, so at mid-range the learned detector is both available and reasonably accurate. Second, at very close range the error *explodes*—to 268 px at 1.5 m—because the station’s bounding cuboid overflows the camera frame and its corners must be regressed from outside the image, exactly where the AprilTag is at its most precise. The two vision detectors are therefore not competitors but complements along the range axis: the learned detector for long-range capture, the fiducial for short-range precision.

5.1.2. Azimuth Coverage

Range is only one half of the envelope; a docking station must also be found from whatever *direction* the robot approaches. Figure 5.2 reports a full 360° orbit at a fixed radius of 3 m. The AprilTag detection rate is punctured by repeated drop-outs to zero, averaging 84.4 % over

the apriltag gives a pose, yolo only gives landmarks from which one could try to determine

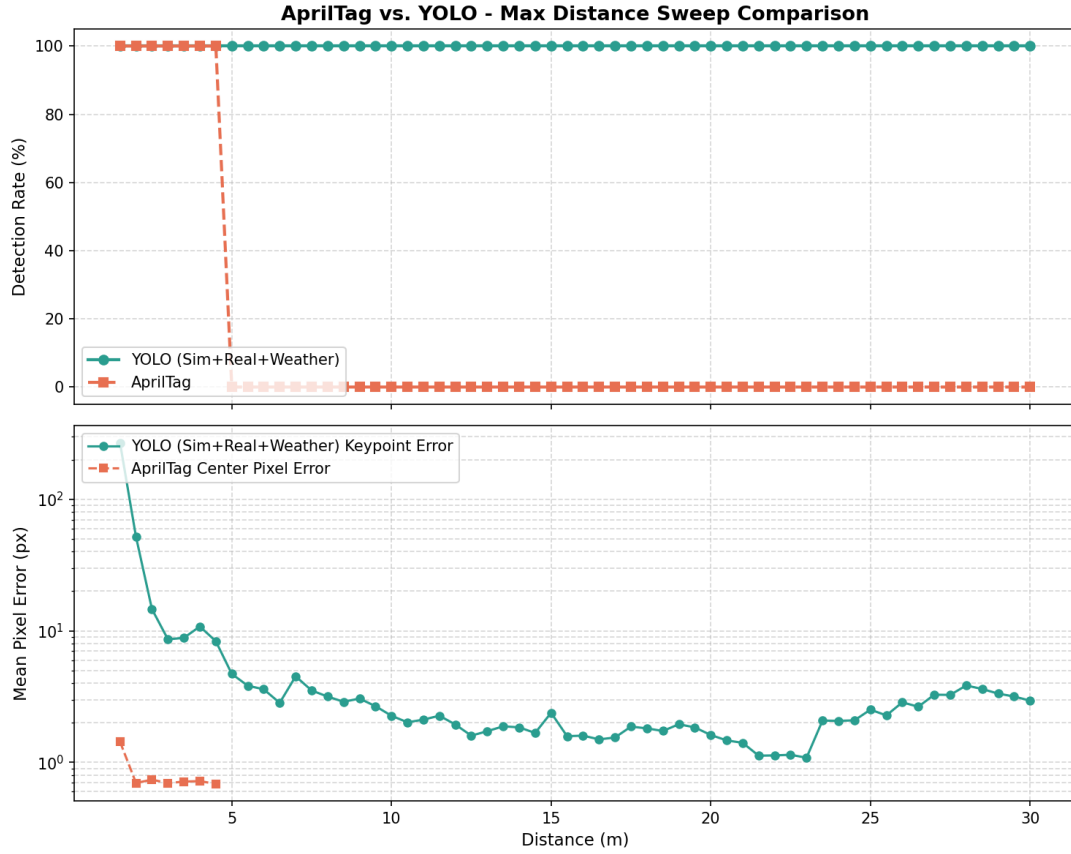


Figure 5.1.: Max-distance sweep along the straight-on approach. **Top:** detection rate—at the 640×480 reference resolution the AprilTag cuts off abruptly at 4.5 m while the learned detector holds 100 % to 30 m. **Bottom:** mean pixel error (log scale)—the AprilTag is sub-pixel within its range, whereas the learned detector settles to a few pixels beyond 3 m but diverges at close range as the station leaves the frame.

the orbit: each of the four wall-mounted tags is only legible within a limited cone around its surface normal, and in the hand-off sectors between adjacent faces—and at the grazing angles where a tag is seen edge-on—no tag resolves and detection fails. The learned detector, which recognises the station as a single object rather than four separate markers, achieves an unbroken 100 % coverage across all azimuths.

Once again precision and coverage trade off. The AprilTag’s centre error stays flat at 0.81 px whenever it detects at all, independent of viewing angle. The learned detector’s keypoint error instead varies periodically with azimuth: it is lowest (5–8 px) at the four face-on orthogonal views, where the cuboid corners project cleanly, and peaks (~ 23 px) at the oblique corner-on views where several corners are mutually self-occluded, for an orbit mean of 11.95 px. The learned detector thus guarantees that *something* is always found from any direction—the property that matters for the searching and capture phases—while the fiducial guarantees precision only within its legibility cones.

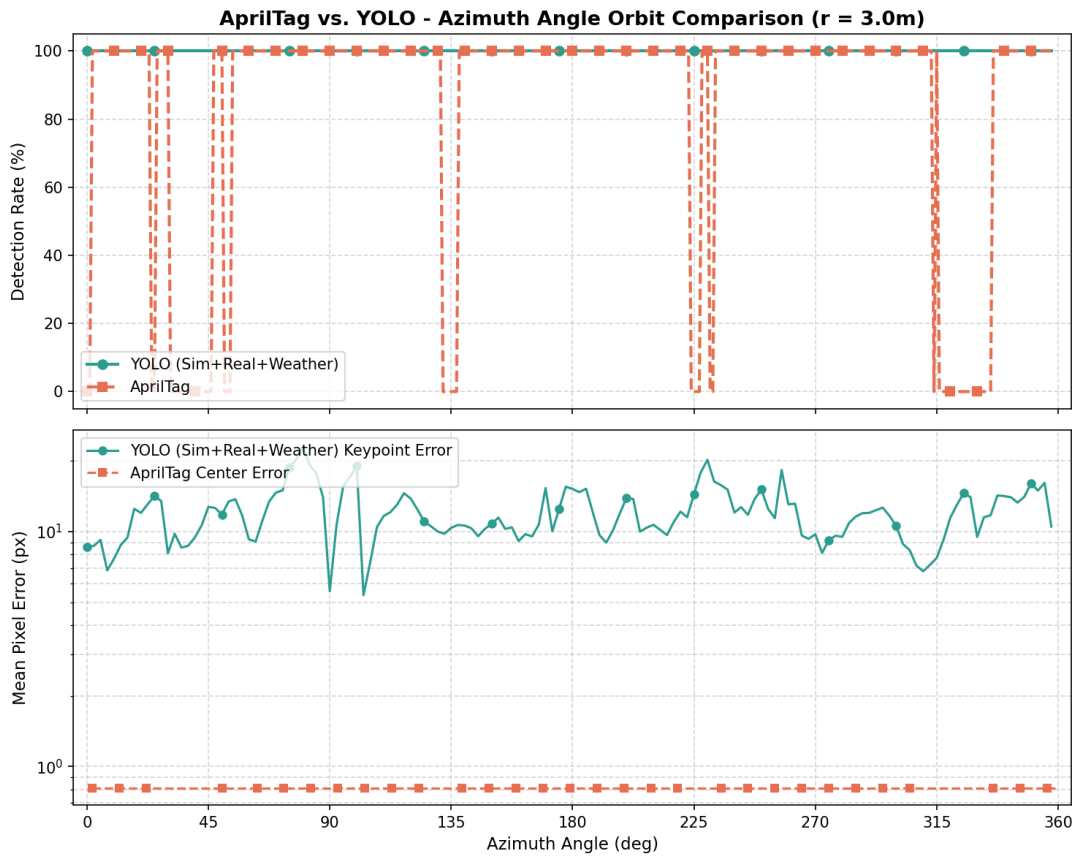


Figure 5.2.: Azimuth-orbit sweep at $r = 3$ m. **Top:** the AprilTag detection rate drops to zero in the hand-off sectors between the four wall tags (mean 84.4 %), whereas the learned detector holds 100 % across the full circle. **Bottom:** the AprilTag centre error is a flat 0.81 px; the learned-detector keypoint error oscillates with viewing angle, lowest at the face-on views and highest at oblique corner views.

note here that in our current configuration yolo does recognise the station from every angle but can't tell us reliably from which angle we are seeing the station. This should not pose a significant challenge though, since we can assume that the GPS based

5.1.3. Levers on the Fiducial Range: Resolution and Tag Size

The 4.5 m reference range of the AprilTag is not intrinsic to the fiducial; it is set by the tag’s apparent *pixel* footprint, which two parameters control directly—the camera resolution and the physical tag size. Both were swept (Section 4.5.3) by repeating the straight-on max-distance run with exactly one of them changed, so that the resulting range change is attributable to that lever alone.

Camera resolution. Figure 5.3 contrasts the reference 640×480 run with a 1280×960 run. Doubling the linear resolution extends the 100 %-detection range from 4.5 m to 12 m—a $\sim 2.7\times$ extension that tracks the doubled pixel footprint of the tag. The cost is borne entirely at the far end: the decision margin stays high until ~ 8 m and then declines, and the PnP translation error—sub-centimetre within the reference range—grows to ~ 0.2 m by 10 m, because at that distance each pixel of corner-localisation error subtends a far larger metric distance. Higher resolution therefore buys *range*, not accuracy: it lets the fiducial be *detected* much further out, but the precise sub-centimetre regime remains confined to short range.

Tag size. Figure 5.4 instead doubles the tag border from 13 cm to 26 cm at the reference resolution. The effect mirrors the resolution sweep almost exactly: the range extends from ~ 4.5 m to ~ 11.5 m, again because the larger marker projects onto more pixels at a given distance, with the same growth of far-range position error. The two levers are thus interchangeable—both act on the single underlying quantity, the tag’s pixel footprint—and either one suffices to lift the fiducial range into the ~ 12 m band. Although the physical robot runs its camera at a higher resolution (1280×720 , Section 4.4.2), the outdoor sweep found its real-world fiducial range to be only ~ 5 m (Section 5.6.3)—close to the low-resolution reference rather than the optimistic simulated 12 m—so the conservative ~ 4.5 m figure is the one carried into the synthesis (Section 5.8).

5.1.4. The Sub-Pixel Detection Limit: Lateral Alignment Micro-Sweep

Between the reliable range and the hard cut-off, detection becomes *intermittent* rather than simply stopping—a behaviour the alignment micro-sweep of Section 4.5.3 isolates. At the two distances where the reference run showed this flicker (5.5 and 7.0 m), the robot was stepped in ± 10 mm camera-plane increments while everything else was held fixed. Figure 5.5 shows the result. At 5.5 m the tag spans only ~ 13 px, so one pixel corresponds to ≈ 10 mm on the ground, and a single 1 cm lateral step is enough to flip detection from certain (1.00) to impossible (0.00): the half-resolution quad detector aliases the tag edge, and whether a given sub-pixel phase happens to land on a resolvable or an aliased sampling grid is what decides detection. The same flip recurs at 7.0 m (tag ~ 10 px, 1 px ≈ 13 mm). This is the mechanism behind the abrupt, all-or-nothing range cut-off, and it carries a concrete operational implication: near the range limit a sub-centimetre pose change can extinguish the fiducial, so the approach must not be planned to dwell at the very edge of the fiducial

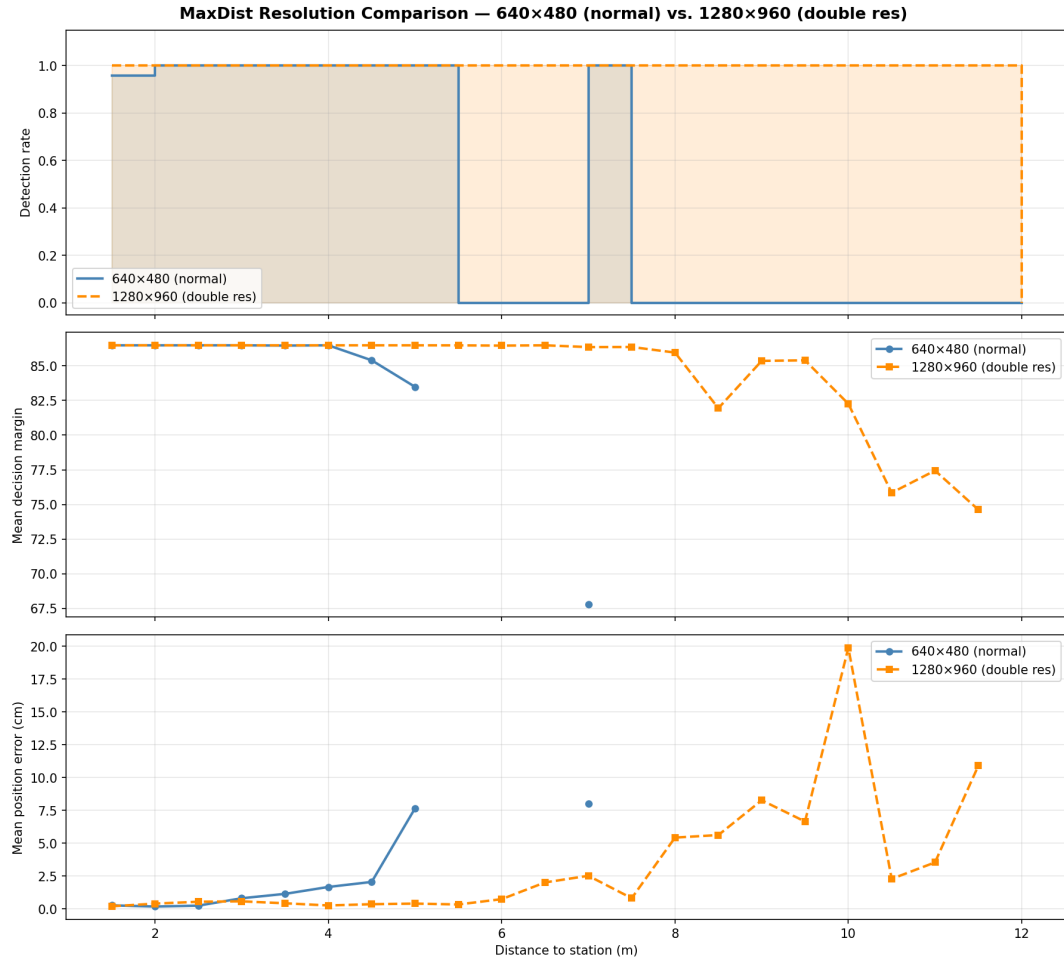


Figure 5.3.: Effect of camera resolution on the AprilTag max-distance sweep. At the 640×480 reference (blue) detection ends at 4.5 m; at 1280×960 (orange) it extends to 12 m.

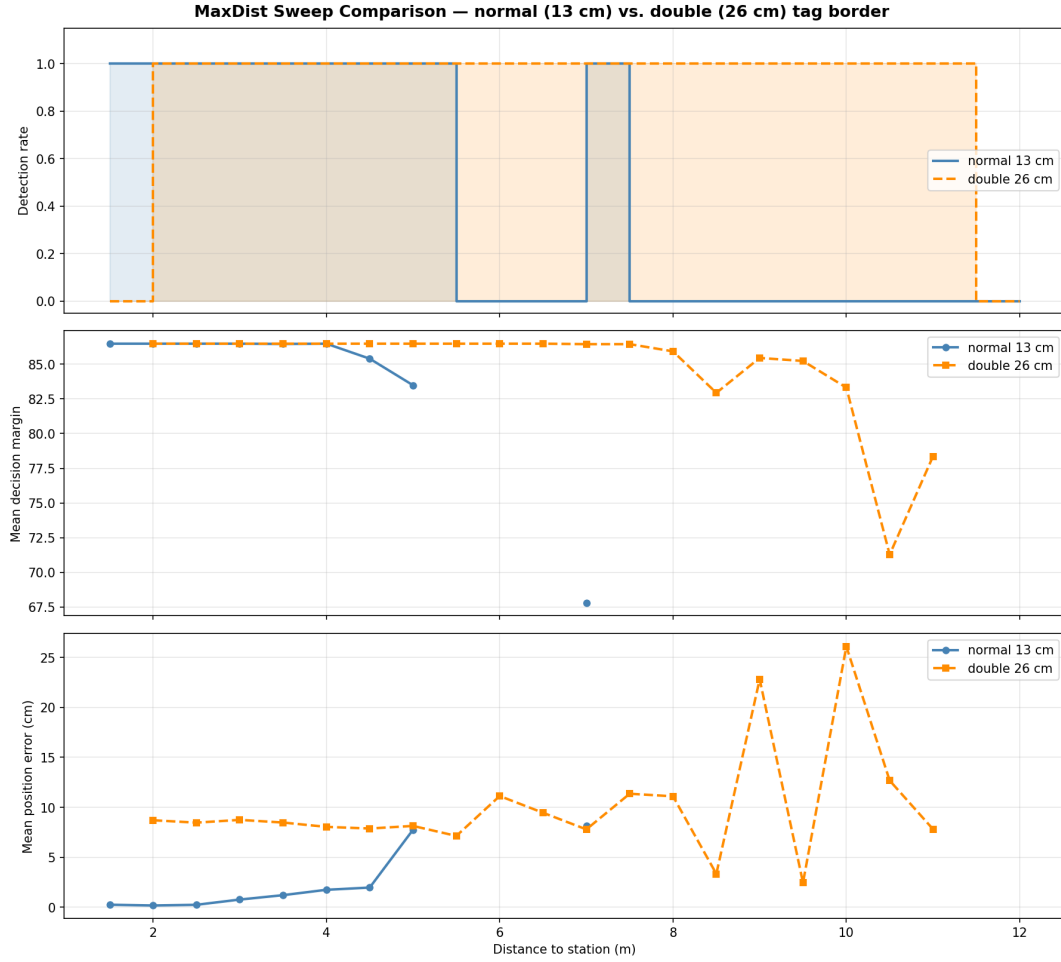


Figure 5.4.: Effect of tag size on the AprilTag max-distance sweep. Doubling the tag border from 13 cm (blue) to 26 cm (orange) extends the detection range from ~ 5 m to ~ 11.5 m, mirroring the resolution lever—both act on the tag’s apparent pixel footprint. The difference between both is that doubling resolution needs more compute while doubling tag size sacrifices mean positional accuracy at low distances.

envelope but to carry a detection buffer into it—exactly the guidance the docking behaviour tree encodes (Section 4.5.6).

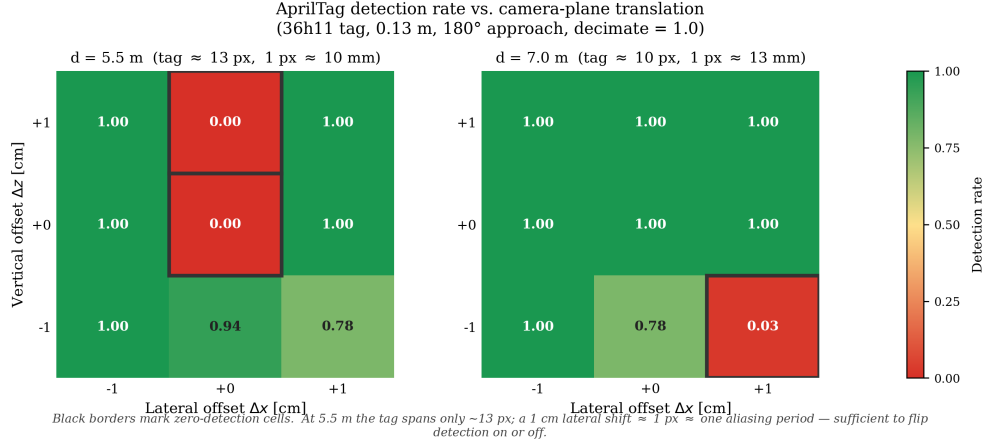


Figure 5.5.: AprilTag detection rate under ± 10 mm camera-plane translation at 5.5 m (left) and 7.0 m (right). Black-bordered red cells are zero-detection. At 5.5 m the tag spans only ~ 13 px (1 px ≈ 10 mm), so a 1 cm lateral shift—one aliasing period—is enough to flip detection on or off, the mechanism behind the abrupt range cut-off seen in Figure 5.1.

5.2. Robustness to Adverse Weather

The geometric envelope above is measured under perfectly clear conditions; the deployment environment is not. Using the decoupled 3D Gaussian-Splatting weather pipeline of Section 4.5.5, rain, snow and fog were injected onto the *same* fixed reference pose, so that any change in detection is attributable to the atmosphere alone and not to a change in viewpoint. Figure 5.6 reports the resulting detection rates and errors across the three weather axes.

The pattern is consistent across all three. The AprilTag, dependent on a high-contrast printed pattern, degrades sharply as the atmosphere veils that pattern: its mean detection rate falls to 68.1 % under rain (collapsing to 1.7 % at the heaviest 200 mm/h), 88.6 % under snow, and 57.1 % under fog, where it fails completely once meteorological visibility drops below 50 m. The learned detector is far more robust: it holds 100 % detection across every rain and snow rate tested, and continues to detect through fog down to 10 m visibility, only failing at the extreme 5 m case. Its keypoint error, meanwhile, is essentially *flat* at ≈ 37 px across every weather condition and intensity (varying by under 1 px from the 37.5 px clear-weather value)—the veiling reduces contrast but does not move the regressed corners, so the detector degrades in a graceful, predictable way rather than failing outright.

This is the decisive robustness argument for the learned detector. Where the fiducial’s precision is its strength in clear close-range conditions, its brittleness under exactly the rain, snow and fog of the target environment (Section 3.1.2) disqualifies it as the sole long-range

detector. The learned detector keeps the station in view through weather that blinds the marker entirely, at a stable if coarse localisation accuracy.

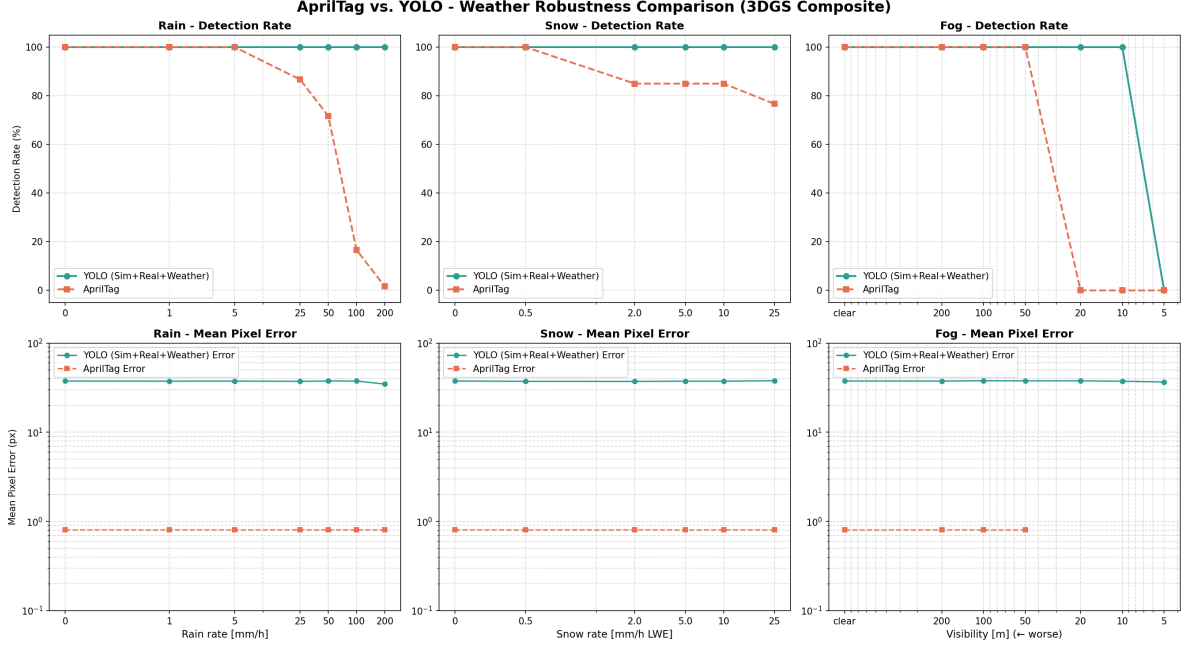


Figure 5.6.: Weather robustness across rain, snow and fog (3DGS composite). **Top row:** detection rate—the AprilTag (orange) degrades sharply with weather intensity and fails completely in heavy rain and fog, while the learned detector (teal) holds near-100 % until the most extreme cases. **Bottom row:** mean pixel error—the AprilTag stays sub-pixel *when* it detects, but the learned detector’s error is flat at ≈ 37 px regardless of condition, i.e. it degrades gracefully rather than dropping out.

Table 5.1 consolidates the head-to-head across all three evaluation axes. Read as a whole it states the complementarity precisely: the AprilTag is one to two orders of magnitude more precise but is range-limited, angle-limited and weather-brittle; the learned detector is robust in range, angle and weather but an order of magnitude coarser.

5.2.1. LiDAR under Adverse Weather: Evidence from the Literature

The weather study above degrades only the two *camera* detectors, because the 3D Gaussian-Splatting pipeline of Section 4.5.5 acts on RGB imagery; the LiDAR detector was characterised geometrically (range and orientation, Section 5.4) but not under rain, snow or fog. A controlled weather evaluation of the LiDAR pipeline—injecting physically modelled scattering into the point cloud, as the augmentation methods of [108, 109, 110] make possible—is deferred to future work. To keep the sensor trade-off complete in the meantime, this subsection synthesises the published evidence on how LiDAR degrades under weather and how that

Table 5.1.: Head-to-head summary of the AprilTag and the best learned detector (SIM+REAL+WEATHER) across the three evaluation axes. Lower pixel error and higher detection rate are better; the bold entries mark the winner on each row. The AprilTag detection range is quoted at the 640×480 reference resolution; raising the resolution or tag size extends it in simulation (Section 5.1.3), but the real-world range stayed near 5 m (Section 5.6.3). The AprilTag wins precision; the learned detector wins range, coverage and weather robustness.

Axis	Metric	AprilTag	Learned
Max distance	Detection range	4.5 m	30.0 m
	Mean pixel error	0.81 px	8.48 px
	Mean transl. error	4.6 cm	—
Azimuth orbit (3 m)	Mean detection rate	84.4 %	100.0 %
	Mean pixel error	0.81 px	11.95 px
Rain	Avg. detection rate	68.1 %	100.0 %
Snow	Avg. detection rate	88.6 %	100.0 %
Fog	Avg. detection rate	57.1 %	85.7 %

degradation differs in kind from the camera failures measured above.

How LiDAR degrades. A LiDAR pulse at 905 or 1550 nm must traverse the same atmosphere twice, and hydrometeors and airborne particulates both *attenuate* that pulse (reducing range and point-cloud density on the target) and *backscatter* it (injecting spurious returns from the medium itself). Controlled fog-chamber and real-driving measurements quantify the first effect: Kutila et al. report roughly a 50 % reduction in target detection performance for 905 nm automotive LiDARs once fog reduces meteorological visibility, with 1550 nm systems faring better owing to their higher available pulse energy under eye-safety limits [111]. Heinzler et al. and Jokela et al. characterise the second effect, showing that dense fog and heavy rain thin the returns from distant structure while simultaneously producing a halo of near-sensor clutter, and that snowfall scatters individual flakes into the foreground as false points [112, 113]. For the dry, dusty conditions characteristic of field and agroforestry operation, Phillips et al. document four distinct LiDAR behaviours in airborne particulates, from full penetration to complete occlusion depending on particulate density—a degradation mode with no camera analogue but directly relevant to a ground robot kicking up soil during the approach [114].

How this differs from the camera failures. The decisive contrast is that the two modalities fail along orthogonal axes. The camera is a *passive* sensor whose weather brittleness—measured above as the AprilTag’s collapse to 1.7 % detection in heavy rain and total failure

below 50 m fog visibility—stems from loss of *contrast*: rain streaks, a fog veil or solar glare wash out the very pattern the detector depends on. LiDAR, being an *active* ranging sensor, is by construction immune to that illumination failure (it is the reason it detects at 100 % regardless of lighting in Section 5.4), but it pays instead with attenuation and backscatter in the propagation medium. The large multimodal benchmark of Bijelic et al.—10,000 km of driving plus a controlled fog chamber spanning camera, LiDAR and radar—makes this asymmetry explicit: under fog the modalities distort *differently and at different rates*, so no single sensor degrades gracefully across all conditions and robust perception comes from fusing their complementary failure modes rather than trusting any one [115]. The survey of Zhang et al. systematises the same picture across the literature: cameras are dominated by illumination- and contrast-driven failure, LiDARs by scattering- and attenuation-driven failure, and the two are therefore most useful in combination [116].

Implication for the docking system. This published evidence reinforces, rather than overturns, the range-layered division of labour of Section 5.8. In the same rain and fog that blind the high-contrast AprilTag, the LiDAR would retain usable mid-range geometric detection—its illumination independence is unaffected—but weather-induced attenuation would lower the already-tight 3 m range ceiling found in Section 5.4.1, and the added near-sensor clutter would aggravate exactly the sparse-return, stray-cluster contamination identified as the pipeline’s position-failure mechanism (Section 5.4.3). The expected ordering under weather is thus that LiDAR degrades more gracefully than the precise-but-brittle fiducial yet less gracefully than the learned detector, whose flat keypoint error survived every condition tested here—a hypothesis that the deferred point-cloud weather experiment is designed to confirm.

5.3. The Cost of Learning: Training-Data Ablation

The robustness of the learned detector is not free: unlike the AprilTag, which works out of the box, it must be *trained*, and its behaviour is governed entirely by the data it is trained on. As argued in Section 4.5.4, the binding constraint is annotation: each training frame needs the pixel coordinates of all eight cuboid corners, and on real imagery those labels must be placed by hand—laborious, error-prone, and limited in volume—whereas in simulation they are projected analytically and are free and exact. To quantify what this buys, the four detector variants of Section 4.5.4 were evaluated on the same five sweeps; Table 5.2 reports the result and Figure 5.7 shows two of the underlying curves.

Three findings follow directly from the table.

The hand-labelled corpus alone is insufficient. The real-only model fails comprehensively: it detects in only 31.0 % of the max-distance frames and 41.7 % of the orbit, with a mean keypoint error an order of magnitude worse than any other variant (76 and 228 px respectively), and it all but vanishes under snow (0.3 %). This is the predicted consequence of training a high-capacity keypoint regressor on a small, viewpoint-limited, manually annotated set: it overfits the handful of captured poses and does not generalise to the rest of the envelope. The

Table 5.2.: Training-data ablation of the learned detector across the evaluation sweeps. Each row is the identical network trained on a different corpus. The real-only model—trained on the small hand-labelled set—collapses; simulation data alone already recovers near-complete performance, and the hand-labelled real and weather sets contribute the final increments.

Training corpus	Maxdist det.	Maxdist err. (px)	Orbit det.	Orbit err. (px)	Snow det.	Fog det.
Real only	31.0 %	76.03	41.7 %	227.77	0.3 %	28.6 %
Sim only	98.3 %	7.02	100.0 %	19.23	100.0 %	85.7 %
Sim + Real	100.0 %	8.32	100.0 %	15.05	100.0 %	85.7 %
Sim + Real + Weather	100.0 %	8.48	100.0 %	11.95	100.0 %	85.7 %

red trace in both panels of Figure 5.7 shows this concretely—it tracks the other models only over the narrow band of distances and angles that the real captures happened to cover, and drops to zero everywhere else.

Simulation data carries almost all of the load. Trained purely on free, exactly labelled synthetic frames, the sim-only model already reaches 98.3 % and 100 % geometric detection at single-digit pixel error, and—because the weather renders are generated the same way—100 % snow detection. This is the central justification for the simulation testbed: the expensive, error-prone manual annotation that crippled the real-only model is almost entirely sidestepped by generating the data where ground truth is analytic.

Real and weather data supply the final increments. Adding the small hand-labelled real set lifts geometric detection from 98.3 % to a full 100 % and sharpens the orbit error ($19.2 \rightarrow 15.1$ px), closing the residual sim-to-real appearance gap; adding the weather renders sharpens it further to the best 11.95 px and secures perfect rain and snow robustness. The one honest cost is a marginal rise in *clear-weather* max-distance error ($7.02 \rightarrow 8.48$ px) as the network spreads its capacity across the harder weather-augmented distribution—a small price for the robustness gain. The practical conclusion is that the hand-labelling bottleneck, while real, can be reduced to a supplementary role: simulation does the heavy lifting and a modest real set is needed only to anchor it to the physical world.

5.4. LiDAR Geometric Detection and its Failure Modes

The two vision detectors share a camera; the third detector is the illumination-independent LiDAR pipeline, which recognises the station from its bare geometry rather than from any marker or texture (Section 4.4.2). It is the natural short-to-mid-range partner to the camera—robust to the lighting extremes that blind the vision detectors—but its reach is bounded by point-cloud sparsity, and, as this section shows, its terminal usefulness is bounded by a geometric orientation ambiguity that no amount of processing can fully remove. The pipeline

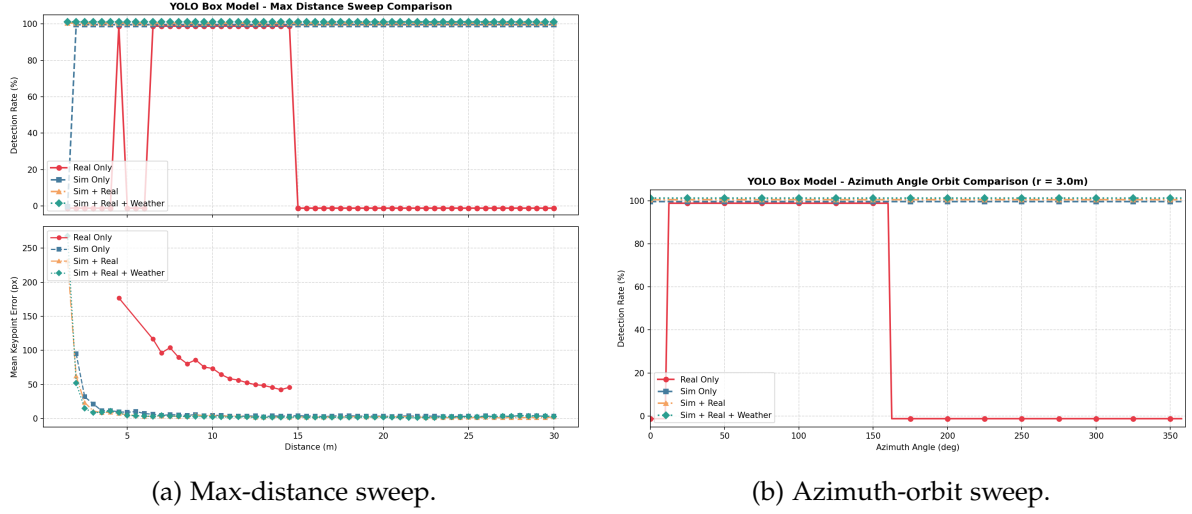


Figure 5.7.: Detection rate of the four training-data variants. The real-only model (red) tracks the others only over the narrow distance and angle band its hand-labelled captures covered, and collapses elsewhere; the three simulation-trained variants are near-indistinguishable at full coverage. Compare the abrupt real-only orbit cut-off in (b) with the AprilTag’s marker-by-marker drop-outs in Figure 5.2—a different failure mechanism with the same operational consequence.

evaluated here projects the height-filtered scan onto a 2D occupancy grid, extracts a candidate by connected-component clustering and a Principal Component Analysis (PCA) bounding box, and refines the pose by registering the full reference station model to the candidate with an ICP seeded from the visible-point centroid—a template-to-scan refinement that replaced an earlier discrete four-yaw search and removed its catastrophic position failures at the orthogonal views. This section characterises the pipeline first along range and then around a 3m orbit, and then dissects, frame by frame, both where it succeeds and where it physically cannot.

5.4.1. Detection Range

Figure 5.8 reports the straight-on range sweep. The pipeline holds a 100 % detection rate at high ICP confidence (≈ 0.97) and centimetre-level position error out to 3.0 m, but the envelope ends abruptly there: at 3.5 m the detection rate collapses to 8 % and the position error of the few surviving frames jumps to ≈ 0.5 m, and beyond 4.0 m the station yields no detection at all. The cause is purely physical: LiDAR point density falls off with distance, and the small station is hit by too few points to survive the clustering stage’s minimum-points gate. This ~ 3 m ceiling is the empirical reason the LiDAR is assigned the short-to-mid-range role, and the reason the orbit below is characterised at exactly 3 m—the hand-off distance to the long-range camera.

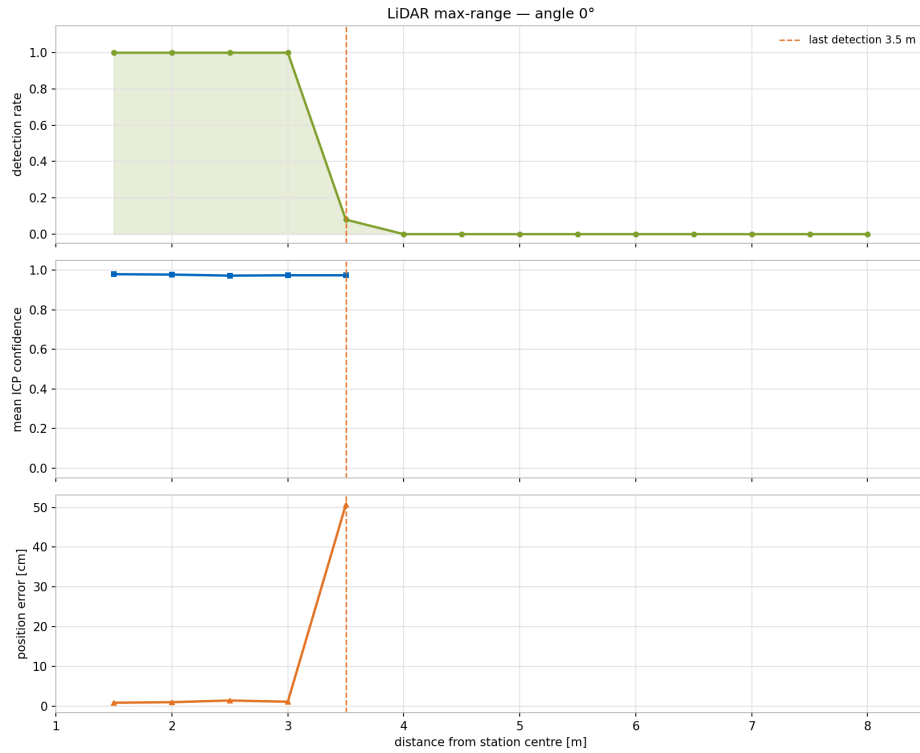


Figure 5.8.: LiDAR straight-on range sweep (0° approach). **Top:** detection rate—a complete 100 % out to 3.0 m, collapsing to 8 % at 3.5 m and to zero beyond 4.0 m as the point return on the small station thins below the clustering threshold. **Middle:** ICP confidence (≈ 0.97 while detected). **Bottom:** position error—a stable ~ 1 cm within range, spiking to ≈ 0.5 m at the 3.5 m drop-out point.

5.4.2. Angular Detection Envelope

Figure 5.9 reports the 3 m orbit in 5° steps. Detection is again a complete 100 %: at this range the pipeline always finds a candidate and reports a pose. What varies around the orbit is the *quality* of that pose, and it splits sharply between translation and orientation. The translation error (top panel of Figure 5.10) is well-behaved: it sits at the centimetre level over most azimuths and rises to 0.20–0.22 m only at the most oblique corner-on views ($\sim 50^\circ, 110^\circ, 200^\circ$), where the station is seen as a thin partial sliver. The centroid-seeded template-to-scan refinement is what keeps position this tight: it is what removed the catastrophic position failures that the earlier four-yaw search produced at the orthogonal views. The orientation error (bottom panel) is the genuine limitation, and it is *not* removed. The yaw estimate is accurate (a few degrees) only at the azimuths where enough of the station’s distinguishing geometry—above all its opening—is visible; at a large fraction of the remaining azimuths it is wrong by roughly 90° or by close to 180° . The ICP confidence partly tracks this: it falls to ≈ 0.5 at many of the ambiguous azimuths (the right panel of Figure 5.9), which gives the controller a usable signal to distrust them—but, as the next subsections show, a subset of the worst orientation errors are still reported at *high* confidence. Against the less than 5 cm / 5° terminal-alignment requirement of Section 3.1.1, the LiDAR pipeline is therefore an excellent *detector* and a reliable source of *position*, but cannot by itself supply terminal *orientation*.

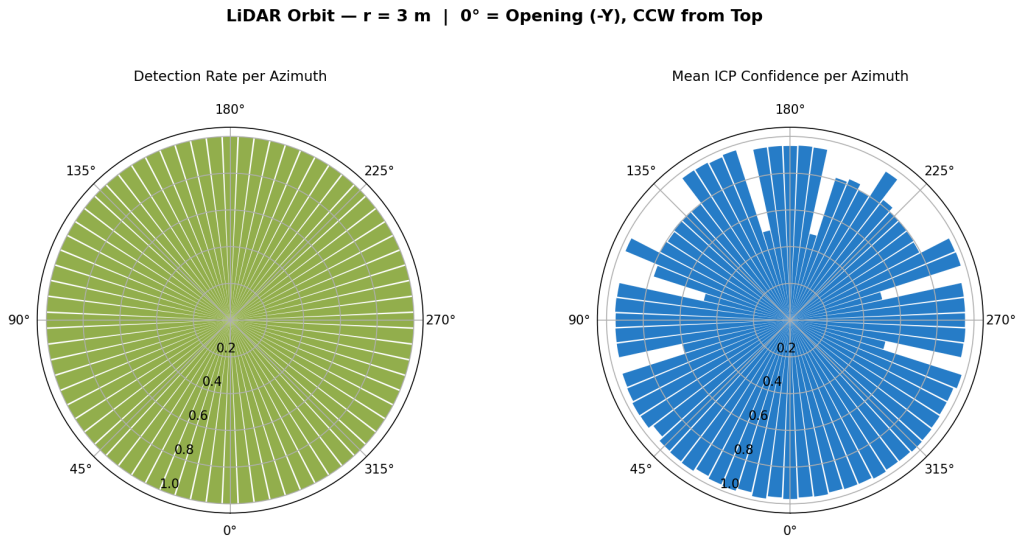


Figure 5.9.: LiDAR detection over the 3 m azimuth orbit (0° = station opening, counter-clockwise). **Left:** detection rate—a complete 100 % at every azimuth. **Right:** ICP confidence—high over the well-conditioned views but collapsing to ≈ 0.5 at the azimuths where the station’s opening is not visible and its orientation is ambiguous.

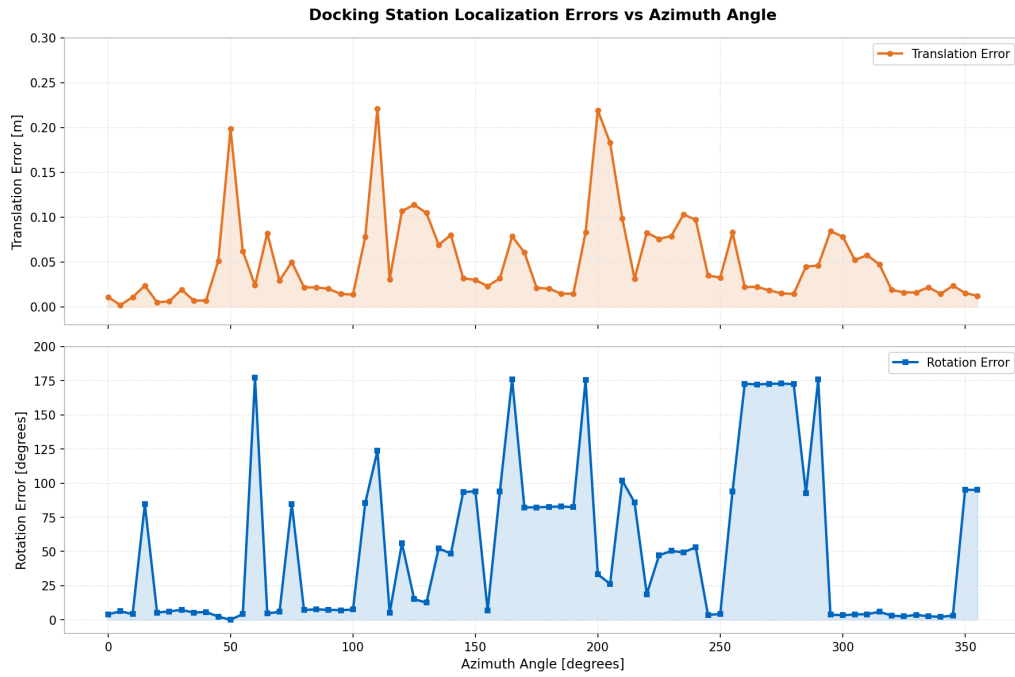


Figure 5.10.: LiDAR pose error over the 3 m orbit. **Top:** translation error—centimetre-level over most azimuths, spiking to 0.20–0.22 m only at the oblique corner views. **Bottom:** rotation error—accurate only where the station’s opening is visible, and otherwise wrong by $\approx 90^\circ$ (a wrong-wall match) or $\approx 180^\circ$ (a front/back flip); this orientation ambiguity is geometric, not a tuning artefact.

5.4.3. Anatomy of the Detection Pipeline

To understand the residual position spikes, Figure 5.11 traces the pipeline stage by stage for the worst-case translation frame—the 110° corner-on view, where the final position error reaches 0.22 m. The trace shows how a sparse, oblique partial view compounds through the filtering, clustering, and matching stages.

Sparsity and filtering. The raw scan (Figure 5.11a) is dominated by the Livox sensor’s characteristic non-repetitive ground pattern—the concentric near-square rings—within which the station return is a tiny cluster of barely twenty points: at 3 m the station is hit by only ~ 15 –21 points per scan. The fixed height-band filter (Figure 5.11b, $Z \in [-0.45, 0.25]$ m) is meant to remove the ground, but the band is wide enough that a ring of ground returns survives into the next stage—the filter cannot cleanly separate the sparse station from the sensor’s own ground footprint on geometry alone.

Clustering contamination. After projection onto a 2D occupancy grid and connected-component labelling, the dominant cluster is the L-shaped pair of walls actually visible from this corner (Figure 5.11c). But the surviving ground returns form several small stray clusters that the merge-and-plausibility stage fails to discard (Figure 5.11d): they are carried forward as part of the candidate.

Biased PCA box and ambiguous matching. The oriented bounding box is fitted by PCA to the candidate points (Figure 5.11e), and because the stray clusters lie well off the L, they drag the box corner outward and inflate it to a shape that matches neither the true 1.25×1.25 m footprint nor its orientation. The centroid-seeded ICP refinement then registers the reference model to this biased, partial candidate, but on only ~ 20 points seen from one oblique corner it converges at a reduced confidence of ~ 0.78 to a pose offset by 0.22 m (Figure 5.11f)—here the lowered confidence at least flags the frame as suspect. None of the three stages is individually catastrophic, but their errors compound: a sparse partial view, contaminated by un-rejected ground clutter, fitted by a global descriptor that the clutter biases, registered from too few points to fully recover.

5.4.4. Two Terminal Failure Modes

The orbit produces two qualitatively different failures, shown side by side in Figure 5.12. The first is the *position* failure already dissected—a contaminated, oblique partial view yielding a 0.22 m offset at a reduced confidence of 0.78 (Figure 5.12a, the 110° view); here the lowered confidence at least flags the estimate as suspect. The second is more insidious: an *orientation* failure (Figure 5.12b, the 60° view) in which the position is recovered well and the ICP confidence is high (0.97), yet the estimated yaw is wrong by close to 180° . Its cause is structural and physical, not numerical. The station’s walls are nearly indistinguishable to the LiDAR: the only feature that breaks the symmetry is the opening of the charging bay, and it is visible from only a limited arc. From every other direction the sensor sees a single flat wall,

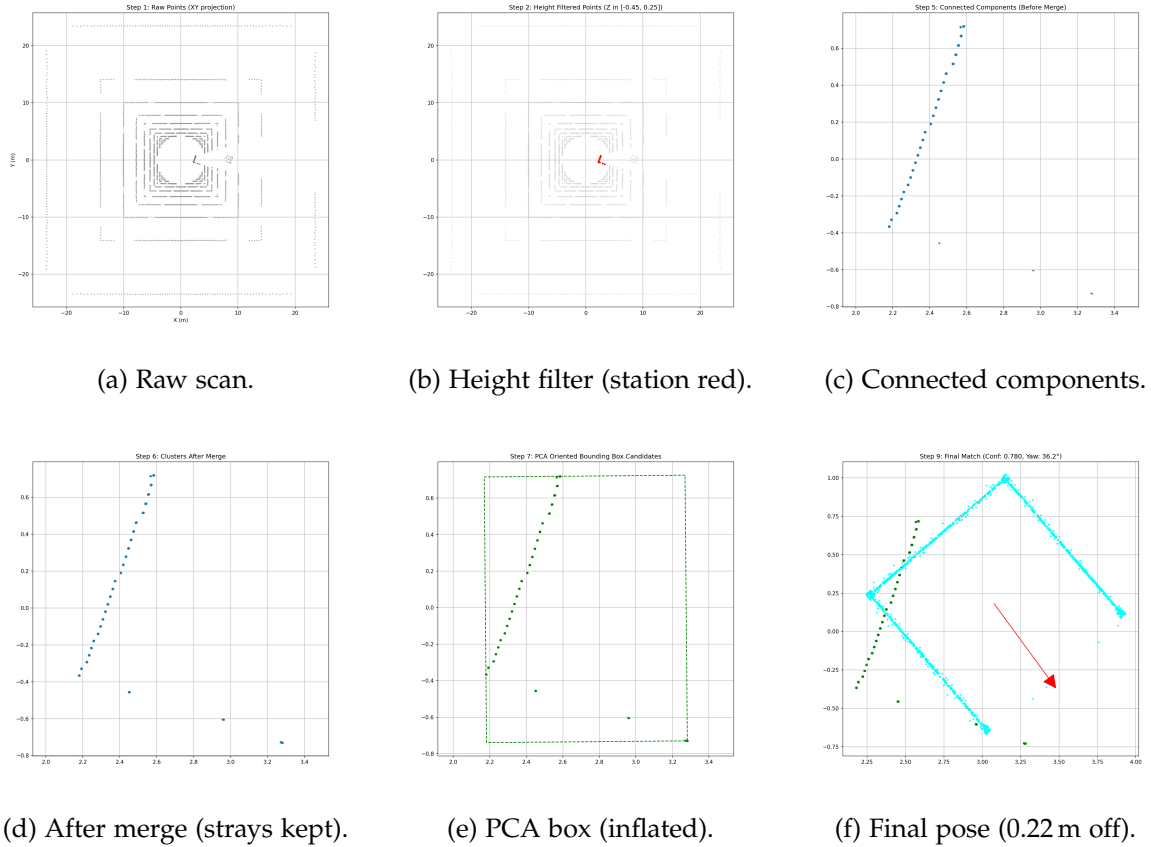
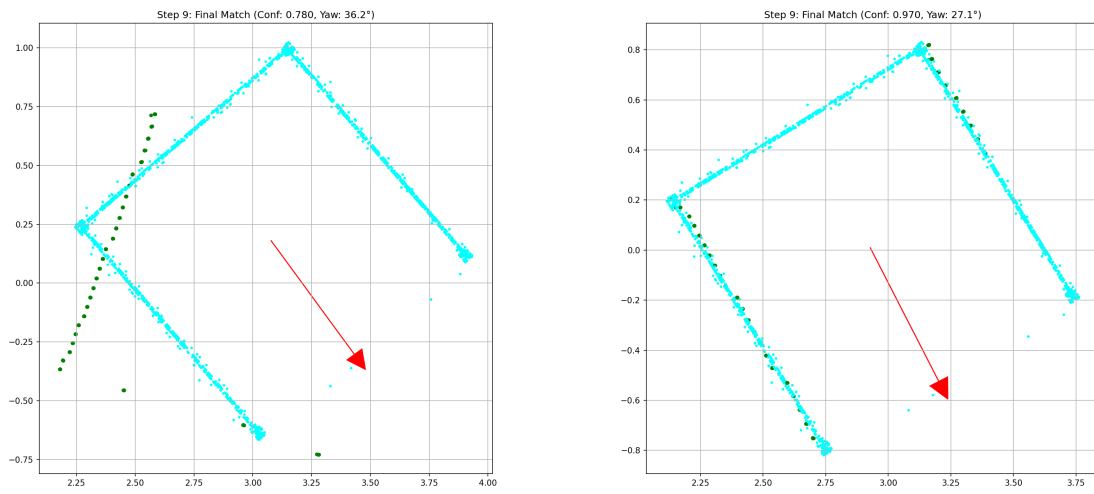


Figure 5.11.: The LiDAR detection pipeline traced through the worst-case translation frame (the 110° corner-on view). The station is hit by only ~ 20 points (a); a wide height filter lets a ground ring survive (b); connected-component clustering recovers the L-shaped partial view but also several stray ground clusters (c) that the merge stage does not reject (d); these bias the PCA bounding box outward (e); and the final pose is offset by 0.22 m (f). Cyan = reference station model, green = observed cluster, red arrow = estimated orientation.

or an L-shaped pair of walls, that is geometrically consistent with several different station orientations—the observed points fit the reference model equally well whether it faces the true way or is rotated onto another, identical-looking wall. The registration then locks, with full geometric support and therefore high confidence, onto whichever of these indistinguishable poses it happens to be seeded near: a $\approx 90^\circ$ wrong-wall match or a $\approx 180^\circ$ front/back flip. A *high* confidence on a *wrong* orientation is the dangerous case for a docking controller, because nothing internal to the LiDAR estimate signals that it should be distrusted—and no refinement of the matching can remove it, because when two poses produce the same point cloud a single scan cannot tell them apart.



(a) Worst position error (110°): 0.22 m offset from a stray-contaminated, partial L-view. (b) Worst orientation error (60°): position recovered and confidence high (0.97), but yaw wrong by $\sim 180^\circ$.

Figure 5.12.: The two terminal failure modes of the LiDAR pipeline. (a) A contaminated oblique partial view biases the position (0.22 m), but at a lowered confidence that flags it. (b) The station’s walls are indistinguishable without a view of the opening, so a partial view admits several orientations; the registration locks onto a flipped one while reporting high confidence—the more dangerous failure for a docking controller. Cyan = reference station model, green = observed points, red arrow = estimated opening direction.

5.4.5. Implications for the LiDAR Detector

These results do not condemn LiDAR for docking; they delimit what it can and cannot supply. Its strengths are real and complementary to the camera: 100 % illumination-independent detection at low latency out to 3 m, and centimetre-level *position* over most of the orbit—exactly

what is wanted for robust mid-range capture and approach. The centroid-seeded template-to-scan refinement evaluated here delivers that position reliably and lets the confidence flag most of the bad frames, a clear improvement over the earlier four-yaw search. The one thing it cannot provide is reliable terminal *orientation*, and that limitation is geometric, not algorithmic: from most viewpoints a single LiDAR scan of the near-symmetric station is consistent with several orientations, and no better front end can resolve an ambiguity that is simply absent from the data. Orientation must therefore come from elsewhere—accumulated over the approach as the opening rotates into view, or, more directly, from the camera/AprilTag at close range, which measures pose unambiguously from the asymmetric fiducial. This is precisely the division of labour formalised by the range-layered strategy of Section 5.8: LiDAR for illumination-independent detection and position, camera for terminal orientation and precision.

5.5. Closed-Loop End-to-End Docking Performance

The static studies above isolate each detector one axis at a time; they do not show whether the assembled stack—perception, the EKF that fuses it, the behaviour tree of Section 4.5.6, and the terminal controller—can actually carry the robot from a field starting pose into the docking tolerances. This section closes that gap with a closed-loop campaign run in the simulation testbed, where exact per-frame ground truth makes the terminal alignment error directly measurable and the runs are bit-for-bit repeatable. It is reported as a demonstration of the maneuver *dynamics* anticipated at the start of this chapter, not as a statistical field success-rate study.

Campaign and pipeline. Ten runs were executed from a single fixed random seed, each spawning the robot at a uniformly random pose in the 3–10 m ring around the station centre (5,2) with a random initial heading. The station opening faces world $-X$, so a *docked* pose requires the robot to reach (5,2) at heading 0, driving straight into the opening. Each run is governed by the three-stage behaviour tree of Section 4.5.6:

1. **Global approach** (code: `gps approach`). Simulated GPS (Gaussian, $\sigma = 1$ m) and compass ($\sigma = 10^\circ$) drive the robot into a docking zone a few metres in front of the $-X$ opening.
2. **Local alignment** (code: `vision align`). The SIM+REAL+WEATHER YOLOv8-pose detector of Section 5.3 centres the station bearing so the robot faces the opening.
3. **Terminal dock** (code: `precise dock`). The controller closes on (5,2) at heading 0 using a global pose estimate produced by an EKF. Throughout the approach the EKF fuses wheel odometry, the compass, the LiDAR station detector (confidence-gated at ≥ 0.7), and the GPS as a loose anchor—deliberately down-weighted with an inflated covariance so it bounds gross drift without capping terminal accuracy; within the final 1.5 m the AprilTag pose is added once the tag enters its range gate.

The terminal stage therefore runs on *fused perception*, not on ground truth: the accuracy reported below is that of the full EKF-fused estimator and controller under the simulated sensor noise, and it inherits the per-detector range and orientation envelopes characterised earlier in this chapter.

Table 5.3 aggregates the campaign. The robot reaches the station in 9 of the 10 runs, and every one of those nine settles inside the ≤ 5 cm / 5° terminal tolerance of Section 3.1.1 (mean 3.5 cm / 2.0°). Figure 5.13 shows the ten ground-truth trajectories converging onto the $-X$ opening, and Figure 5.14 the distribution of the terminal errors over the docked runs.

Table 5.3.: Closed-loop end-to-end docking performance in the simulation testbed: ten runs from random spawns in the 3–10 m ring, each driven by the full GPS $\rightarrow$ learned-vision $\rightarrow$ precise-dock behaviour tree with an EKF-fused terminal pose. Success rates are over all ten runs; the accuracy and timing statistics are over the nine docked runs.

Metric	Value	Notes
<i>Success ($N = 10$)</i>		
Reached station (docked)	90 % (9/10)	
≤ 1 cm & 5°	0 %	EKF/controller settling floor
≤ 5 cm & 5°	90 % (9/10)	target terminal tolerance
≤ 10 cm & 10°	90 % (9/10)	
<i>Terminal accuracy (9 docked runs), mean $\pm$ std (median)</i>		
Translational error	3.5 $\pm$ 0.8 cm (3.6)	min 2.5, max 4.9 cm
Rotational error	2.0 $\pm$ 1.2 $^\circ$ (2.7)	min 0.5, max 3.6 $^\circ$
<i>Timing [s], mean $\pm$ std (median)</i>		
Total docking time	90.2 $\pm$ 17.0 (87.0)	min 66, max 115
Stage 1 (GPS approach)	31.0 $\pm$ 12.6 (31.9)	scales with spawn range
Stage 2 (vision align)	9.1 $\pm$ 2.0 (9.1)	
Stage 3 (precise dock)	52.2 $\pm$ 10.0 (49.7)	dominant cost

5.5.1. Analysis of Performance and Failure Mechanisms

The campaign confirms in closed loop the division of labour that the static envelopes implied, and its single failure and its accuracy floor are both explained directly by those envelopes.

Success and the single failure. Nine of the ten runs dock. The one failure (Figure 5.13, red path) is the longest-range spawn at 9.86 m: it reached the docking zone under GPS guidance but did not converge onto the station inside the run timeout, halting roughly 3 m short at the zone boundary. The next-longest spawn (9.7 m) docked successfully, so this is a marginal terminal-convergence case at the edge of the sampled range rather than a hard range limit of the pipeline.

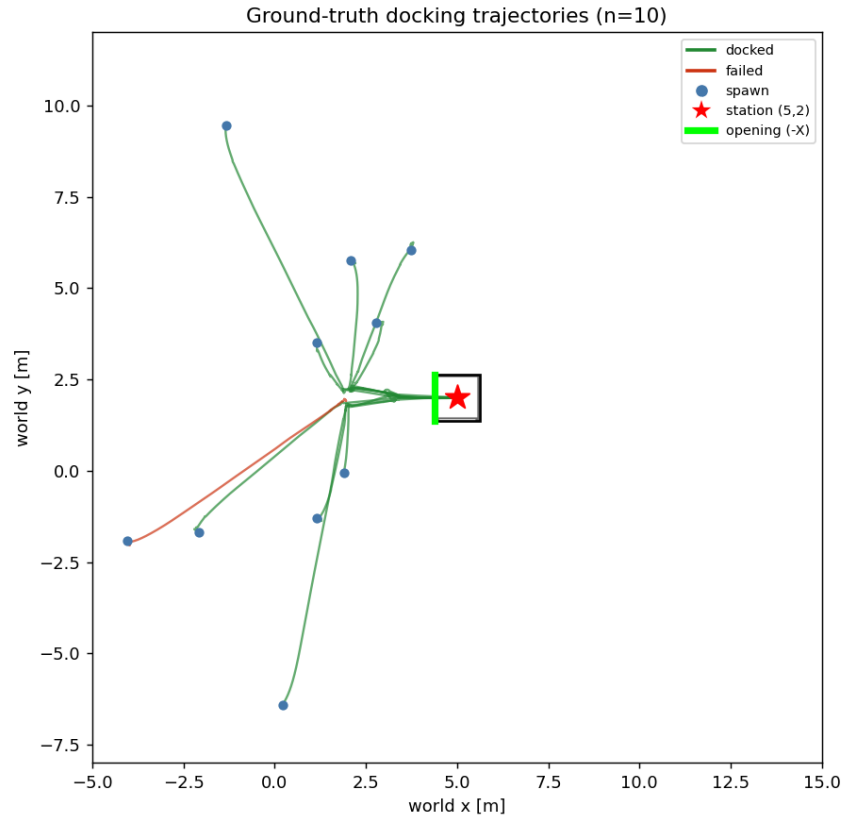
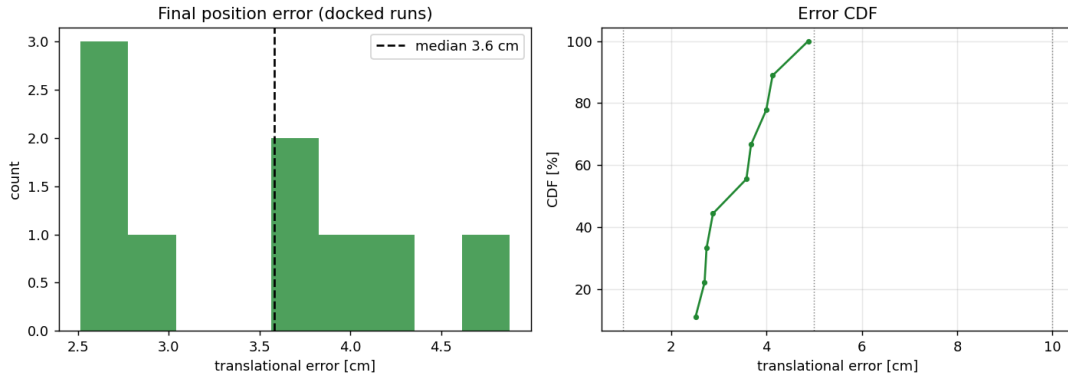
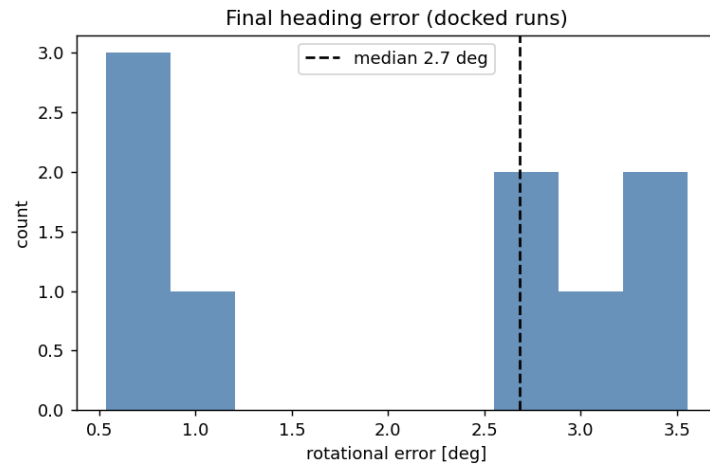


Figure 5.13.: Ground-truth trajectories of the ten closed-loop docking runs, from random spawns (blue dots) in the 3–10 m ring to the station at (5,2) (red star) whose opening faces $-X$ (green bar). Nine runs dock (green paths); the single failure (red) is the longest-range spawn, which reaches the docking-zone boundary but does not converge onto the station within the timeout.



(a) Translational error: distribution (left) and CDF (right).



(b) Rotational error.

Figure 5.14.: Terminal error over the nine docked runs. **(a)** translational error (median 3.6 cm): every docked run falls between 2.5 and 4.9 cm, inside the 5 cm tolerance but above the 1 cm mark. **(b)** rotational error (median 2.7°), bimodal between a sub- 1.2° cluster and a 2.7 – 3.6° cluster.

Terminal accuracy and its floor. Across the nine docked runs the translational error is 3.5 ± 0.8 cm (median 3.6, range 2.5–4.9 cm) and the rotational error $2.0 \pm 1.2^\circ$ (median 2.7, range 0.5–3.6°); all nine clear the ≤ 5 cm / 5° tolerance, but none reaches the tighter ≤ 1 cm mark. This few-centimetre floor is the expected signature of a terminal pose drawn from a fused, noisy estimator rather than from a direct sub-pixel AprilTag read. It is consistent with the static finding (Section 5.1.1) that only the AprilTag delivers sub-centimetre precision, and it quantifies the sharpness the fusion trades away for robustness. The rotational-error histogram in Figure 5.14 is visibly bimodal—a tight cluster below 1.2° and a second near 2.7–3.6°—reflecting which corrective source dominated the final heading estimate in each run.

Timing. The precise-dock stage dominates the time budget at 52.2 ± 10.0 s, followed by the GPS approach at 31.0 ± 12.6 s (which scales with spawn range) and the brief learned-vision alignment at 9.1 ± 2.0 s, for a total of 90.2 ± 17.0 s per run. The terminal stage is the most expensive because it must null the residual of the fused estimate down to the mechanical tolerance, exactly the regime where the centimetre-level corrections accrue slowly.

Scope of the result. This campaign is a maneuver-dynamics demonstration ($N = 10$, clear weather), not the multi-configuration weather success-rate study that a full field characterisation—placed out of scope at the start of this chapter—would demand. Its value is to confirm in closed loop what the static envelopes imply: the learned detector carries the long-range capture and alignment, and the EKF-fused LiDAR-plus-AprilTag terminal stage brings the robot into centimetre-level docking contact, the same division of labour formalised by the range-layered strategy of Section 5.8. Extending the campaign across the weather axis, and to a perception-only terminal stage that drops the GPS prior, is left to future work.

5.6. Real-World Validation: The Sim-to-Real Counter-Check

Every result so far rests on the controlled simulation testbed, whose exact ground truth and bit-for-bit repeatability are what make the single-variable comparisons above possible (Section 4.5). That same artificiality is its weakness: a detector that excels on rendered frames need not transfer to a real camera and a real LiDAR. This section is the reality counter-check. The learned vision detector and the LiDAR pipeline—the two detectors whose behaviour the previous sections derived in simulation—are re-evaluated on physical captures of the real station, on the same max-distance and azimuth-orbit sweeps, to test whether each simulation finding survives the transfer. A concluding outdoor experiment (Section 5.6.3) then measures the localisation accuracy of all three on-board sensors together against an RTK-GPS ground truth. The decisive point for the learned detector is that the network used here is the `SIM+REAL+WEATHER` model of Section 5.3, trained predominantly on synthetic frames; reproducing its simulated envelope on real imagery is therefore also a direct validation of the simulation-first training strategy.

5.6.1. Learned Vision on Real Captures

Figure 5.15 consolidates the real-world evaluation of the learned detector across all three axes. Each of the three simulation findings reappears. *Range*: on the straight-on approach the detector holds a 100 % detection rate out to 8 m—the full extent over which the real station was reliably captured—at a mean confidence that stays above 0.79 throughout, before degrading beyond 8 m as the structure subtends too few pixels. This reproduces the simulated long-range behaviour: the learned detector keeps the station in view well past the fiducial’s marker-legibility range, and does so on real pixels. *Azimuth*: the 3 m orbit returns an unbroken 100 % detection across all ten captured azimuths (mean confidence 0.76–0.97), exactly the gap-free angular coverage that distinguished the learned detector from the AprilTag’s inter-tag drop-outs in Figure 5.2. *Weather*: re-running the 3DGS rain, snow and fog injection on the *real* reference photo reproduces the graceful-degradation signature of Figure 5.6—100 % detection through rain to 200 mm/h (confidence $0.95 \rightarrow 0.84$), through all snow rates, and through fog down to 10 m visibility, failing only at the extreme 5 m case. That a network trained almost entirely on synthetic data behaves on real captures as it did in simulation is the empirical close of the sim-to-real loop opened by the training-data ablation: it confirms that the free, exactly-labelled synthetic corpus does transfer, and that the simulation envelope is a faithful predictor of field behaviour rather than an optimistic artefact.

5.6.2. LiDAR on Real Point Clouds

The real Livox Mid-360 captures reproduce the LiDAR pipeline’s two simulated findings just as cleanly: robust detection at short range, but unreliable terminal *pose*. Figure 5.16 traces the straight-on distance sweep. The clustering-PCA-ICP pipeline locks onto the station from 1 to 4 m at high ICP confidence (0.84–0.97), but at 5 m it returns no detection at all—the real sensor’s point return on the small station has thinned below the threshold the clustering stage needs, the physical manifestation of the sparsity limit that capped the simulated pipeline’s usable range to the 3 m band characterised in Section 5.4.1. Figure 5.17 shows the 2 m orbit, and here the simulated failure mode resurfaces unmistakably. The station is detected at every one of the six captured azimuths, yet the estimated yaw scatters wildly across them (-80° , -127° , -15° , -160° , $+140^\circ$, $+120^\circ$) while ICP confidence stays high (0.84–0.98)—the same high-confidence orientation ambiguity, born of the station’s indistinguishable walls under a partial view, that Figure 5.12 dissected in simulation. On real data as in simulation, the LiDAR pipeline is therefore a competent short-range *detector* and position source but cannot be trusted for terminal *orientation*, confirming the failure analysis of Section 5.4 on physical measurements.

5.6.3. Outdoor Static Multi-Sensor Accuracy Sweep

The detector re-runs above confirm the simulated *detection* behaviour on real data; a final outdoor experiment closes the remaining gap by measuring the *localisation accuracy* of all three on-board sensors simultaneously against an absolute reference. Following the static-sweep

5. Evaluation and Results

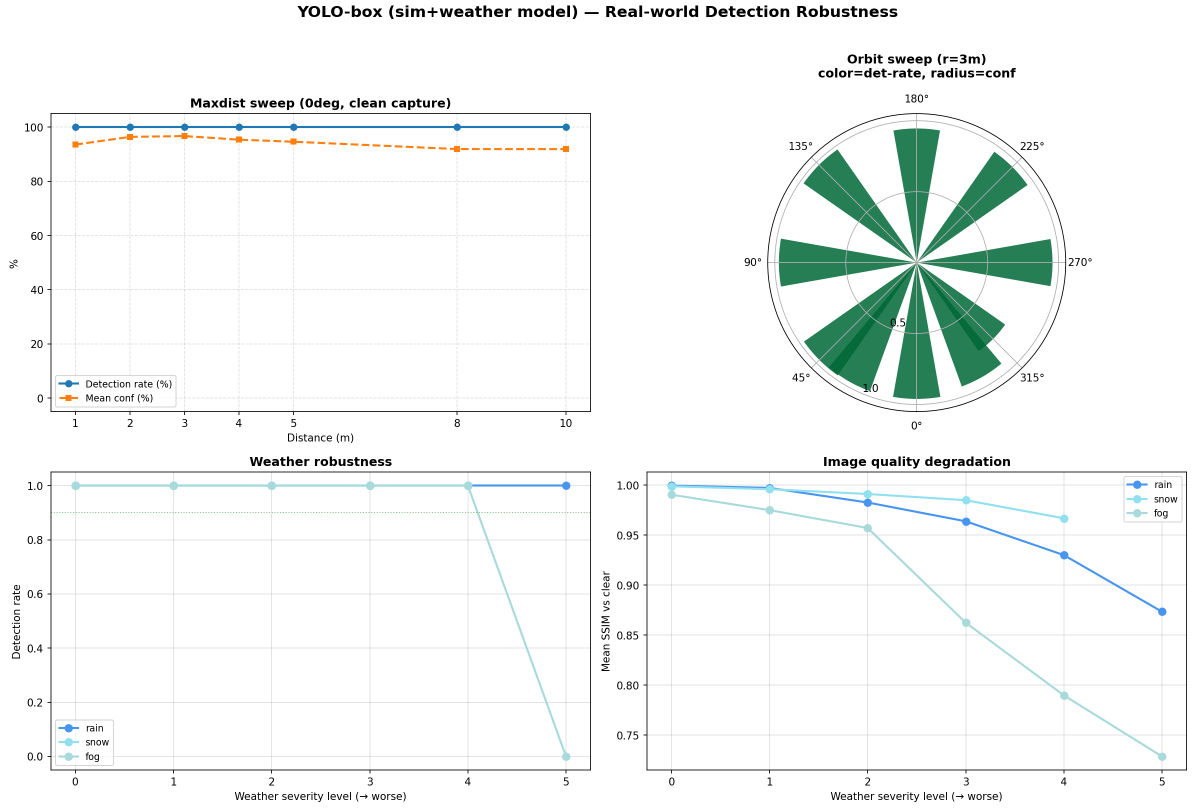


Figure 5.15.: Real-world evaluation of the SIM+REAL+WEATHER learned detector. **Top-left:** max-distance sweep—100 % detection to 8 m at high confidence. **Top-right:** 3 m azimuth orbit—full 360° coverage (bar length = detection rate, shade = confidence). **Bottom-left:** weather robustness on the real reference—near-perfect detection until the most extreme fog. **Bottom-right:** image-quality degradation (Structural Similarity Measure (SSIM) vs. clear) driving each weather axis. Every trend matches the simulation results of Sections 5.1–5.2.

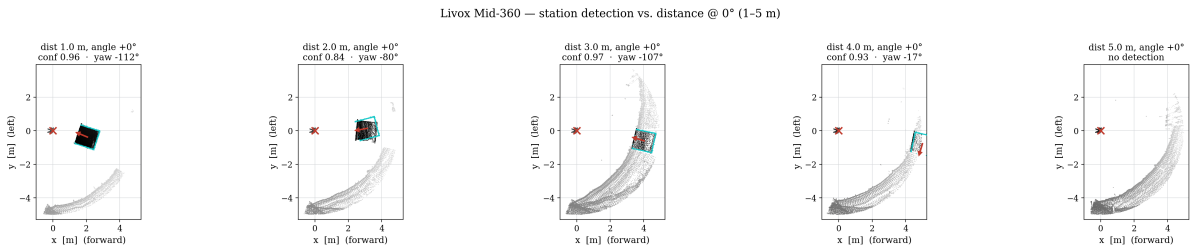


Figure 5.16.: Real Livox Mid-360 station detection on the straight-on approach (0°, 1–5 m). The matched reference template (turquoise) with its red opening arrow is locked onto the real point cloud (grey, height-shaded) from 1 to 4 m at high ICP confidence, but the return thins below the clustering threshold by 5 m, where detection fails—the physical counterpart of the simulated sparsity limit.

Livox Mid-360 — station detection orbit @ 2 m (angles 0/45/90/180/225/270°)

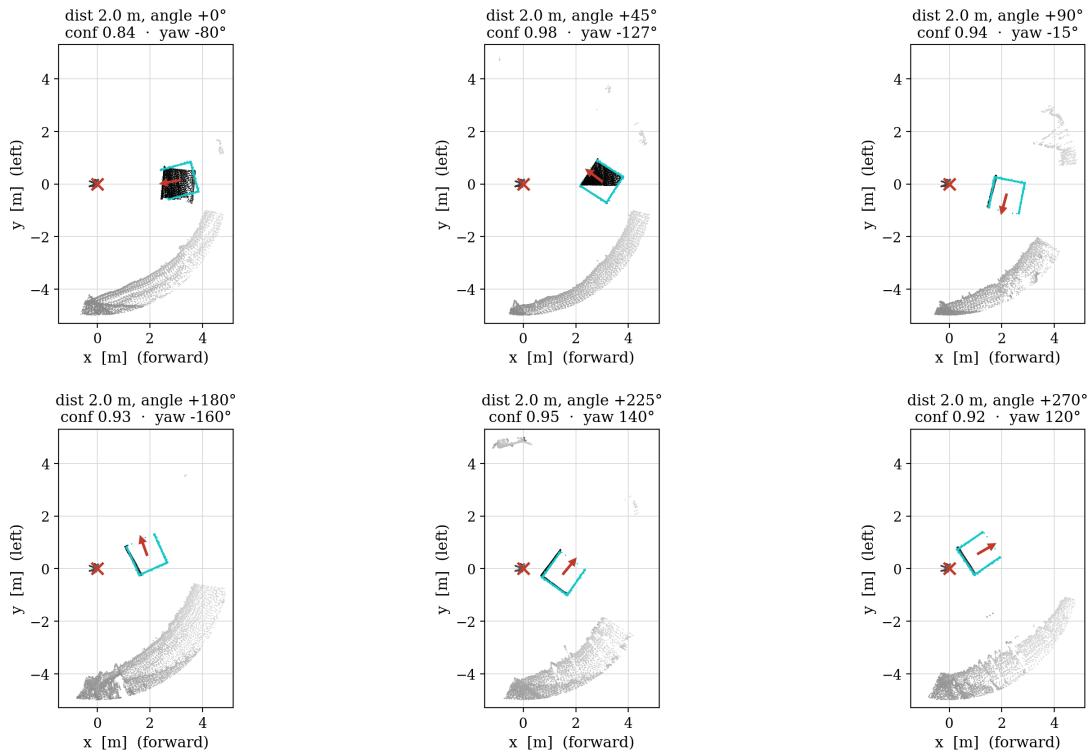


Figure 5.17.: Real Livox Mid-360 station detection over the 2m orbit (azimuths 0/45/90/180/225/270°). The station is detected at every azimuth at high ICP confidence, but the reported yaw (per-panel title) flips and scatters across views—the real-data signature of the same wall-ambiguity orientation problem identified in simulation (Figure 5.12).

protocol of Section 4.4.3, the robot was placed at the 36 positions of the polar grid (6 angles $\times$ 6 distances) around the physical station, and 10 s of AprilTag, LiDAR and RTK-GPS data were logged at each. With the GPS fix (RTK-fixed mode) taken as ground truth, every camera and LiDAR estimate is projected into the common polar setup frame and compared against it; Figure 5.18 shows the resulting per-position estimates with their 95 % covariance ellipses.

Two findings transfer directly from simulation to the field. First, the *detection envelopes* match: the camera returned an AprilTag pose only out to ~ 5 m and the LiDAR pipeline only out to ~ 3 m, beyond which neither resolved the station and only the GPS reference remained—the same short fiducial range (Section 5.1.1) and ~ 3 m LiDAR ceiling (Section 5.4.1) found in simulation, now confirmed on real hardware. Second, within those ranges the per-position covariance clouds are tight and centred near the GPS ground truth, with the camera clouds smaller than the LiDAR clouds at matched range, consistent with the camera’s higher terminal precision. A per-row rigid-plus-scale calibration was applied to register each sensor’s mounting frame to the setup grid; the calibration constants and residuals are documented with the analysis pipeline in the repository, and the modest residuals (decimetre-level at the widest angles) bound the absolute accuracy that can be claimed from a hand-laid outdoor grid. The sweep is therefore best read as a real-world confirmation of the range envelopes and the relative sensor noise, complementing the exact, single-variable accuracy figures that only the simulation can provide.

5.7. Computational Cost on the Edge Platform

The axes evaluated so far—range, angle, weather, training data—all ask *whether and how well* each detector sees the station. A docking robot must also be able to *afford* to run the detector: the perception stack executes on the single NVIDIA Jetson Orin NX companion computer of the test platform (subsection 4.4.2), which it shares with localisation, the ROS 2 middleware and the control loop. A detector that localises perfectly but cannot keep up with the camera frame rate, or that monopolises the processor needed by the rest of the stack, is unusable regardless of its accuracy. This section therefore characterises the *computational* cost of the three detectors and—more usefully for deployment—how that cost responds to the parameters one can actually tune.

The measurements are *isolated micro-benchmarks*: each detector is timed on pre-loaded in-memory inputs in a tight loop, with no ROS 2 transport, no simulation and no disk I/O inside the timed region, after a warm-up phase and over 40–60 repetitions with a 95 % bootstrap confidence interval on the mean.¹ Isolating the algorithmic cost in this way removes scheduling and transport noise and yields the *lower bound* on the per-frame cost; the end-to-end node latency is higher, but it is this isolated cost that scales with the parameters studied here. One caveat governs the absolute numbers: the benchmark host is a laptop-class machine (AMD Ryzen 7 8845HS, 8 C/16 T, with an RTX 3050 6 GB GPU), not the Jetson Orin NX of the

¹Full protocol, per-call raw timings and provenance are archived with the benchmark harness; the LiDAR core is a parity-checked, ROS 2-free extraction of the deployed node and the AprilTag library is the same C detector (family 36h11) wrapped by the ROS 2 node, so the tunables map one-to-one onto the deployed parameters.

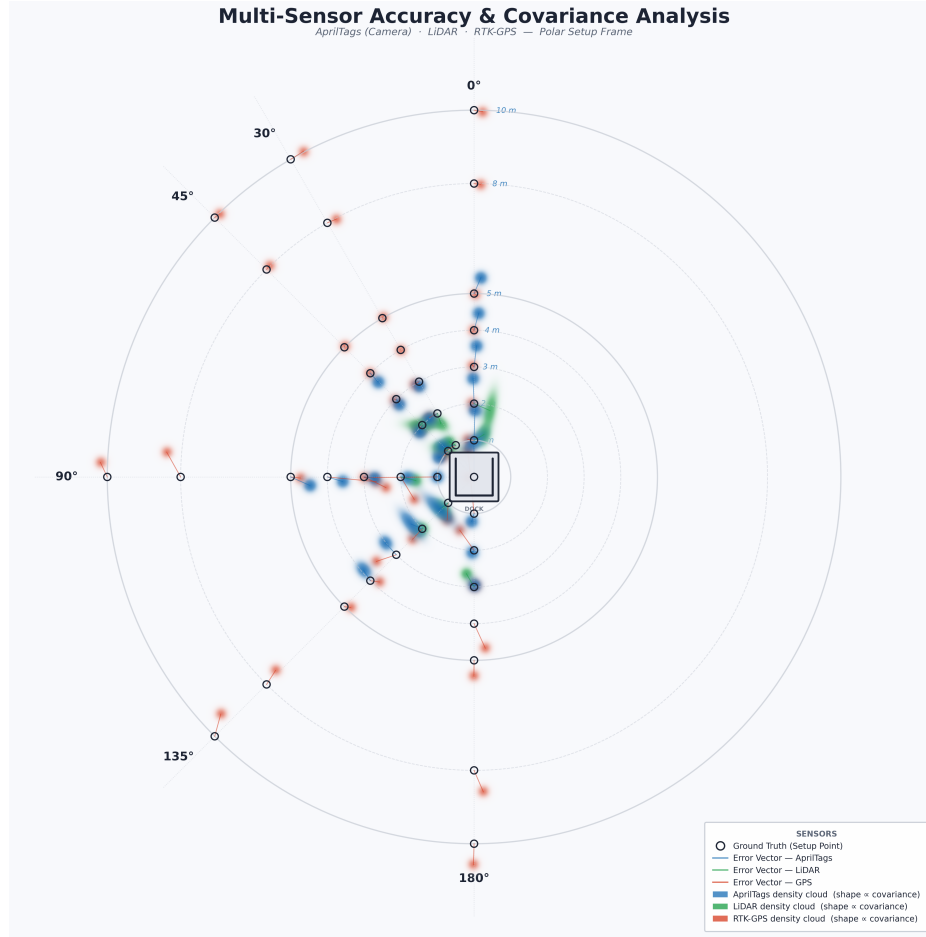


Figure 5.18.: Outdoor static accuracy sweep over the 36-position polar grid, with RTK-GPS (treated as ground truth) at the grid points. Each sensor’s estimate is drawn as a density cloud with its 95 % covariance ellipse and an error vector to the GPS reference. The camera (AprilTag) returns poses to ~ 5 m and the LiDAR to ~ 3 m—the same envelopes found in simulation—beyond which only GPS remains.

deployed platform. The Jetson is slower in absolute terms, so the millisecond figures below should be read as representative rather than as the on-robot latency; the *scaling trends and cross-method ratios*—which parameter buys the most speed, and how each method grows with its inputs—are the transferable results and the reason the study is reported at all.

5.7.1. Baseline Cost of the Deployed Configurations

Table 5.4 and Figure 5.19 report each detector at the configuration in which it is actually deployed. The headline result is that all four run comfortably in real time—every configuration clears 50 FPS, well above the 30 FPS camera and 10 Hz LiDAR rates the stack must sustain—so compute does *not* disqualify any detector on this hardware, and the range-layering argument of section 5.8 is not forced by a latency ceiling. The cost *structure*, however, differs sharply and dictates how each detector should be configured. The AprilTag detector is the cheapest single-thread CPU option at 6.8 ms on a full 1280 px frame, using barely one core and 144 MiB. The learned detector is the most expensive on the CPU (19.3 ms) and, tellingly, saturates all eight cores to get there—it is the one detector that contends directly with the rest of the stack for the processor. Moving it onto the GPU collapses its latency to 5.4 ms (the fastest of all four) while *freeing seven CPU cores* and costing only 24 MiB of GPU memory; this single change is the strongest argument for the GPU-accelerated deployment already adopted on the physical robot (subsection 4.4.2). The LiDAR pipeline sits in between at 13.4 ms on a single core and the smallest footprint of all (72 MiB), confirming that the lightweight clustering-PCA-ICP front end is cheap enough for the mid-range duty assigned to it—its limitation is accuracy (section 5.4), not cost.

Table 5.4.: Isolated compute cost of the three detectors at their deployed configurations (benchmark host: AMD Ryzen 7 8845HS + RTX 3050 6 GB; absolute values are host-specific). Latency is the mean per-frame inference time; CPU load is the mean number of busy cores; peak memory is process RSS, or CUDA memory for the GPU configuration. All four clear real-time at > 50 FPS.

Detector (configuration)	Latency	Throughput	CPU load	Peak mem.
AprilTag (1280 px, CPU, 1 thr)	6.8 ms	148 FPS	1.0 cores	144 MiB
YOLO box (640 px, CPU)	19.3 ms	52 FPS	8.1 cores	805 MiB
YOLO box (640 px, GPU)	5.4 ms	186 FPS	1.1 cores	24 MiB (CUDA)
LiDAR ICP (~23 k pts, CPU)	13.4 ms	74 FPS	1.1 cores	72 MiB

5.7.2. What Drives the Cost

The baseline is only half the story; the deployable insight is which knob moves the cost. One-factor-at-a-time sweeps from each baseline isolate the dominant levers.

Camera resolution dominates the vision methods. Figure 5.20 shows both camera detectors scaling steeply with input resolution. The AprilTag grows roughly quadratically with image

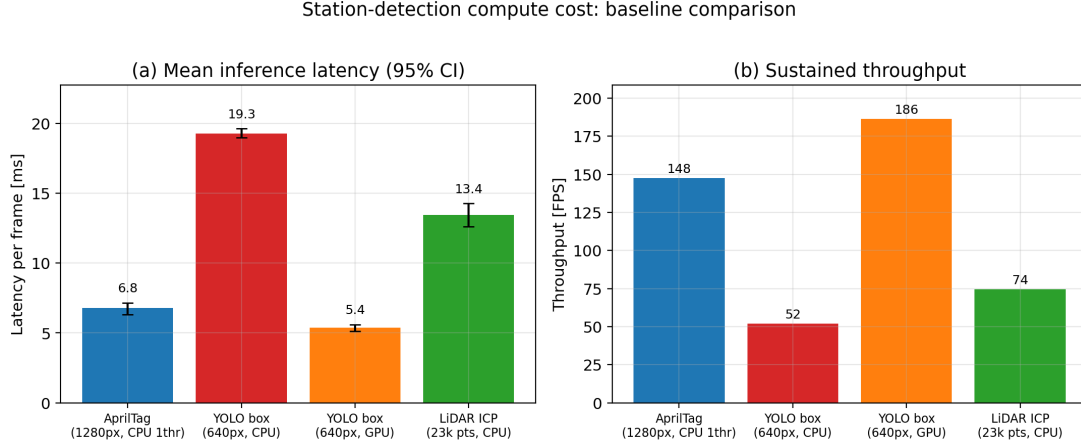


Figure 5.19.: Isolated per-frame compute cost of the three detectors at their deployed configurations. **Left:** mean inference latency (95 % confidence interval). **Right:** sustained throughput. AprilTag is the cheapest single-thread CPU option; the learned detector is the most expensive on the CPU but the fastest of all once moved onto the GPU; the LiDAR pipeline is inexpensive on a single core.

width—0.5 ms at 320 px, 6.8 ms at 1280 px, 14 ms at 1920 px—while YOLO on the CPU runs from 8.5 ms at `imgsz 320` to 57 ms at 1280, crossing out of real time above ~ 960 px. On the GPU the same network stays in the 4–14 ms band across the entire range, which is precisely why full-resolution outdoor operation is only affordable with GPU offload. Resolution is thus the first cost lever for both, and the tension is direct: the long-range detection that the learned detector wins (subsection 5.1.1) and the marker legibility the AprilTag needs (section 5.1) both demand pixels, which cost compute.

The accuracy/speed dials are cheap; GPU offload is decisive. For the AprilTag, `quad_decimate` is the cheap speed-for-range trade: it cuts latency from 22.3 ms at 1.0 to 6.6 ms at the deployed 2.0 to 2.0 ms at 4.0, at the cost of dropping the smallest (most distant) tags, and thread count scales it further ($6.6 \rightarrow 2.4$ ms over $1 \rightarrow 8$ threads). Crucially, the remaining AprilTag knobs—`decode_sharpening`, `quad_sigma`, `refine_edges`—have *negligible* compute cost: they are accuracy dials, not speed dials, and can be tuned for robustness without a latency penalty. For the learned detector, the device is the single biggest lever: the $3.6\times$ CPU→GPU speed-up already noted, with fp16 precision giving *no* further gain on this small model (it is launch-overhead bound, not arithmetic bound).

LiDAR cost is set by ICP, not by point count. The LiDAR cost is dominated by the registration stage, not the raw cloud size: the cell-assignment loop is capped, so latency is flat up to ~ 23 k points and is instead governed by `icp_max_points` (5.5 ms at 100 to 23 ms at 1000) and `icp_iters` (which plateaus after ~ 25 iterations as the registration converges early). The most consequential implementation choice is the nearest-neighbour backend: a `cKDTree`

search rather than a brute-force one is a $4.6\times$ speed-up ($64 \rightarrow 14$ ms), which is what brings the pipeline into its real-time budget in the first place.

The compute study therefore reinforces, rather than complicates, the perception strategy. No detector is priced out on this hardware, so accuracy and robustness remain the deciding criteria; but the cost structure prescribes *how* each is run—the learned detector on the GPU (where it is both fastest and frees the CPU), the AprilTag with `quad_decimate` traded against the range it must cover, and the LiDAR pipeline with a KD-tree backend and a bounded ICP. These are exactly the configurations carried into the deployment of subsection 4.4.2.

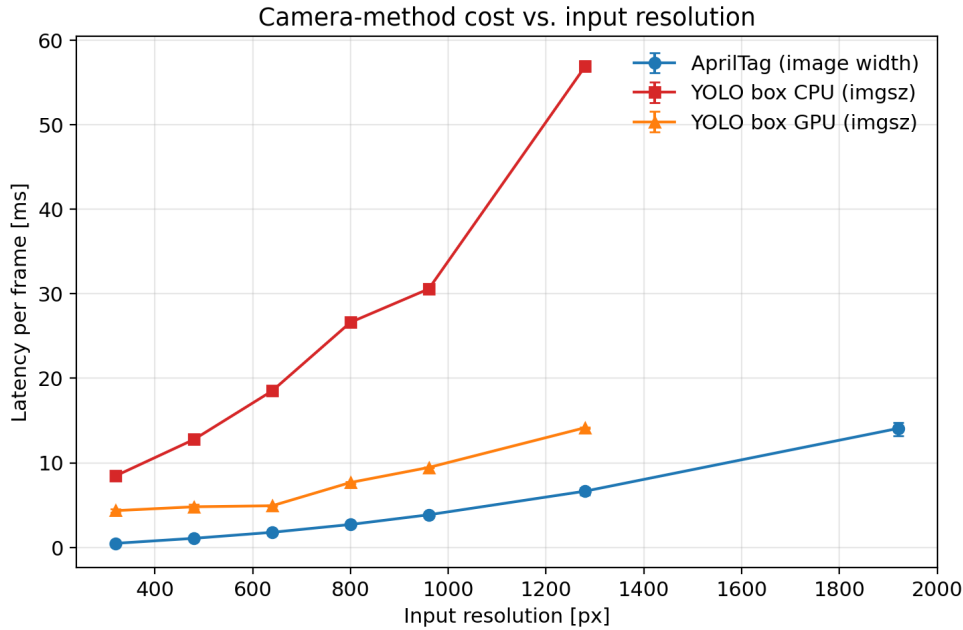


Figure 5.20.: Per-frame latency of the camera detectors versus input resolution. The AprilTag (blue) scales roughly quadratically with image width; YOLO on the CPU (red) grows steeply and leaves the real-time budget above ~ 960 px, whereas the same network on the GPU (orange) stays in a 4–14 ms band across the full range. Higher resolution buys the detection range of subsection 5.1.1 but costs compute, which GPU offload absorbs.

5.8. Synthesis: A Range-Layered Docking Perception Strategy

Taken together, the four simulation studies—now corroborated on real hardware by the counter-check of Section 5.6—answer RQ2 not by crowning a single detector but by assigning each to the range band where it dominates. No detector is good everywhere, and the failure modes are cleanly separated by range: the learned detector loses precision and leaves the frame at close range; the AprilTag is range-limited (reliable only to ~ 4.5 m, extendable in simulation but still only ~ 5 m in the outdoor test), drops out in the inter-tag sectors, and

is brittle under weather; the LiDAR pipeline detects everywhere and gives good position but cannot resolve terminal orientation from a single partial view. Their envelopes are complementary rather than overlapping, which is exactly the condition under which a layered strategy outperforms any single sensor.

Table 5.5 states the resulting allocation. At long range—roughly 4 to 30 m, and through rain, snow and fog—the marker-free learned detector provides reliable capture and a coarse bearing, complemented by RTK-GPS for absolute position in the open inter-row corridors; this is the regime in which neither the fiducial nor a precise geometric fit is available, and robust *detection* is what matters. At short range—inside ~ 4 m, down to the terminal coupling—the AprilTag delivers the sub-pixel, sub-centimetre precision the energy interface demands (Section 3.1.1), with the LiDAR providing an illumination-independent geometric cross-check. The hand-off between the two regimes falls around 3–4 m, which is why the LiDAR orbit was characterised at exactly 3 m. This empirically grounds the two-detector fusion proposed on architectural grounds in Section 3.4.1—LiDAR for robust intermediate-range detection, camera for high-accuracy terminal alignment—and extends it with a third, learned element that pushes the reliable *capture* range from the fiducial’s ~ 4.5 m ceiling out to 30 m and through the weather that the fiducial cannot survive.

Table 5.5.: The range-layered perception strategy supported by the evaluation. Each detector is assigned the range band in which the measurements above show it dominant, and the dominant failure that excludes it from the others.

Phase / range	Primary detectors	Why it dominates here	Why not the others
Capture & approach (~ 4 –30 m, all weather)	Learned vision + RTK-GPS	100 % detection to 30 m and through rain/snow/fog; coarse bearing suffices	AprilTag range-limited (~ 4.5 m) and weather-brittle; LiDAR pose flips on partial views
Terminal alignment ($< \sim 4$ m, clear)	AprilTag + LiDAR	Sub-pixel (0.81 px, ≈ 4.6 cm) precision; LiDAR geometric cross-check	Learned keypoints coarse (~ 8 px) and diverge as the station leaves the frame

6. Conclusion

This thesis set out to address a single precondition for long-term autonomous cultivation—autonomous resource replenishment through docking—and posed it as two coupled research questions: how to structure a control architecture for resource-aware LTA with docking embedded in it (RQ1), and how the available sensing modalities and detection algorithms compare for the docking maneuver itself (RQ2). This chapter closes both threads. Section 6.1 restates each research question and answers it from the results; Section 6.2 reflects on the limitations, tied back to the scope drawn in Section 1.2.1; and Section 6.3 sets out the directions that follow.

6.1. Summary of Contributions

The work makes two distinct contributions, one architectural and one empirical, corresponding to the two research questions.

Contribution 1 — An Architecture for Resource-Aware Long-Term Autonomy (RQ1)

RQ1 (Architecture). Which architectural principles does an outdoor field robot require for resource-aware long-term autonomy, and how can autonomous docking be embedded within such an architecture?

Chapter 2 surveyed the major autonomy-architecture paradigms along a common set of dimensions, and Chapter 3 derived from the resource-aware LTA requirements a concrete answer. The architectural principles reduce to three. First, *no single classical paradigm spans the requirement space*: resource-aware self-maintenance needs strategic planning over a persistent world model, while survival in an unstructured field needs reflexive, bounded-latency reactions, and a SPA, a purely reactive, or an end-to-end learned controller each satisfies only one end of this range. A *hybrid (deliberative–reactive) three-layer architecture*, realised on ROS 2 with a BT as the executive layer, is the construction that spans it—the ROS 2 component model directly answering the modularity, edge-deployability, and maintainability requirements, and the symbolic BT confining any learning to a bounded, optimisable sub-problem rather than the whole control loop. Second, persistent operation requires *decoupling the fast, hard-real-time control loop from the slow, compute-heavy reasoning*: the proposed system splits the SPA loop across an on-robot real-time domain and an asynchronous cloud agronomic simulation, mediated entirely through shared producer–consumer databases, so that a best-effort predictor and a hard-real-time controller cooperate without either blocking the other.

Third, and answering the second half of RQ1, *docking is not a standalone behaviour but a state the executive layer enters* whenever an energy-range prediction—continuously comparing the remaining charge against the cost of the planned return path—falls below a safety margin; docking is thereby embedded in the resource loop rather than bolted onto it. The contribution here is the conceptual architecture itself (Section 3.2) together with the realised ROS 2 docking pipeline (Section 3.4) that instantiates its Sense–Plan–Act slice on the test platform.

Contribution 2 — A Comparative Evaluation of Docking Perception (RQ2)

RQ2 (Sensing). *How do different sensor modalities and detection algorithms compare in positional accuracy, reliability, and computational efficiency during the autonomous docking of a mobile robot under varying environmental conditions?*

Chapters 4 and 5 answered this with a single-variable comparison of three detectors—a hand-crafted AprilTag fiducial pipeline, a marker-free learned-keypoint (YOLOv8-pose) pipeline, and a geometric LiDAR pipeline—on identical poses against exact simulation ground truth, corroborated on physical hardware. The central finding is that *no single detector wins, because their failure modes are cleanly separated by range*, so the answer is a division of labour rather than a winner:

- On **accuracy**, the AprilTag is one to two orders of magnitude more precise than the alternatives (0.81 px, $\approx$ 4.6 cm mean translation), but only within a short range ($\sim$ 4.5 m in simulation, confirmed at $\sim$ 5 m outdoors); the learned detector is an order of magnitude coarser ($\sim$ 8 px); and the LiDAR delivers centimetre-level *position* but cannot resolve terminal *orientation*—a geometric ambiguity of the near-symmetric station under a partial view, not an algorithmic defect, and the dangerous case because a wrong yaw is reported at high confidence.
- On **reliability**, the ranking inverts: the learned detector holds 100 % detection out to 30 m, across the full azimuth, and through rain, snow and fog (its keypoint error flat at $\approx$ 37 px, degrading gracefully), exactly where the high-contrast fiducial collapses; the LiDAR, being an active sensor, is illumination-independent but range-limited to $\sim$ 3 m and, per the published evidence, brittle to scattering and backscatter under weather and dust.
- On **computational efficiency**, no detector is priced out of the edge platform—all configurations clear > 50 FPS in isolated benchmarking—so cost decides *how* each is run rather than *whether*: the learned detector belongs on the GPU (where it is both the fastest at 5.4 ms and frees seven CPU cores), the AprilTag is the cheapest single-thread CPU option, and the LiDAR pipeline is inexpensive once backed by a KD-tree nearest-neighbour search.

The synthesis (Section 5.8) is therefore a *range-layered perception strategy*: marker-free learned vision plus RTK-GPS for long-range, all-weather capture and approach ($\sim$ 4–30 m), handing

off around 3–4 m to the AprilTag for sub-centimetre terminal alignment with the LiDAR as an illumination-independent geometric cross-check. A closed-loop simulation campaign confirmed that the assembled stack realises this division of labour end to end: 9 of 10 runs docked inside the ≤ 5 cm / 5° tolerance (mean 3.5 cm / 2.0°), and the few-centimetre accuracy floor was exactly the signature of an EKF-fused estimate trading the AprilTag’s sub-pixel sharpness for robustness. The empirical contribution is thus both the per-axis characterisation and the layered strategy it grounds—which, notably, also extends the two-detector fusion proposed on purely architectural grounds in Chapter 3 with the learned third element.

6.2. Discussion of Limitations

The results must be read against the scope deliberately drawn in Section 1.2.1; the principal limitations follow directly from those boundaries.

Terminal phase only. The thesis addresses the *terminal* docking phase—local perception, approach, and alignment—and the architecture that situates it. Global mission planning, the long-horizon route optimisation framed as a fuel-constrained Orienteering problem, and the cloud agronomic simulation are developed conceptually but neither implemented nor evaluated. The empirical claims therefore concern only the local approach and alignment, not the full mission cycle.

Threshold-based rather than predictive resource management. The docking trigger is a threshold on the path-cost-to-return energy estimate, not a predictive thermodynamic or terrain-energy model of battery depletion. This was a conscious simplification: it is sufficient to decide *when* to return but does not anticipate depletion under changing load, terrain, or temperature.

Architecture versus prototype. Only the docking subsystem is realised in hardware and simulation. The cloud-coupled agronomic simulation, the fluid and material replenishment, the agricultural actuators, and—importantly—the physical energy-transfer interface and station mating frame remain conceptual; the docking maneuver is evaluated up to terminal *alignment*, not live electrical *engagement*.

Simulation-anchored evaluation with static real-world validation. The single-variable accuracy, range, and weather comparisons depend on the controlled simulation testbed, whose exact ground truth and repeatability make a fair comparison possible but whose realism is inherently bounded. This was counter-checked on physical hardware—the learned detector and the LiDAR pipeline re-run on real captures, and an outdoor multi-sensor accuracy sweep against RTK-GPS—and every simulation finding (the fiducial and LiDAR range envelopes, the learned detector’s robustness, the LiDAR orientation ambiguity) survived the transfer. The validation is nonetheless *static*: it confirms per-detector behaviour, not the dynamics of the full maneuver outdoors.

Closed-loop validation is primarily in simulation. The end-to-end maneuver was characterised statistically only in simulation ($N = 10$, clear weather), as a demonstration of the maneuver dynamics rather than a field success-rate study; a statistical field characterisation was placed out of scope because exact per-maneuver outdoor ground truth is impractical to obtain at the needed repetition count. On the physical robot the full closed loop was exercised only as a small qualitative feasibility check.

GAP: physical closed-loop runs (≤ 3) to be collected 2026-06-16; insert the one-sentence qualitative result here and remove this note.

The LiDAR weather axis is from the literature. The 3D Gaussian-Splatting weather pipeline degrades only the RGB detectors; the LiDAR pipeline’s weather behaviour was argued from the published evidence (Section 5.2.1) rather than measured, so its expected ordering under rain and fog remains a hypothesis pending a controlled point-cloud weather experiment.

6.3. Conclusion and Future Work

This thesis treated autonomous docking as the precondition for long-term autonomous cultivation and addressed it on two fronts. Architecturally, it derived a ROS 2-based hybrid control architecture in which docking is embedded as a resource-triggered executive state rather than an isolated behaviour. Empirically, it showed that the docking perception problem is not solved by choosing one sensor but by layering three across the range axis—learned vision for robust long-range capture, the AprilTag for sub-centimetre terminal precision, and the LiDAR for illumination-independent position—a strategy validated statically on real hardware and end to end in simulation. Together they take one concrete step from the mission-based paradigm toward persistent field autonomy.

Two directions are the natural continuations of this work.

From the docking prototype to a complete long-term-autonomy system. The most consequential extension is to close the gap between the conceptual architecture and the realised prototype. The threshold-based docking trigger should be replaced by the *predictive resource model* identified in Section 1.2.1: a terrain- and load-aware energy forecast that anticipates depletion rather than merely reacting to a path-cost threshold, ideally coupled to the occupancy map so the return-energy estimate tracks the actual planned route. Beyond resource management, realising the remaining architectural pillars—the cloud agronomic “digital twin” that emits prioritised waypoints, the long-horizon route optimisation, the persistent world model, and the agricultural actuators—would turn the docking subsystem into the full resource-aware loop the architecture describes, and is what would let the platform be evaluated over the multi-day horizons that define LTA.

Completing the physical docking and energy interface. The maneuver is presently evaluated up to terminal alignment, not electrical engagement. Closing this requires selecting

and integrating the energy interface—the inductive etaLINK 3000 or WiBotic systems being the strongest matches to the requirement profile (Section 3.3.1), their generous misalignment tolerance also relaxing the precision demand on the maneuver—and co-designing the station’s mechanical mating frame, after which the Charge node and the full dock-charge-resume cycle can be exercised on hardware. Finalising the station geometry would additionally fix the LiDAR ICP target model, presently provisional.

Further empirical work. Two narrower follow-ups would sharpen the RQ2 result. The deferred *controlled LiDAR weather experiment*—injecting physically modelled rain, snow and fog scattering into the point cloud—would replace the literature-based argument of Section 5.2.1 with measurement and confirm the expected degradation ordering. And a sensor-fusion front end that resolves the LiDAR orientation ambiguity by accumulating the station’s opening over the approach—or simply deferring to the AprilTag at close range, as the layered strategy already prescribes—would remove the one failure mode the geometry cannot. Finally, a large- N outdoor closed-loop campaign across the weather axis, together with a perception-only terminal stage that drops the GPS prior, would lift the end-to-end result from the simulation-plus-qualitative-check of this thesis to a full statistical field characterisation.

A. Appendix

A.1. Bill of Materials (BOM)

The bill of materials required for the construction of the custom docking station is detailed in Table A.1.

Table A.1.: Bill of Materials (BOM) for the custom docking station.

Component	Type / Specifications	Dimensions / Size	Quantity
Profiles	8020 40-4040	0.84 m	4 pcs.
Profiles	8020 40-4040	0.112 m	6 pcs.
Aluminum Corner Bracket	40 Series	–	12 pcs.
Screws	M8 × 12 mm	–	24 pcs.
Nuts	M8	–	24 pcs.
Board	Thickness: 4 mm	1.15 m × 0.8 m	2 pcs.
Board for Roof	Thickness: 10 mm	1.25 m × 1.25 m	1 pc.

A.2. Electrical Schematics and Wiring Diagrams

The functional block diagram of the Holybro Pixhawk Jetson companion computer baseboard is shown in Figure A.1 (sourced from [103]). This diagram illustrates the power distribution pathways, communication interfaces, and peripheral connections between the Nvidia Jetson Orin NX and the integrated Pixhawk autopilot module.

A. Appendix

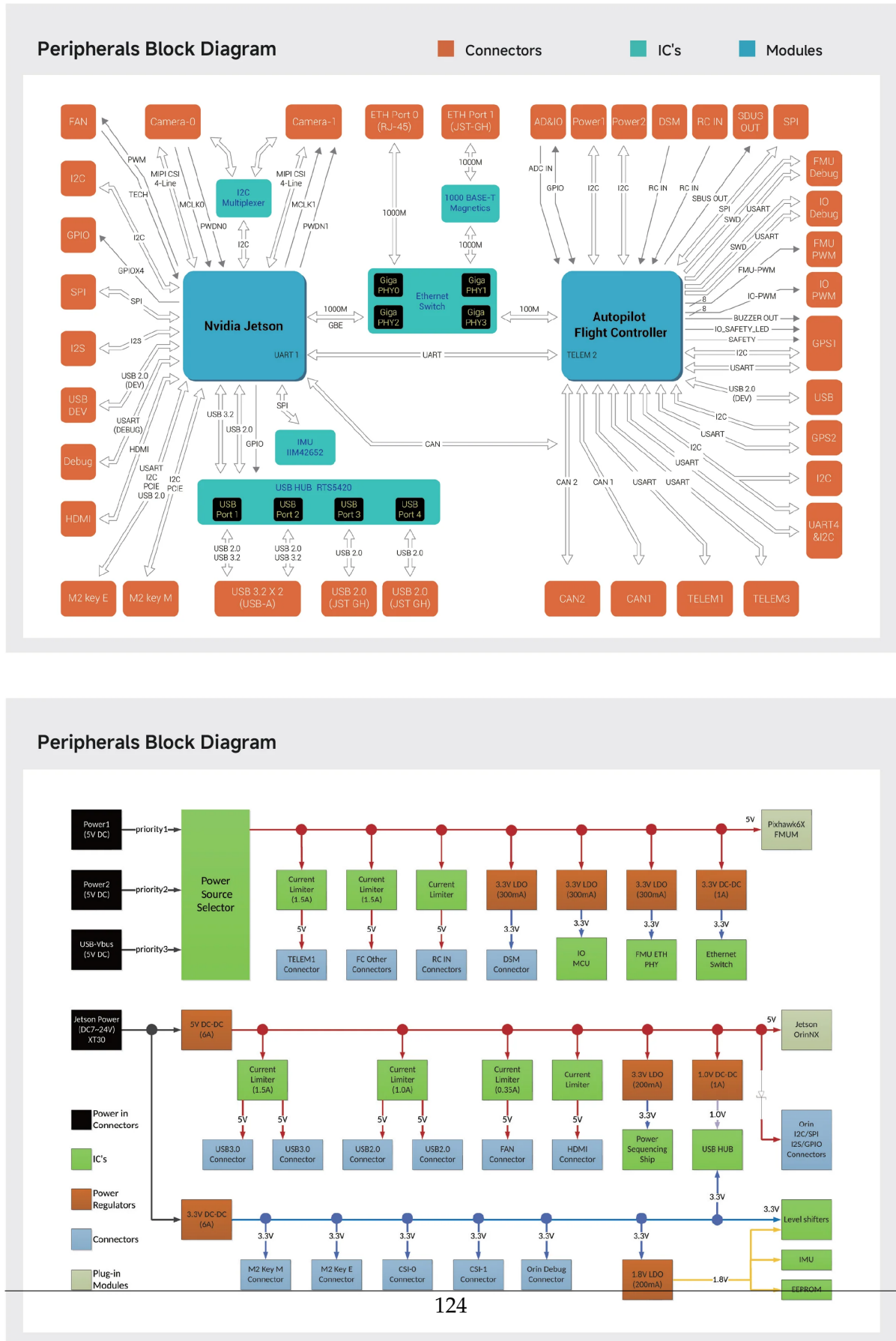


Figure A.1.: Functional block diagram of the Holybro Pixhawk Jetson companion computer baseboard, showing the internal power routing, communication links, and external peripheral interfaces.

The system-level wiring diagram of the complete robot platform is shown in Figure A.2. It covers the full power distribution chain from the LiFePO₄ battery through the fuse block and step-down regulators to every subsystem, together with the data and signal links (Ethernet, UART, PWM, USB, and RS-232) between the Pixhawk 6X Pro, the Jetson Orin NX, the drive electronics, the tool-motor drive, and the communication subsystem.

A.3. CAD Models of Custom Components

The CAD Models are in the GitHub Repo, in the folder: `models`. The full parametric CAD model of the custom docking station is hosted on Onshape and can be accessed via the following link:



A.4. Manufacturing Drawing of the Docking Station

A detailed technical overview drawing of the custom docking station with dimensions is shown in Figure A.3.

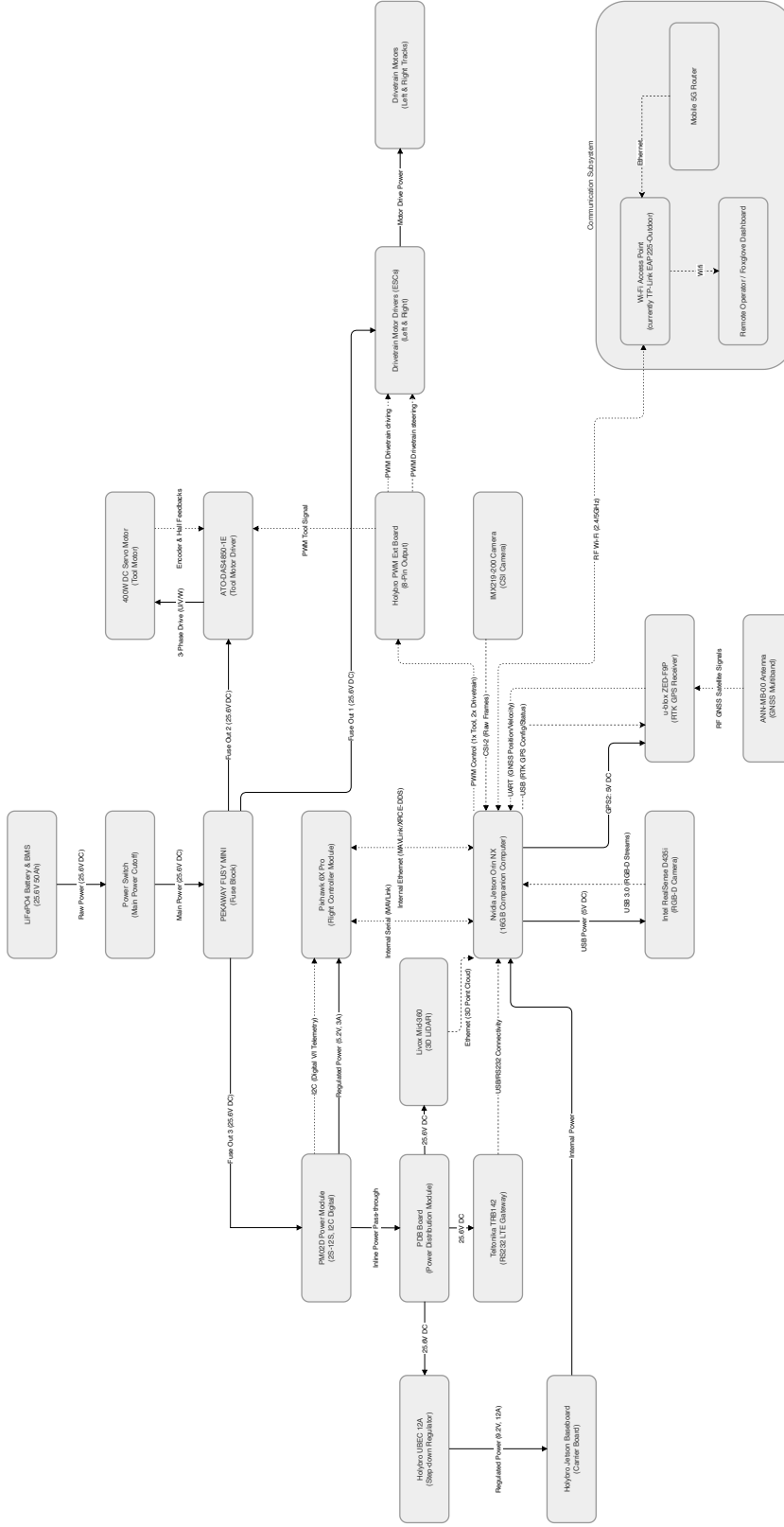


Figure A.2.: System-level wiring and power-distribution diagram of the complete robot platform, showing every subsystem from the LiFePO₄ battery to the drive ESCs, tool-motor drive, companion computer, flight controller, sensors, and communication subsystem. Solid arrows indicate power lines; dashed arrows indicate data or signal connections.

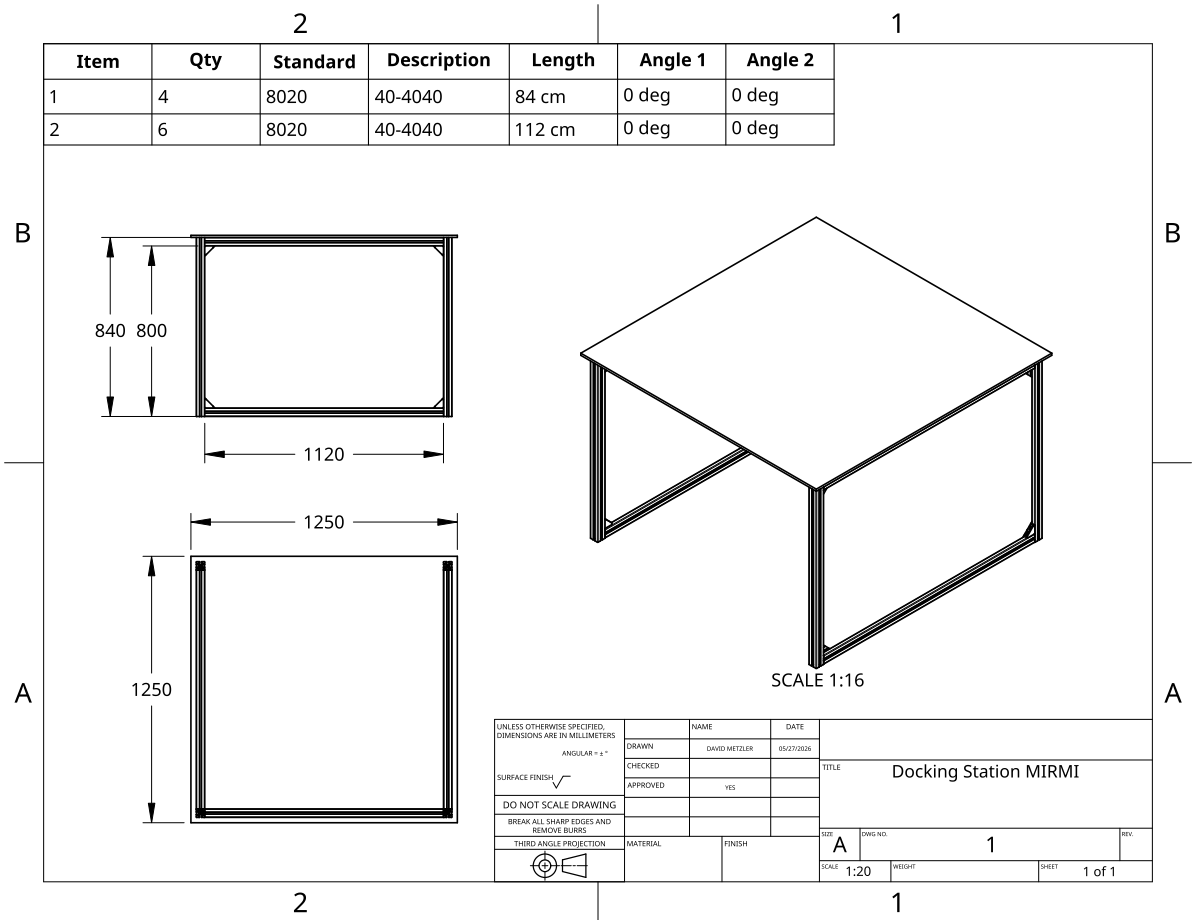


Figure A.3.: Detailed technical overview drawing of the custom docking station with dimensions.

A.5. Data Model (Entity–Relationship Schemas)

The databases that decouple the real-time on-robot loop from the asynchronous cloud pipeline in Figure 3.1 are backed by the relational schemas shown below. Figure A.4 models the on-robot raw-sensor logging (*Raw Measurements* database), Figure A.5 the agronomic sparse-measurement store (*Sparse Measurements* database), Figure A.6 the mission, waypoint and action-task hierarchy (*Planned Actions* database), and Figure A.7 the cloud crop-simulation history (*Historic* database).

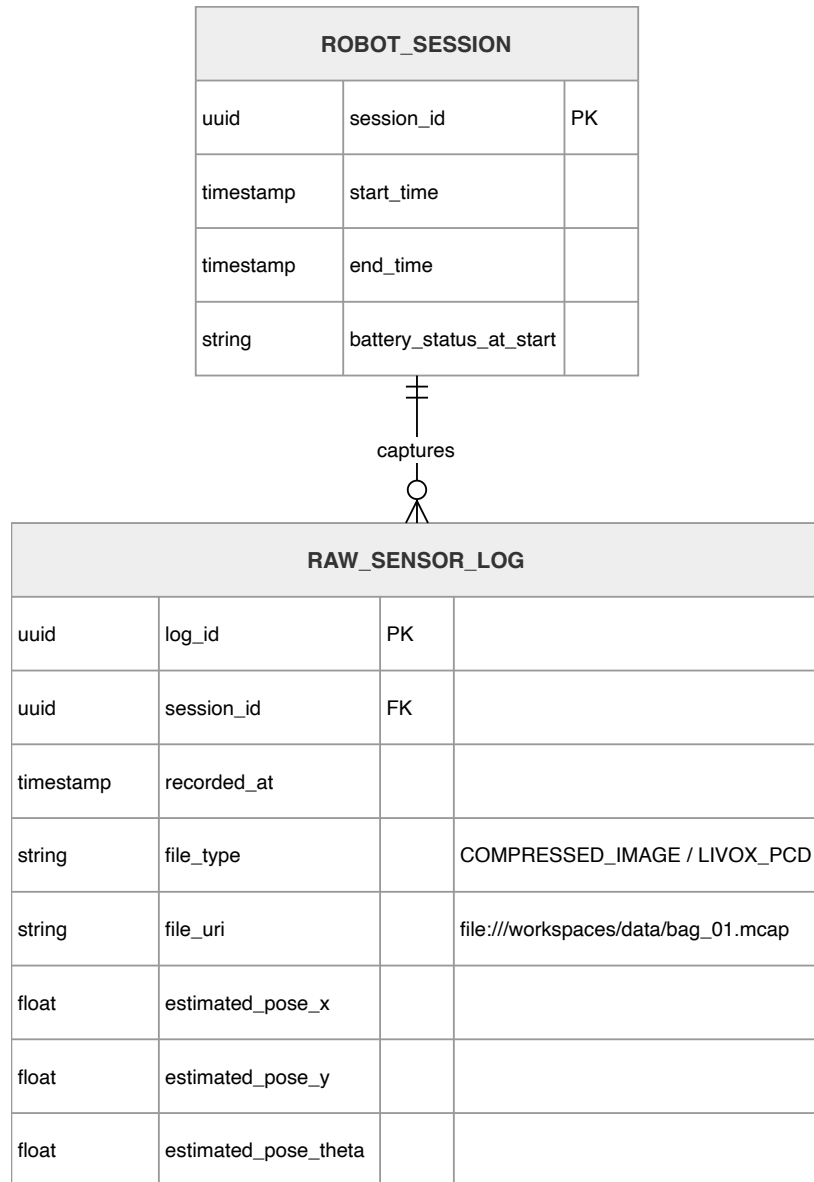


Figure A.4.: Raw sensor-log schema: each robot session captures a sequence of raw sensor logs (compressed images, LiDAR point clouds) with their estimated capture pose.

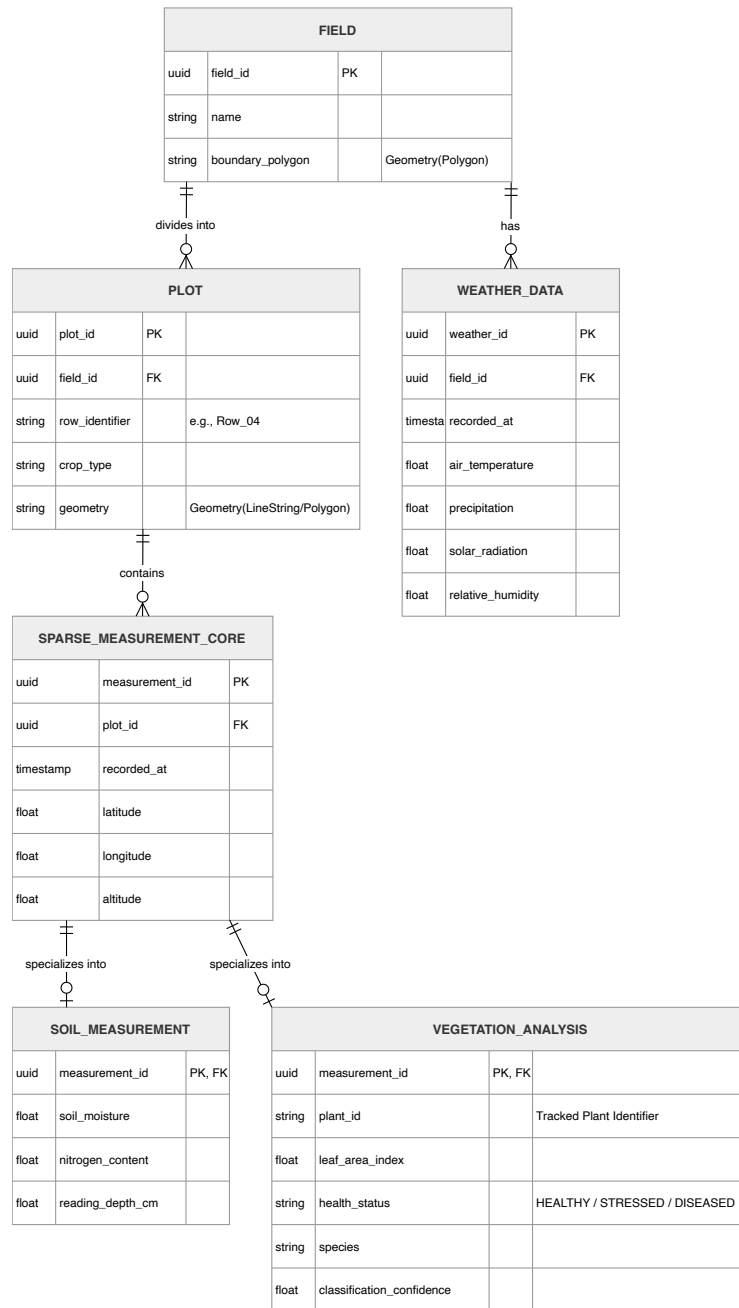


Figure A.5.: Agronomic sparse-measurement schema: a field divides into plots and carries weather data; each sparse measurement specialises into a soil or a vegetation analysis record.

A. Appendix

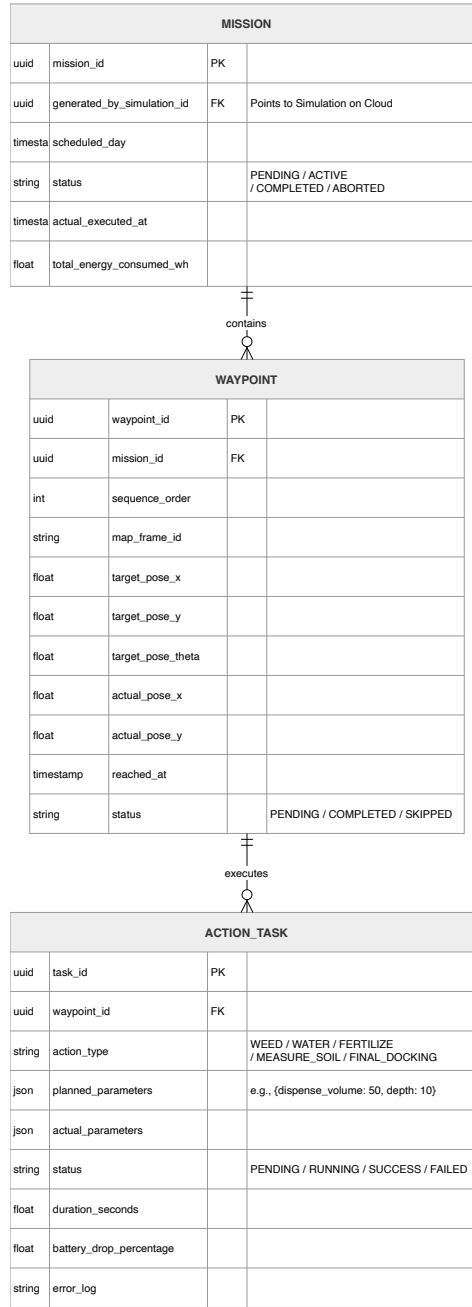


Figure A.6.: Mission and action-task schema: a mission contains an ordered sequence of waypoints, each of which executes one or more agricultural action tasks with their planned and actual parameters.

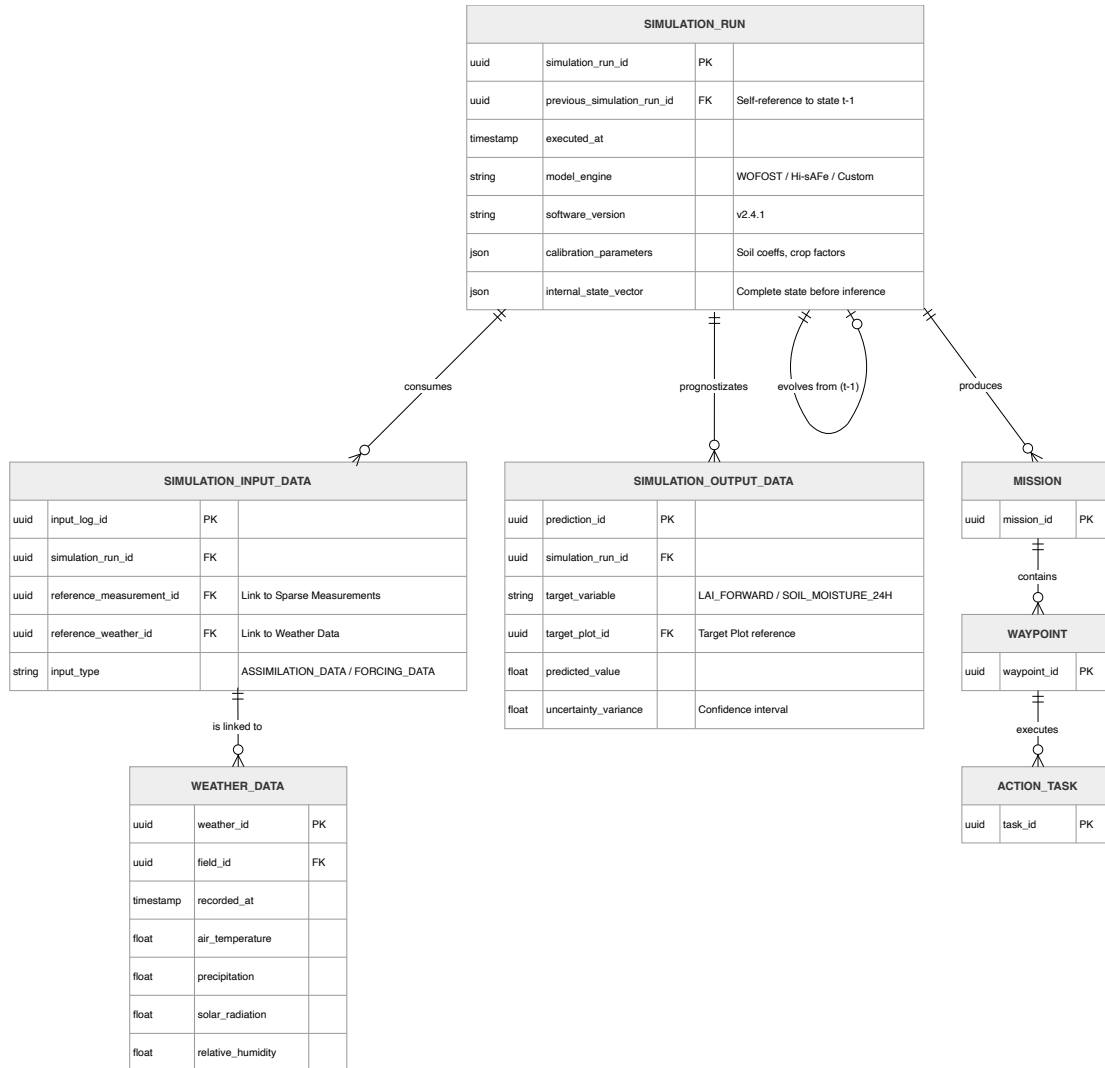


Figure A.7.: Cloud crop-simulation schema: each simulation run evolves from its predecessor, consumes assimilation and forcing data linked to the sparse measurements and weather records, and produces the predictions that seed the next mission.

A.6. Component Specifications and Datasheets

For reference, the key technical specifications of the principal implemented hardware components and the surveyed charging interfaces are summarized in Table A.2 and Table A.3, respectively. The complete official datasheets are archived in the repository folder `Bib_pdf/datasheets/` to ensure long-term reproducibility, and are cited individually in the tables below:

Table A.2.: Summary of implemented hardware components.

Component / System	Manufacturer	Key Technical Specifications	Citation
Intel RealSense D400	Intel Corporation	• Active IR stereo depth camera • Depth range: 0.4–10 m • Global/rolling shutter, up to 90 fps	[73]
u-blox ZED-F9P	u-blox AG	• Multi-band (L1/L2/L5) GNSS receiver • Centimeter-level RTK accuracy (0.01 m + 1 ppm) • Up to 20 Hz update rate	[104]
ArduSimple RTK Starter Kit	ArduSimple	• Dual-band (L1/L2) GNSS evaluation platform • Integrates u-blox ZED-F9P receiver module • Medium-range radio link (up to 1.2 km) for corrections	[105]
Pixhawk Jetson Baseboard	Holybro	• Companion computer carrier board • Integrates Jetson Orin NX and Pixhawk • Dual Ethernet, CAN, UART, PWM interfaces	[100]
TP-Link EAP225-Outdoor	TP-Link	• Outdoor AC1200 dual-band Wi-Fi access point • Weatherproof IP65-rated enclosure • Supports local bulk data backup and telemetry sync	[99]
Holybro PWM Extension Board	Holybro	• PWM signal expansion board for Pixhawk carrier boards • Supports both MAIN (I/O) and AUX (FMU) PWM outputs • Exposes 8-pin header interface for servos/motors	[102]
Holybro UBEC 12A	Holybro	• Synchronous step-down DC-DC power regulator • Input: 3–14S LiPo (up to 60 V) • Output: 6.0/7.2/8.0/9.2 V, up to 12 A continuous	[101]

Table A.3.: Summary of surveyed charging interfaces.

Component / System	Manufacturer	Key Technical Specifications	Citation
WiBotic OC-301 / TR-302	WiBotic Inc.	• Wireless inductive charging (up to 300 W) • OC-301: 12–58.8 V output, up to 30 A • TR-302 transmitter power supply	[93, 94]
WirelessCharger 3.0	Conductix-Wampfler	• Inductive wireless charging (up to 3.0 kW) • Up to 80 A charging current • 10–40 mm air gap, >90% efficiency	[117, 118]
etaLINK 3000	Wiferion GmbH	• Contactless inductive power transfer • Output: 3.0 kW (up to 60 A at 15–60 V) • IP65 ingress protection, >90% efficiency	[92]
MultiMag 15	Rosenberger	• Self-locating magnetic break-away connector • 15-pin interface rated up to 4 A/contact • IP65/IP67 ingress protection	[89]
Pogo Pins / Contact Pads	Harwin Plc	• High-durability spring-loaded contacts • Rated up to 2 A per contact (P70-220) • 4.0 mm working height, 10k mating cycles	[91]

A.7. Foxglove GUI Layout Configurations

To replicate the interactive visualization and monitoring environment, the JSON configuration files for the custom Foxglove Studio layouts are included in the repository folder: `layouts/`.

These files can be imported directly into Foxglove Studio by opening the application, selecting *Layout* → *Import from file...*, and choosing one of the following configurations:

- `Robot_Control_Layout.json` — The main operational control interface (shown in Figure 4.5) displaying real-time camera views, 3D coordinate frame transformations, point clouds, and active state indicators.
- `Robot_Testing_Layout.json` — A diagnostic dashboard optimized for conducting automated test sweeps, visualizing sensor telemetry plots, error signals, and tracking trajectory alignment.

A.8. Source Code

The Code running on the Robot can be found in GitHub under



The Code for running the simulation can be found in Github under:



A.9. Documentation

The Code Documentation can be found in GitHub under:



List of Figures

2.1.	The six dimensions of robotic architectures, divided into the Temporal & Information Space (left) and the Structural & Representation Space (right). These axes provide a framework for classifying the capabilities and structural paradigms of autonomous systems.	13
2.2.	The Sense-Plan-Act (SPA) architecture depicted in a discrete grid environment. (a) The robot maps obstacles to construct a world model. (b) A pathfinding algorithm computes a collision-free trajectory. (c) The controller executes the generated path.	17
2.3.	A reactive architecture (Motor Schemas) navigating a discrete grid environment. In contrast to the SPA architecture, the robot does not maintain a global world model or pre-compute a trajectory. Instead, movement is driven continuously by a localized vector field. The robot reacts purely to stimuli—attraction to the goal and repulsion from obstacles—that fall strictly within its immediate sensor horizon, resulting in a direct, emergent trajectory.	18
2.4.	A 3-tiered hybrid architecture managing a dynamic environment. (a) The deliberative layer generates a static, global trajectory. (b) The sequencer monitors execution and yields control to the reactive layer when an unexpected dynamic obstacle blocks the path, generating a localized vector field to avoid collisions. (c) After successfully navigating the obstacle, the reactive layer hands control back, allowing the robot to seamlessly rejoin and complete the global plan. . .	20
2.5.	The Vision-Language-Action (VLA) paradigm applied to navigation. (a) The policy takes raw egocentric visual observations and a natural language instruction as input. (b) Rather than generating a symbolic map or pre-calculating a trajectory, the model directly outputs an immediate low-level action vector. (c) Execution is closed-loop; the model continuously predicts next actions based on real-time visual feedback.	23
2.6.	The four constituent layers of a digital twin [53]. Physical state flows upward through the <i>data</i> , <i>model</i> , and <i>application</i> layers (blue), while the <i>application</i> layer's outputs—predictions, optimized parameters, commands—are fed back to the physical asset (orange), closing the bidirectional loop that distinguishes a digital twin from a digital shadow. The <i>network</i> layer provides the low-latency communication that couples all layers.	25
2.7.	The Hi-sAFe biophysical model and simulation scene. (a) Daily crop growth is simulated per voxel cell, resolving below- and above-ground competition. (b) A 3D simulated agroforestry scene showing tree–crop spatial interactions. Figure adapted from [60].	27

2.8.	Sensor modalities organized by docking phase. Each phase requires progressively higher precision at shorter range, necessitating complementary sensor constellations. Dots mark the junction between the vertical spine and each sensor branch.	28
2.9.	Angle-of-arrival localization. Triangulation estimates the receiver position from the intersection of bearing rays (left), while antenna-array AoA derives those bearings from phase differences between spatially separated receiver elements (right).	29
2.10.	Range-based localization. Trilateration (left) determines position from the intersection of range circles around known anchors. TDoA multilateration (right) replaces circles with hyperbolae defined by time-difference-of-arrival constraints; their intersection uniquely identifies the target.	30
2.11.	Phase- and frequency-based ranging. Left: a received sinusoid is delayed relative to the transmitted reference by a phase offset $\varphi = 2\pi d/\lambda$, from which the sub-wavelength distance is recovered. Right: relative motion of the target compresses the returned waveform, producing a Doppler frequency shift $\Delta f = 2v/\lambda$ that encodes radial velocity.	31
2.12.	Conceptual overview of the RANSAC algorithm for geometric model fitting. Outliers (orange) are discarded, and an optimal model (blue) is fitted using the inliers (green) within a consensus threshold band of 2ϵ	32
2.13.	Conceptual overview of the Iterative Closest Point (ICP) algorithm. Left: Step 1 searches for the closest target points $q_j \in Q$ for each source point $p_i \in P$. Right: Step 2 computes the optimal transformation (R, t) that minimizes alignment error, bringing the transformed source cloud P' into alignment with target cloud Q	33
2.14.	Conceptual overview of the Normal Distributions Transform (NDT) algorithm. The target point cloud is voxelized into cells, and the points inside are represented as a local Gaussian probability density function (mean μ , covariance Σ , represented by ellipses). Source points x_k are registered by maximizing their likelihood on this continuous PDF.	34
2.15.	The four localization topologies for autonomous docking. Emission-based and world-based topologies place the signal source outside the robot; reflection-based places the active emitter on the robot; proprioceptive requires no external signal at all.	36
2.16.	The FarmDroid FD20 agricultural robot, which harvests solar energy via on-board photovoltaic panels to power seeding and weeding operations.	46
3.1.	High-level system architecture	54
3.2.	Notation used in the architecture diagrams	55
3.3.	Preliminary test with active infrared emitters	62
3.4.	The Sense subsystem	64
3.5.	The Plan subsystem	65
3.6.	The Act subsystem	66

4.1. The physical mobile test platform carrying the companion computer and sensor suite, positioned in front of the docking station in the outdoor experimental environment.	76
4.2. Holybro Pixhawk Jetson Baseboard	77
4.3. Top-down view of the static sweep measurement grid. The docking station (red) is at the origin; blue squares mark the 36 robot positions at which sensor data were recorded. Marker symbols on the station indicate the four AprilTag faces.	80
4.4. The docking station with its four AprilTag fiducial markers: three mounted on the outer walls and one on the inner back wall facing the opening.	81
4.5. The customized Foxglove user interface layout used for monitoring the robot's sensor data, state transitions, and pose estimation during docking operations.	84
5.1. Max-distance sweep: AprilTag versus learned keypoints	87
5.2. Azimuth orbit: AprilTag versus learned keypoints	88
5.3. Max-distance sweep: camera resolution	90
5.4. Max-distance sweep: tag size	91
5.5. Lateral alignment micro-sweep	92
5.6. Weather robustness: AprilTag versus learned keypoints	93
5.7. Training-data ablation of the learned detector	97
5.8. LiDAR max-distance sweep	98
5.9. LiDAR orbit: detection rate and confidence	99
5.10. LiDAR orbit: pose errors	100
5.11. LiDAR detection pipeline, stage by stage	102
5.12. The two LiDAR terminal failure modes	103
5.13. End-to-end docking trajectories	106
5.14. Terminal docking error distributions	107
5.15. Real-world validation of the learned detector	110
5.16. Real LiDAR: detection versus distance	110
5.17. Real LiDAR: orbit and the orientation ambiguity	111
5.18. Outdoor multi-sensor accuracy sweep	113
5.19. Compute cost: baseline comparison	115
5.20. Compute cost: camera-method resolution scaling	116
A.1. Pixhawk Jetson Baseboard block diagram	124
A.2. System-level wiring diagram	126
A.3. Manufacturing drawing of the docking station	127
A.4. Raw sensor-log schema	128
A.5. Agronomic sparse-measurement schema	129
A.6. Mission and action-task schema	130
A.7. Cloud simulation schema	131

List of Tables

2.1.	Classification of the surveyed architectural archetypes along the six dimensions of Section 2.2.1. Reactive control is represented by both Motor Schemas (MS) and the Subsumption Architecture (Sub.). Spans (e.g. “Strategic ↔ Reflexive”) indicate that an architecture deliberately occupies the full range of a dimension across its internal tiers.	16
2.2.	Synthesis evaluation matrix of sensor modalities for outdoor autonomous docking. Ordinal scale (robustness, cost rows): 0 = None, 1 = Very Low, 2 = Low, 3 = Medium, 4 = High, 5 = Very High. LOS column: N = not required, P = partial, Y = required. Phase column: G = Global, L = Local, T = Terminal, A = All. Quantitative figures are indicative for representative hardware. [†] Depth camera accuracy is for controlled indoor conditions only; outdoor performance is effectively none due to solar saturation. [‡] Magnetometer accuracy is heading (angular), not position.	44
3.1.	Evaluation of candidate electrical energy transfer options against the five governing requirements. Rating: + = meets requirement; o = partially meets; – = does not meet.	58
3.2.	Evaluation of candidate data transfer options based on real performance and cost metrics compiled from cited sources. Cost figures refer to typical hardware procurement and monthly service subscriptions as of late 2025/2026.	58
4.1.	Numeric ranges of the experimental parameters, split horizontally into simulation sweeps (further grouped by vision modalities, LiDAR modality, and simulated weather degradation) and real-world experiments. Each row is varied independently while the others are held at their reference value.	75
4.2.	Simulated environmental degradation parameters applied to vision modalities (AprilTag & YOLOv8-pose).	75
4.3.	Sensor payload of the mobile test platform and the mounting height of each device above ground. The localisation sensors are arranged so that the camera looks slightly downward onto the AprilTag markers at 0.425 m.	77

5.1.	Head-to-head summary of the AprilTag and the best learned detector (SIM+REAL+WEATHER) across the three evaluation axes. Lower pixel error and higher detection rate are better; the bold entries mark the winner on each row. The AprilTag detection range is quoted at the 640×480 reference resolution; raising the resolution or tag size extends it in simulation (Section 5.1.3), but the real-world range stayed near 5 m (Section 5.6.3). The AprilTag wins precision; the learned detector wins range, coverage and weather robustness.	94
5.2.	Training-data ablation of the learned detector across the evaluation sweeps. Each row is the identical network trained on a different corpus. The real-only model—trained on the small hand-labelled set—collapses; simulation data alone already recovers near-complete performance, and the hand-labelled real and weather sets contribute the final increments.	96
5.3.	Closed-loop end-to-end docking performance in the simulation testbed: ten runs from random spawns in the 3–10 m ring, each driven by the full GPS → learned-vision → precise-dock behaviour tree with an EKF-fused terminal pose. Success rates are over all ten runs; the accuracy and timing statistics are over the nine docked runs.	105
5.4.	Isolated compute cost of the three detectors at their deployed configurations (benchmark host: AMD Ryzen 7 8845HS + RTX 3050 6 GB; absolute values are host-specific). Latency is the mean per-frame inference time; CPU load is the mean number of busy cores; peak memory is process RSS, or CUDA memory for the GPU configuration. All four clear real-time at > 50 FPS.	114
5.5.	The range-layered perception strategy supported by the evaluation. Each detector is assigned the range band in which the measurements above show it dominant, and the dominant failure that excludes it from the others.	117
A.1.	Bill of Materials (BOM) for the custom docking station.	123
A.2.	Summary of implemented hardware components.	132
A.3.	Summary of surveyed charging interfaces.	133

Glossary

AprilTags A visual fiducial system used for tasks like camera calibration, pose estimation, and object identification. They are similar to QR codes but designed specifically for robotics and computer vision applications.. 34, 35

autonomy The ability of a system to perform tasks without continuous human guidance or intervention.. i, 2, 4, 7, 8, 11, 13, 15, 16, 20, 45, 49, 50, 52, 118

base station A central hub or fixed location where a mobile robot returns to replenish resources, upload data, or receive new mission parameters.. 38

behavior tree A highly modular, hierarchical mathematical model of plan execution used extensively in navigation stacks to structure complex, reactive robotic tasks through execution nodes that return success, failure, or running states.. 21, 22, 51, 52, 67, 68, 78

docking station A stationary platform that provides the mobile robot with power, data, and/or material interfaces for autonomous recharging and mission preparation.. 7, 21, 28, 29, 31, 35–37, 39, 40, 42, 45, 46, 49–51, 53, 59–61, 68, 74, 76, 79–81, 83, 125, 137

Foxglove A specialized visualization and debugging platform used to inspect topics, visualize telemetry, and control robotic systems during simulation and live operations.. 134

Gazebo An advanced 3D robotics simulation environment used to model physical interactions, sensor data, and dynamic constraints before deploying code to hardware.. 79

interface A connection point or shared boundary between two systems or components, allowing for the transfer of power, data, or materials.. 2–4, 47–51, 57, 66–69, 134

odometry The use of data from motion sensors, such as wheel encoders or inertial measurement units, to continuously estimate the change in a robot's position relative to its starting location.. 43, 64, 66

point cloud A dense set of data points in space, typically generated by LiDAR sensors or depth cameras, representing the external surfaces of the environment for obstacle detection and spatial registration.. 31, 39, 56, 83, 128, 134

sensor suite A collection of different sensors (e.g., UWB, LiDAR, IR camera) used together to provide a comprehensive perception of the environment for navigation and task execution.. 1, 63

ultrasound A sensor modality utilizing high-frequency sound waves to measure the distance to objects, commonly employed for close-range obstacle detection and avoidance.. 31, 37, 41, 44

Acronyms

AoA Angle of Arrival. 29, 41, 136

BMS Battery Management System. 77

BT Behavior Tree. 21, 22, 52, 68, 69, 118

DDS Data Distribution Service. 52

EKF Extended Kalman Filter. 43, 47, 73, 104, 105, 108, 120, 139

FPFH Fast Point Feature Histograms. 34

FSM Finite State Machine. 22

GNSS Global Navigation Satellite System. 37, 38, 43

GPS Global Positioning System. 1, 37, 38, 43, 44, 50, 60, 61, 63, 66, 69, 77, 78, 85, 104, 105, 108, 112, 113, 117, 119, 120, 122

HSM Hierarchical State Machine. 22

ICP Iterative Closest Point. 32, 33, 109–111, 136

IMU Inertial Measurement Unit. 42–44, 63, 66, 77

IR Infrared. 36, 38–41, 44, 46, 48

LiDAR Light Detection and Ranging. iv, 3, 4, 31, 37, 39, 41, 43, 44, 46, 50, 59–61, 63, 64, 66, 67, 69, 71, 72, 74, 77, 78, 81, 83, 85, 93–97, 99, 101–104, 108, 109, 112–117, 119–122, 128

LTA Long Term Autonomy. 1–4, 8, 21, 45, 49, 118, 121

MIRMI Munich Institute of Robotics and Machine Intelligence. iii

Nav2 Navigation 2. 52, 65, 67, 69

NDT Normal Distributions Transform. 33, 34, 136

PCA Principal Component Analysis. 97, 101, 102

- PID** Proportional-Integral-Derivative. 14
- RANSAC** Random Sample Consensus. 31, 32, 136
- RL** Reinforcement Learning. 22, 24
- ROS 2** Robot Operating System 2. iv, 3, 4, 19, 21, 46, 50, 52, 54, 55, 59, 63, 64, 66, 67, 69, 77, 78, 83, 112, 118, 119, 121
- RSSI** Received Signal Strength Indicator. 30, 40
- RTK** Real-Time Kinematic. 38, 44, 60, 61, 63, 66, 69, 77, 78, 85, 108, 112, 113, 117, 119, 120
- SLAM** Simultaneous Localization and Mapping. 39, 60, 66, 69, 77
- SOTA** State of the Art. 4, 22, 45
- SPA** Sense Plan Act. 16–18, 20, 52, 63, 118, 135
- SSIM** Structural Similarity Index Measure. 110
- TDoA** Time Difference of Arrival. 30, 40, 136
- ToF** Time of Flight. 29, 31, 39, 41
- TUM** Technical University of Munich. i, iii
- URDF** Unified Robot Description Format. 77
- UWB** Ultra-Wideband. 36, 38, 40, 44
- YOLO** You Only Look Once. iv, 71, 72, 85, 104, 119

Bibliography

- [1] FarmDroid. *FarmDroid FD20: Autonomous Seeding and Weeding Robot*. FarmDroid ApS. URL: <https://farmdroid.com/products/farmdroid-fd20/> (visited on 06/03/2026).
- [2] Naïo Technologies. *OZ and Dino: Autonomous Weeding Robots*. Naïo Technologies. URL: <https://www.naio-technologies.com/en/> (visited on 06/03/2026).
- [3] Ecorobotix. *ARA: Ultra-High Precision Spot Sprayer*. Ecorobotix SA. URL: <https://ecorobotix.com/en-us/crop-care/ara-field-sprayer/> (visited on 06/03/2026).
- [4] John Deere. *John Deere 8R Autonomous Tractor*. Deere & Company. URL: <https://www.deere.com/en/technology-products/precision-ag-technology/autonomous-equipment/8r-autonomous-tractor/> (visited on 06/05/2026).
- [5] AgXeed. *AgBot: Autonomous Field Robot*. AgXeed B.V. URL: <https://www.agxeed.com/solutions/agbots/> (visited on 06/05/2026).
- [6] S. Royo and M. Ballesta-Garcia. “An Overview of Lidar Imaging Systems for Autonomous Vehicles”. In: *Applied Sciences* 9.19 (Oct. 1, 2019), p. 4093. ISSN: 2076-3417. DOI: 10.3390/app9194093. URL: <https://www.mdpi.com/2076-3417/9/19/4093> (visited on 06/13/2026).
- [7] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion”. In: *Proceedings of Robotics: Science and Systems (RSS)*. 2023. DOI: 10.15607/RSS.2023.XIX.026. arXiv: 2303.04137. URL: <https://arxiv.org/abs/2303.04137>.
- [8] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa. “Learning from Interventions: Human-Robot Interaction as Both Explicit and Implicit Feedback”. In: *Proceedings of Robotics: Science and Systems XVI*. 2020. DOI: 10.15607/RSS.2020.XVI.055.
- [9] S. G. Vougioukas. “Agricultural Robotics”. In: *Annual Review of Control, Robotics, and Autonomous Systems* 2.1 (May 3, 2019), pp. 365–392. DOI: 10.1146/annurev-control-053018-023617. URL: <https://www.annualreviews.org/content/journals/10.1146/annurev-control-053018-023617>.
- [10] D. Gackstetter, M. von Bloh, V. Hannus, S. T. Meyer, W. W. Weisser, C. Luksch, and S. Asseng. “Autonomous Field Management – An Enabler of Sustainable Future in Agriculture”. In: *Agricultural Systems* 206 (2023), p. 103607. DOI: 10.1016/j.agry.2023.103607.

- [11] M. Sollen-Norrin, B. B. Ghaley, and N. L. J. Rintoul. “Agroforestry Benefits and Challenges for Adoption in Europe and Beyond”. In: *Sustainability* 12.17 (Aug. 27, 2020), p. 7001. ISSN: 2071-1050. DOI: 10.3390/su12177001. URL: <https://www.mdpi.com/2071-1050/12/17/7001> (visited on 11/18/2025).
- [12] O. I. Ollinaho and M. Kröger. “Agroforestry Transitions: The Good, the Bad and the Ugly”. In: *Journal of Rural Studies* 82 (Feb. 2021), pp. 210–221. ISSN: 07430167. DOI: 10.1016/j.jrurstud.2021.01.016. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0743016721000164> (visited on 11/18/2025).
- [13] A. Quandt, H. Neufeldt, and K. Gorman. “Climate Change Adaptation through Agroforestry: Opportunities and Gaps”. In: *Current Opinion in Environmental Sustainability* 60 (Feb. 2023), p. 101244. ISSN: 18773435. DOI: 10.1016/j.cosust.2022.101244. URL: <https://linkinghub.elsevier.com/retrieve/pii/S1877343522000963> (visited on 11/18/2025).
- [14] L. Troesken and D. Duecker. “Biodiversity in the Loop: Can Field Robots Learn Ecologically Aware Management”. In: ().
- [15] S. J. Scherr. “On-Farm Research: The Challenges of Agroforestry”. In: *Agroforestry Systems* 15.2–3 (Sept. 1991), pp. 95–110. ISSN: 0167-4366, 1572-9680. DOI: 10.1007/BF00120183. URL: <http://link.springer.com/10.1007/BF00120183> (visited on 11/18/2025).
- [16] L. Trosken and D. Ducker. “Autonomous Robotics as an Enabler for Sustainable Agroforestry Systems”. In: ().
- [17] N. Kapetanović, J. Goričanec, I. Vatauvuk, I. Hrabar, D. Stuhne, G. Vasiljević, Z. Kovačić, N. Mišković, N. Antolović, M. Anić, and B. Kozina. “Heterogeneous Autonomous Robotic System in Viticulture and Mariculture: Vehicles Development and Systems Integration”. In: *Sensors* 22.8 (Apr. 12, 2022), p. 2961. ISSN: 1424-8220. DOI: 10.3390/s22082961. URL: <https://www.mdpi.com/1424-8220/22/8/2961> (visited on 04/23/2026).
- [18] L. Kunze, N. Hawes, T. Duckett, M. Hanheide, and T. Krajnik. “Artificial Intelligence for Long-Term Robot Autonomy: A Survey”. In: *IEEE Robotics and Automation Letters* 3.4 (Oct. 2018), pp. 4023–4030. ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2018.2860628. URL: <https://ieeexplore.ieee.org/document/8421618/> (visited on 10/31/2025).
- [19] A. Romero, F. Bellas, R. J. Duro, A. Romero, F. Bellas, and R. J. Duro. “A Perspective on Lifelong Open-Ended Learning Autonomy for Robotics through Cognitive Architectures”. In: *Sensors* 23.3 (Feb. 2, 2023). ISSN: 1424-8220. DOI: 10.3390/s23031611. URL: <https://www.mdpi.com/1424-8220/23/3/1611> (visited on 11/01/2025).
- [20] L. Wijayathunga, A. Rassau, and D. Chai. “Challenges and Solutions for Autonomous Ground Robot Scene Understanding and Navigation in Unstructured Outdoor Environments: A Review”. In: *Applied Sciences* 13.17 (Aug. 31, 2023), p. 9877. ISSN: 2076-3417.

- DOI: 10.3390/app13179877. URL: <https://www.mdpi.com/2076-3417/13/17/9877> (visited on 04/23/2026).
- [21] N. Hawes, C. Burbridge, F. Jovan, L. Kunze, B. Lacerda, L. Mudrová, J. Young, J. Wyatt, D. Hebesberger, T. Kortner, R. Ambrus, N. Bore, J. Folkesson, P. Jensfelt, L. Beyer, A. Hermans, B. Leibe, A. Aldoma, T. Faulhammer, M. Zillich, M. Vincze, E. Chinellato, M. Al-Omari, P. Duckworth, Y. Gatsoulis, D. C. Hogg, A. G. Cohn, C. Dondrup, J. Pulido Fentanes, T. Krajník, J. M. Santos, T. Duckett, and M. Hanheide. "The STRANDS Project: Long-Term Autonomy in Everyday Environments". In: *IEEE Robotics & Automation Magazine* 24.3 (2017), pp. 146–156. DOI: 10.1109/MRA.2016.2636359.
 - [22] A. Birk. "AUTONOMOUS RECHARGING OF MOBILE ROBOTS". In: ().
 - [23] A. M. Romanov and A. A. Tararin. "An Automatic Docking System for Wheeled Mobile Robots". In: *2021 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus)*. 2021 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus). St. Petersburg, Moscow, Russia: IEEE, Jan. 26, 2021, pp. 1040–1045. ISBN: 978-1-6654-0476-1. DOI: 10.1109/ElConRus51938.2021.9396509. URL: <https://ieeexplore.ieee.org/document/9396509/> (visited on 11/01/2025).
 - [24] C. Dupraz, K. Wolz, I. Lecomte, G. Talbot, G. Vincent, R. Mulia, F. Bussière, H. Ozier-Lafontaine, S. Andrianarisoa, N. Jackson, G. Lawson, N. Dones, H. Sinoquet, B. Lusiana, D. Harja, S. Domenicano, F. Reyes, M. Gosme, and M. Van Noordwijk. "HisAFe: A 3D Agroforestry Model for Integrating Dynamic Tree–Crop Interactions". In: *Sustainability* 11.8 (Apr. 16, 2019), p. 2293. ISSN: 2071-1050. DOI: 10.3390/su11082293. URL: <https://www.mdpi.com/2071-1050/11/8/2293> (visited on 05/16/2026).
 - [25] S. Kyberd, J. Attias, P. Get, P. Murcutt, C. Prahacs, M. Towlson, S. Venn, A. Vasconcelos, M. Gadd, D. D. Martini, and P. Newman. "The Hulk: Design and Development of a Weather-Proof Vehicle for Long-Term Autonomy in Outdoor Environments". In: *Field and Service Robotics*. Ed. by G. Ishigami and K. Yoshida. Vol. 16. Singapore: Springer Singapore, 2021, pp. 101–114. ISBN: 978-981-15-9459-5 978-981-15-9460-1. DOI: 10.1007/978-981-15-9460-1_8. URL: http://link.springer.com/10.1007/978-981-15-9460-1_8 (visited on 11/01/2025).
 - [26] A. Belyaev and O. Kushnarev. "Energy-Based Surface Classification for Mobile Robots in Known and Unexplored Terrains". In: *Robotics* 14.9 (Sept. 21, 2025), p. 130. ISSN: 2218-6581. DOI: 10.3390/robotics14090130. URL: <https://www.mdpi.com/2218-6581/14/9/130> (visited on 04/23/2026).
 - [27] K. Lee and K. Lee. "Terrain-aware Path Planning via Semantic Segmentation and Uncertainty Rejection Filter with Adversarial Noise for Mobile Robots". In: *Journal of Field Robotics* 42.1 (Jan. 2025), pp. 287–301. ISSN: 1556-4959, 1556-4967. DOI: 10.1002/rob.22411. URL: <https://onlinelibrary.wiley.com/doi/10.1002/rob.22411> (visited on 04/23/2026).

- [28] J. M. Bradshaw, R. R. Hoffman, M. Johnson, and D. D. Woods. "The Seven Deadly Myths of "Autonomous Systems"". In: *IEEE Intelligent Systems* 28.3 (May 2013), pp. 54–61. ISSN: 1541-1672. DOI: 10.1109/MIS.2013.70. URL: <http://ieeexplore.ieee.org/document/6588858/> (visited on 04/23/2026).
- [29] S. Braynov and H. Hexmoor. "Quantifying Relative Autonomy in Multiagent Interaction". In: *Agent Autonomy*. Ed. by H. Hexmoor, C. Castelfranchi, and R. Falcone. Red. by G. Weiss. Vol. 7. Boston, MA: Springer US, 2003, pp. 55–73. ISBN: 978-1-4613-4833-7 978-1-4419-9198-0. DOI: 10.1007/978-1-4419-9198-0_4. URL: http://link.springer.com/10.1007/978-1-4419-9198-0_4 (visited on 04/23/2026).
- [30] E. Shi, T. M. Gasser, A. Seeck, and R. Auerswald. "The Principles of Operation Framework: A Comprehensive Classification Concept for Automated Driving Functions". In: *SAE International Journal of Connected and Automated Vehicles* 03.1 (Feb. 18, 2020), pp. 27–37. ISSN: 2574-0741, 2574-075X. DOI: 10.4271/12-03-01-0003. URL: <https://saemobilus.sae.org/articles/principles-operation-framework-a-comprehensive-classification-concept-automated-driving-functions-12-03-01-0003> (visited on 04/23/2026).
- [31] J. M. Beer, A. D. Fisk, and W. A. Rogers. "Toward a Framework for Levels of Robot Autonomy in Human-Robot Interaction". In: *Journal of Human-Robot Interaction* 3.2 (June 1, 2014), p. 74. ISSN: 2163-0364. DOI: 10.5898/JHRI.3.2.Beer. URL: <http://dl.acm.org/citation.cfm?id=3109833> (visited on 10/31/2025).
- [32] M. Zaffar, S. Ehsan, R. Stolkin, and K. M. Maier. "Sensors, SLAM and Long-term Autonomy: A Review". In: *2018 NASA/ESA Conference on Adaptive Hardware and Systems (AHS)*. 2018 NASA/ESA Conference on Adaptive Hardware and Systems (AHS). Edinburgh: IEEE, Aug. 2018, pp. 285–290. ISBN: 978-1-5386-7753-7. DOI: 10.1109/AHS.2018.8541483. URL: <https://ieeexplore.ieee.org/document/8541483/> (visited on 10/31/2025).
- [33] D. Ha and J. Schmidhuber. "Recurrent World Models Facilitate Policy Evolution". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 31. 2018. arXiv: 1809.01999.
- [34] Y. LeCun. "A Path Towards Autonomous Machine Intelligence". Version 0.9.2. In: *OpenReview* (2022). URL: <https://openreview.net/forum?id=BZ5a1r-kVsf>.
- [35] T. Estlin, R. Volpe, I. Nesnas, D. Mutz, F. Fisher, B. Engelhardt, and S. Chien. "Decision-Making in a Robotic Architecture for Autonomy". In: ().
- [36] R. A. Brooks. *A Robust Layered Control System for a Mobile Robot*. A. I. Memo 864. Cambridge, MA: Massachusetts Institute of Technology, Artificial Intelligence Laboratory, Sept. 1985.
- [37] G. Konidaris, L. P. Kaelbling, and T. Lozano-Perez. "Symbol Acquisition for Task-Level Planning". In: ().
- [38] E. Gat. "On Three-Layer Architectures". In: ().

- [39] C. M. Clark, S. M. Rock, and J.-C. Latombe. “Dynamic Robot Networks: A Coordination Platform for Multi-Robot Systems”. In: ().
- [40] X. Zhang, G. Li, S. Shokouhi, and M.-W. L. Thein. “Modern Control Meets Machine Learning: A Review and Taxonomy of Synergistic Approaches for Robotics Applications”. In: *Actuators* 15.5 (Apr. 27, 2026), p. 235. issn: 2076-0825. doi: 10.3390/act15050235. URL: <https://www.mdpi.com/2076-0825/15/5/235> (visited on 05/02/2026).
- [41] N. J. Nilsson. *Shakey the Robot*. Technical Note 323. Menlo Park, CA: SRI International, Artificial Intelligence Center, Apr. 1984. URL: <https://ai.stanford.edu/~nilsson/OnlinePubs-Nils/shakey-the-robot.pdf>.
- [42] H. P. MORAVEE. “The Stanford Cart and the CMU Rover”. In: ().
- [43] R. C. Arkin. “Motor Schema — Based Mobile Robot Navigation”. In: *The International Journal of Robotics Research* 8.4 (Aug. 1989), pp. 92–112. issn: 0278-3649, 1741-3176. doi: 10.1177/027836498900800406. URL: <https://journals.sagepub.com/doi/10.1177/027836498900800406> (visited on 11/30/2025).
- [44] F. Martin, J. G. Clavero, V. Matellan, and F. J. Rodriguez. “PlanSys2: A Planning System Framework for ROS2”. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Prague, Czech Republic: IEEE, Sept. 27, 2021, pp. 9742–9749. ISBN: 978-1-6654-1714-3. doi: 10.1109/IROS51168.2021.9636544. URL: <https://ieeexplore.ieee.org/document/9636544/> (visited on 04/26/2026).
- [45] D. Harel. “Statecharts: A Visual Formalism for Complex Systems”. In: *Science of Computer Programming* 8.3 (June 1987), pp. 231–274. issn: 01676423. doi: 10.1016/0167-6423(87)90035-9. URL: <https://linkinghub.elsevier.com/retrieve/pii/0167642387900359> (visited on 11/30/2025).
- [46] M. Colledanchise and P. Ögren. *Behavior Trees in Robotics and AI: An Introduction*. July 20, 2018. doi: 10.1201/9780429489105. arXiv: 1709.00084 [cs]. URL: <http://arxiv.org/abs/1709.00084> (visited on 11/30/2025).
- [47] K. French, S. Wu, T. Pan, Z. Zhou, and O. C. Jenkins. “Learning Behavior Trees From Demonstration”. In: *2019 International Conference on Robotics and Automation (ICRA)*. 2019 International Conference on Robotics and Automation (ICRA). Montreal, QC, Canada: IEEE, May 2019, pp. 7791–7797. ISBN: 978-1-5386-6027-0. doi: 10.1109/ICRA.2019.8794104. URL: <https://ieeexplore.ieee.org/document/8794104/> (visited on 05/03/2026).
- [48] A. Psiris, V. Argyriou, E. K. Markakis, P. Sarigiannidis, E. Gavves, K. Bekris, and A. A. a. G. T. Papadopoulos. *Foundation Models in Robotics: A Comprehensive Review of Methods, Models, Datasets, Challenges and Future Research Directions*. Apr. 16, 2026. doi: 10.48550/arXiv.2604.15395. arXiv: 2604.15395 [cs]. URL: <http://arxiv.org/abs/2604.15395> (visited on 05/03/2026). Pre-published.

- [49] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. *OpenVLA: An Open-Source Vision-Language-Action Model*. Sept. 5, 2024. doi: 10.48550/arXiv.2406.09246. arXiv: 2406.09246 [cs]. URL: <http://arxiv.org/abs/2406.09246> (visited on 05/03/2026). Pre-published.
- [50] T. Haarnoja, B. Moran, G. Lever, S. H. Huang, D. Tirumala, J. Humplik, M. Wulfmeier, S. Tunyasuvunakool, N. Y. Siegel, R. Hafner, M. Bloesch, K. Hartikainen, A. Byravan, L. Hasenclever, Y. Tassa, F. Sadeghi, N. Batchelor, F. Casarini, S. Saliceti, C. Game, N. Sreendra, K. Patel, M. Gwira, A. Huber, N. Hurley, F. Nori, R. Hadsell, and N. Heess. “Learning Agile Soccer Skills for a Bipedal Robot with Deep Reinforcement Learning”. In: *Science Robotics* 9.89 (Apr. 10, 2024), eadi8022. issn: 2470-9476. doi: 10.1126/scirobotics.adi8022. URL: <https://www.science.org/doi/10.1126/scirobotics.adi8022> (visited on 05/03/2026).
- [51] M. Grieves and J. Vickers. “Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems”. In: *Transdisciplinary Perspectives on Complex Systems*. Ed. by F.-J. Kahlen, S. Flumerfelt, and A. Alves. Cham: Springer International Publishing, 2017, pp. 85–113. isbn: 978-3-319-38754-3. doi: 10.1007/978-3-319-38756-7_4.
- [52] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn. “Digital Twin in Manufacturing: A Categorical Literature Review and Classification”. In: *IFAC-PapersOnLine* 51.11 (2018), pp. 1016–1022. issn: 2405-8963. doi: 10.1016/j.ifacol.2018.08.474.
- [53] H. Wu, P. Ji, H. Ma, and L. Xing. “A Comprehensive Review of Digital Twin from the Perspective of Total Process: Data, Models, Networks and Applications”. In: *Sensors* 23.19 (Oct. 8, 2023), p. 8306. issn: 1424-8220. doi: 10.3390/s23198306. URL: <https://www.mdpi.com/1424-8220/23/19/8306>.
- [54] Z. Lv, H. H. Song, J. Shen, and N. Vaughan. “Editorial: Digital Twins of Plant and Forest”. In: *Frontiers in Plant Science* 13 (Dec. 23, 2022), p. 1049240. issn: 1664-462X. doi: 10.3389/fpls.2022.1049240. URL: <https://www.frontiersin.org/articles/10.3389/fpls.2022.1049240/full> (visited on 05/16/2026).
- [55] A. C. Tagarakis, L. Benos, G. Kyriakarakos, S. Pearson, C. G. Sørensen, and D. Bochtis. “Digital Twins in Agriculture and Forestry: A Review”. In: *Sensors* 24.10 (May 14, 2024), p. 3117. issn: 1424-8220. doi: 10.3390/s24103117. URL: <https://www.mdpi.com/1424-8220/24/10/3117> (visited on 05/16/2026).
- [56] A. J. W. de Wit, H. L. Boogaard, I. Supit, and M. van den Berg, eds. *System Description of the WOFOST 7.2 Cropping Systems Model*. Wageningen: Wageningen Environmental Research, 2020.
- [57] I. Athanasiadis, F. Alidoost, F. Nattino, S. Ciarella, M. Kallenberg, R. van Bree, and A. de Wit. *diffwofst: A Differentiable Implementation of the WOFOST Crop Growth Model*. Version 0.5.0. PyTorch-based differentiable crop growth model. Wageningen University

- & Research and Netherlands eScience Center, May 29, 2026. URL: <https://github.com/WUR-AI/diffwofost>.
- [58] INRAE. *STICS: A Generic Soil–Crop Simulation Model*. Soil–crop simulation model maintained by INRAE. 2024. URL: <https://stics.inrae.fr/eng/> (visited on 06/13/2026).
 - [59] INRAE. *Hi-sAFe: A Process-Based Agroforestry Model*. Open-source agroforestry simulation model. 2024. URL: <https://github.com/hisafe/hisafe>.
 - [60] INRAE. *Hi-sAFe: Presentation Generale*. INRAE. URL: <https://www1.montpellier.inra.fr/wp-inra/hi-safe/presentation-generale/> (visited on 06/16/2026).
 - [61] H. Liu, H. Darabi, P. Banerjee, and J. Liu. “Survey of Wireless Indoor Positioning Techniques and Systems”. In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 37.6 (Nov. 2007), pp. 1067–1080. DOI: 10.1109/TSMCC.2007.905750.
 - [62] R. Zekavat and R. M. Buehrer, eds. *Handbook of Position Location: Theory, Practice, and Advances*. 2nd ed. Wiley-IEEE Press, 2019. ISBN: 978-1-119-43458-0.
 - [63] F. Zafari, A. Gkelias, and K. K. Leung. “A Survey of Indoor Localization Systems and Technologies”. In: *IEEE Communications Surveys & Tutorials* 21.3 (2019), pp. 2568–2599. DOI: 10.1109/COMST.2019.2911558.
 - [64] M. A. Fischler and R. C. Bolles. “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395.
 - [65] P. Besl and N. D. McKay. “A Method for Registration of 3-D Shapes”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (Feb. 1992), pp. 239–256. ISSN: 0162-8828, 2160-9292. DOI: 10.1109/34.121791. URL: <http://ieeexplore.ieee.org/document/121791/> (visited on 11/08/2025).
 - [66] Y. Chen and G. Medioni. “Object modelling by registration of multiple range images”. In: *Image and vision computing* 10.3 (1992), pp. 145–155.
 - [67] P. Biber and W. Straßer. “The normal distributions transform: A new approach to laser scan matching”. In: *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003)*. Vol. 3. IEEE. 2003, pp. 2743–2748.
 - [68] M. Magnusson, A. Lilienthal, and A.-J. Lu. “The three-dimensional normal distributions transform—an efficient approach for registration of 3D range data”. In: *International Journal of Robotics Research* 26.9 (2007), pp. 911–927.
 - [69] R. B. Rusu, N. Blodow, and M. Beetz. “Fast Point Feature Histograms (FPFH) for 3D Registration”. In: *2009 IEEE International Conference on Robotics and Automation*. 2009 IEEE International Conference on Robotics and Automation (ICRA). Kobe: IEEE, May 2009, pp. 3212–3217. ISBN: 978-1-4244-2788-8. DOI: 10.1109/ROBOT.2009.5152473. URL: <http://ieeexplore.ieee.org/document/5152473/> (visited on 11/08/2025).

- [70] T. Krajník, P. Cristóforis, K. Kusumam, P. Neubert, and T. Duckett. “Image Features for Visual Teach-and-Repeat Navigation in Changing Environments”. In: *Robotics and Autonomous Systems* 88 (Feb. 2017), pp. 127–141. ISSN: 09218890. DOI: 10.1016/j.robot.2016.11.011. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0921889016300574> (visited on 11/01/2025).
- [71] S. M. Abbas, S. Aslam, K. Berns, and A. Muhammad. “Analysis and Improvements in AprilTag Based State Estimation”. In: *Sensors* 19.24 (Dec. 12, 2019), p. 5480. ISSN: 1424-8220. DOI: 10.3390/s19245480. URL: <https://www.mdpi.com/1424-8220/19/24/5480> (visited on 11/20/2025).
- [72] T. Moore and D. Stouch. “A Generalized Extended Kalman Filter Implementation for the Robot Operating System”. In: *Intelligent Autonomous Systems 13*. Ed. by E. Menegatti, N. Michael, K. Berns, and H. Yamaguchi. Vol. 302. Cham: Springer International Publishing, 2016, pp. 335–348. ISBN: 978-3-319-08337-7 978-3-319-08338-4. DOI: 10.1007/978-3-319-08338-4_25. URL: https://link.springer.com/10.1007/978-3-319-08338-4_25 (visited on 10/31/2025).
- [73] Intel Corporation. *Intel RealSense D400 Series Product Family Datasheet*. Revision 005, Document Number: 337029-005. Jan. 2019. URL: <https://www.intelrealsense.com/wp-content/uploads/2019/01/Intel-RealSense-D400-Series-Datasheet.pdf>.
- [74] R. B. Rusu and S. Cousins. “3D Is Here: Point Cloud Library (PCL)”. In: *2011 IEEE International Conference on Robotics and Automation*. 2011 IEEE International Conference on Robotics and Automation (ICRA). Shanghai, China: IEEE, May 2011, pp. 1–4. ISBN: 978-1-61284-386-5 978-1-61284-385-8. DOI: 10.1109/ICRA.2011.5980567. URL: <https://ieeexplore.ieee.org/document/5980567/> (visited on 11/08/2025).
- [75] Y.-L. Zhao, Y.-T. Hong, and H.-P. Huang. “Comprehensive Performance Evaluation between Visual SLAM and LiDAR SLAM for Mobile Robots: Theories and Experiments”. In: *Applied Sciences* 14.9 (May 6, 2024), p. 3945. ISSN: 2076-3417. DOI: 10.3390/app14093945. URL: <https://www.mdpi.com/2076-3417/14/9/3945> (visited on 10/31/2025).
- [76] C. D. Singh, B. He, C. Fermüller, C. Metzler, and Y. Aloimonos. “Minimal Perception: Enabling Autonomy in Resource-Constrained Robots”. In: *Frontiers in Robotics and AI* 11 (Sept. 18, 2024), p. 1431826. ISSN: 2296-9144. DOI: 10.3389/frobt.2024.1431826. URL: <https://www.frontiersin.org/articles/10.3389/frobt.2024.1431826/full> (visited on 10/31/2025).
- [77] Y. Gutnik and M. Groper. “Terminal Phase Navigation for AUV Docking: An Innovative Electromagnetic Approach”. In: *Journal of Marine Science and Engineering* 12.1 (Jan. 21, 2024), p. 192. ISSN: 2077-1312. DOI: 10.3390/jmse12010192. URL: <https://www.mdpi.com/2077-1312/12/1/192> (visited on 03/29/2026).
- [78] D. Teso-Fz-Betoño, I. Aramendia, J. A. Ramos-Hernanz, D. Caballero-Martin, H. Affou, and J. M. Lopez-Guede. “Optimization of Energy Efficiency with a Predictive Dynamic Window Approach for Mobile Robot Navigation”. In: *Sustainability* 17.10 (May 15, 2025),

- p. 4526. ISSN: 2071-1050. DOI: 10.3390/su17104526. URL: <https://www.mdpi.com/2071-1050/17/10/4526> (visited on 04/23/2026).
- [79] Saildrone. *Saildrone: Wind- and Solar-Powered Autonomous Surface Vehicles*. Saildrone, Inc. URL: <https://www.saildrone.com/> (visited on 06/16/2026).
- [80] C. L. Gentemann, J. P. Scott, P. L. F. Mazzini, C. Pianca, S. Akella, P. J. Minnett, P. Cornillon, B. Fox-Kemper, I. Cetinić, T. M. Chin, J. Gomez-Valdes, J. Vazquez-Cuervo, V. Tsontos, L. Yu, R. Jenkins, S. De Halleux, D. Peacock, and N. Cohen. "Saildrone: Adaptively Sampling the Marine Environment". In: *Bulletin of the American Meteorological Society* 101.6 (2020), E744–E762. DOI: 10.1175/BAMS-D-19-0015.1.
- [81] R. L. Cataldo and G. L. Bennett. "U.S. Space Radioisotope Power Systems and Applications: Past, Present and Future". In: *Radioisotopes: Applications in Physical Sciences*. Ed. by N. Singh. NASA Technical Reports Server, document 20120000731. InTech, 2011. DOI: 10.5772/23914.
- [82] A. Thelen, X. Huan, N. Paulson, S. Onori, Z. Hu, and C. Hu. "Probabilistic Machine Learning for Battery Health Diagnostics and Prognostics—Review and Perspectives". In: *npj Materials Sustainability* 2.1 (June 3, 2024), p. 14. DOI: 10.1038/s44296-024-00011-1.
- [83] M. Visinsky, J. Cavallaro, and I. Walker. "Robotic Fault Detection and Fault Tolerance: A Survey". In: *Reliability Engineering & System Safety* 46.2 (Jan. 1994), pp. 139–158. ISSN: 09518320. DOI: 10.1016/0951-8320(94)90132-5. URL: <https://linkinghub.elsevier.com/retrieve/pii/0951832094901325> (visited on 10/31/2025).
- [84] A. L. Christensen. "Fault Detection in Autonomous Robots". In: ().
- [85] Muhamad Azhar A. Alobaidy, Dr. Jassim M. Abdul-Jabbar, and Saad Zaghlul Al-khayyt. "Faults Diagnosis in Robot Systems: A Review". In: *Al-Rafidain Engineering Journal (AREJ)* 25.2 (Dec. 1, 2020), pp. 164–175. ISSN: 2220-1270, 1813-0526. DOI: 10.33899/rengj.2020.127782.1051. URL: <https://jamh.uomosul.edu.iq/index.php/arej/article/view/31115> (visited on 10/31/2025).
- [86] W. Hijikata, T. Ohori, X. Li, H. Nakanishi, and S. Ozawa. *Wireless Power Transfer System for Mobile Robots via Magnetic Resonant Coupling with Impedance Matching*. Jan. 5, 2022. DOI: 10.21203/rs.3.rs-1229883/v1.
- [87] H. Kabir, M.-L. Tham, and Y. C. Chang. "Internet of Robotic Things for Mobile Robots: Concepts, Technologies, Challenges, Applications, and Future Directions". In: *Digital Communications and Networks* 9.6 (Dec. 2023), pp. 1265–1290. ISSN: 23528648. DOI: 10.1016/j.dcan.2023.05.006. URL: <https://linkinghub.elsevier.com/retrieve/pii/S2352864823000883> (visited on 11/04/2025).
- [88] S. Macenski, F. Martín, R. White, and J. G. Clavero. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 24, 2020, pp. 2718–2725. DOI: 10.1109/IROS45743.2020.9341207. arXiv: 2003.00368 [cs]. URL: <http://arxiv.org/abs/2003.00368> (visited on 04/26/2026).

- [89] Rosenberger Hochfrequenztechnik. *MultiMag 15 Magnetic Connector – Technical Data Sheet*. Part M9K102-199L. Rosenberger Hochfrequenztechnik GmbH & Co. KG. URL: <https://www.rosenberger.com/>.
- [90] ODU. *ODU-DOCK Silver Line Magnetic Docking Connector*. ODU GmbH & Co. KG. URL: <https://odu-connectors.com/products/modular-connectors/odu-dock-silver-line/> (visited on 06/13/2026).
- [91] Harwin. *Spring-Loaded Contacts (Pogo Pins) and Contact Pads – Datasheet*. Includes technical drawing of part P70-220. Harwin Plc. URL: <https://www.harwin.com/products/spring-contacts/>.
- [92] Wiferion. *etaLINK 3000 Wireless Inductive Charging System – Product Flyer*. Wiferion GmbH (Conductix-Wampfler). URL: <https://www.wiferion.com/>.
- [93] WiBotic. *WiBotic Mobile Power Solutions: OC-301 and TR-302 Product Datasheet*. WiBotic Inc. June 2025. URL: <https://www.wibotic.com/>.
- [94] WiBotic. *WiBotic Product Information Brief*. WiBotic Inc. URL: <https://www.wibotic.com/>.
- [95] Opensignal. *Germany Mobile Network Experience Report*. Nov. 2025. URL: <https://www.opensignal.com/reports/2025/11/germany/mobile-network-experience> (visited on 06/14/2026).
- [96] Y. Liu and S. Choi. “A First Look at Wi-Fi 6 in Action: Throughput, Latency, Energy Efficiency, and Security”. In: *Proceedings of the International Conference on Mobile Computing and Networking*. 2023. DOI: 10.1145/3570361.3592534.
- [97] IEEE Computer Society. *IEEE 802.3ab: Physical Layer Parameters and Specifications for 1000 Mb/s Operation over 4-Pair Category 5 Balanced Copper Cabling, Type 1000BASE-T*. 1999. DOI: 10.1109/IEEESTD.1999.90566.
- [98] Ookla. *Starlink Performance Report: Q4 2025*. Dec. 2025. URL: <https://www.speedtest.net/global-index/starlink> (visited on 06/14/2026).
- [99] TP-Link. *Omada Business Wi-Fi Series: EAP225-Outdoor Product Datasheet*. TP-Link Technologies Co., Ltd. URL: <https://www.tp-link.com/>.
- [100] Holybro. *Holybro Pixhawk Jetson Baseboard: PX4 Guide*. 2026. URL: https://docs.px4.io/main/en/companion_computer/holybro_pixhawk_jetson_baseboard.
- [101] Holybro. *UBEC 12A (3-14S) Step-Down Regulator Overview*. Holybro.
- [102] Holybro. *PWM Extension Board for Pixhawk Autopilot Carrier Boards*. Holybro.
- [103] Premium-Modellbau. *Holybro Pixhawk 6X Pro mit Jetson Orin und Baseboard Komplettsset*. Premium-Modellbau GmbH. URL: <https://www.premium-modellbau.de/holybro-pixhawk-6x-pro-mit-jetson-orin-und-baseboard-komplettsset-20280> (visited on 06/13/2026).
- [104] u-blox AG. *ZED-F9P-04B High Precision GNSS Module Data Sheet*. Revision R05, Document Number: UBX-21044850. Mar. 2024. URL: <https://www.u-blox.com/en/product/zed-f9p-module>.

- [105] ArduSimple. *Medium Range Radio - RTK Starter Kit (SKU: AS-STARTKIT-MR-L1L2-NH-01) Datasheet*. Apr. 2026. URL: <https://www.ardusimple.com/>.
- [106] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (2023). doi: 10.1145/3592433.
- [107] C. Qian, W. Li, Y. Guo, and G. Markkula. *WeatherEdit: Controllable Weather Editing with 4D Gaussian Field*. 2025. arXiv: 2505.20471.
- [108] V. Kilic, D. Hegde, V. Sindagi, A. B. Cooper, M. A. Foster, and V. M. Patel. *Lidar Light Scattering Augmentation (LISA): Physics-based Simulation of Adverse Weather Conditions for 3D Object Detection*. 2021. arXiv: 2107.07004.
- [109] M. Hahner, C. Sakaridis, D. Dai, and L. Van Gool. "Fog Simulation on Real LiDAR Point Clouds for 3D Object Detection in Adverse Weather". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021.
- [110] M. Hahner, C. Sakaridis, M. Bijelic, F. Heide, F. Yu, D. Dai, and L. Van Gool. "LiDAR Snowfall Simulation for Robust 3D Object Detection". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.
- [111] M. Kutila, P. Pykönen, H. Holzhüter, M. Colomb, and P. Duthon. "Automotive LiDAR Performance Verification in Fog and Rain". In: *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. 2018, pp. 1695–1701. doi: 10.1109/ITSC.2018.8569624.
- [112] R. Heinzler, P. Schindler, J. Seekircher, W. Ritter, and W. Stork. "Weather Influence and Classification with Automotive Lidar Sensors". In: *2019 IEEE Intelligent Vehicles Symposium (IV)*. 2019, pp. 1527–1534. doi: 10.1109/IVS.2019.8814205. arXiv: 1906.07675.
- [113] M. Jokela, M. Kutila, and P. Pykönen. "Testing and Validation of Automotive Point-Cloud Sensors in Adverse Weather Conditions". In: *Applied Sciences* 9.11 (2019), p. 2341. doi: 10.3390/app9112341.
- [114] T. G. Phillips, N. Guenther, and P. R. McAree. "When the Dust Settles: The Four Behaviors of LiDAR in the Presence of Fine Airborne Particulates". In: *Journal of Field Robotics* 34.5 (2017), pp. 985–1009. doi: 10.1002/rob.21701.
- [115] M. Bijelic, T. Gruber, F. Mannan, F. Kraus, W. Ritter, K. Dietmayer, and F. Heide. "Seeing Through Fog Without Seeing Fog: Deep Multimodal Sensor Fusion in Unseen Adverse Weather". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 11682–11692. doi: 10.1109/CVPR42600.2020.01170. arXiv: 1902.08913.
- [116] Y. Zhang, A. Carballo, H. Yang, and K. Takeda. "Perception and Sensing for Autonomous Vehicles under Adverse Weather Conditions: A Survey". In: *ISPRS Journal of Photogrammetry and Remote Sensing* 196 (2023), pp. 146–177. doi: 10.1016/j.isprsjprs.2022.12.021.

- [117] Conductix-Wampfler. *WirelessCharger 3.0: Program 9200 Product Datasheet*. Document: KAT9200-0002-EN. Conductix-Wampfler. 2024. URL: <https://www.conductix.com/>.
- [118] R. Herry. *Wireless Charger 3.0: Quick Start Guide*. Version V_RH2407. Conductix-Wampfler. July 2024. URL: <https://www.conductix.com/>.